

第一章 项目环境搭建

创建小程序

- 项目名称 `mxg-order-miniapp`
- 目录选择，注意：文件名也要跟到最后；目录名不要有中文、空格和特殊字符；
- 后端服务选择 不使用云服务
- 模板选择 JS-基础模板



确认项目版本号：3.6.0

The screenshot shows the Mengxuegu IDE interface. On the left, the `project.config.json` file is open, showing configuration for a mini-program. A red box highlights the `"libVersion": "3.6.0"` property. On the right, the '本地设置' (Local Settings) panel is open, showing the '调试基础库' (Debug Base Library) section. A red box highlights the '3.6.0' version and the '推送' (Push) button. The '性能质量' (Performance Quality) section shows options like '将JS编译成ES5' (Compile JS to ES5) and '使用SWC编译脚本文件' (Use SWC to compile script files), both of which are checked.

项目结构调整

项目采用云开发，为了项目结构更加符合官方云开发标准，我们对项目结构做如下调整：

1. 修改项目配置文件 `project.config.json`，并在项目根目录下创建 `cloudfunctions` 和 `miniprogram` 文件夹

注意：

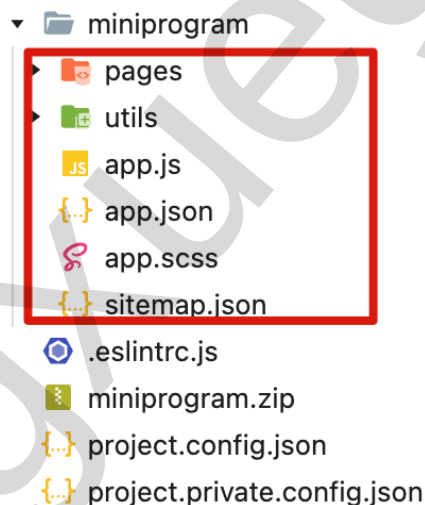
- 修改后一定要记得 `Ctrl + S` 保存才会生效
- json 文件中不能写注释 `//`，要把注释删除掉

The screenshot shows the IDE interface with the project structure on the left and the `project.config.json` file open on the right. The project structure shows the `cloudfunctions` and `miniprogram` folders created. The `project.config.json` file is updated with the following configuration:

```
{
  "description": "项目配置文件",
  "miniprogramRoot": "miniprogram/",
  "srcMiniprogramRoot": "miniprogram/",
  "cloudfunctionRoot": "cloudfunctions/",
  "compileType": "miniprogram",
  "libVersion": "trial",
  "packOptions": {
    "ignore": [],
    "include": []
  },
  "setting": {
    "useCompilerPlugins": [
      "sass"
    ],
    "coverView": true,
    "es6": true,
    "postcss": true
  }
}
```

```
1 {
2   "description": "项目配置文件",
3   "miniprogramRoot": "miniprogram/", // 指定小程序源码的目录(需为相对路径)
4   "srcMiniprogramRoot": "miniprogram/", // 保存后自动添加,也可手动添加与
    miniprogramRoot 相同的值
5   "cloudfunctionRoot": "cloudfunctions/", // 云函数代码根目录(需为相对路径)
6   "compileType": "miniprogram",
7   "libVersion": "trial",
8   "packOptions": {
9     "ignore": [],
10    "include": []
11  },
12  "setting": {
13    "useCompilerPlugins": [ // 编译插件配置,目前支持编译插件有 typescript、less、
    sass
14      "sass" // 对sass的支持
15  ],
16  ....省略
```

2. 将 `pages`、`utils`、`app.js`、`app.json`、`app.wxss`、`sitemap.json` 移动到新建的 `miniprogram` 文件夹下



3. 将 `app.wxss` 重命名为 `app.scss`

添加全局资源

1. 将课程资源中的 `03-配套资料/utils` 和 `03-配套资料/assets` 文件夹拷贝到项目的 `miniprogram` 文件夹下

< > 03-配套资料

返回/前进

名称

- > 模板代码
- > 云开发
- > assets
- > utils

2. 在 `app.scss` 中引入字体图标

```
1  /**app.scss**/  
2  
3  /* 导入字体图标 */  
4  @import './assets/icon/iconfont.scss';  
5  
6  /* 导入自定义全局样式 */  
7  @import './assets/style/index.scss';
```

app.json配置 tabBar

1. 在 app.json 文件中配置 pages、tabBar、window、resolveAlias

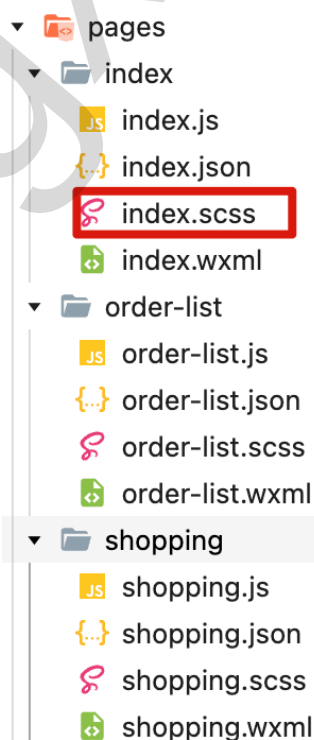
```
1  {  
2    "pages": [  
3      "pages/index/index",  
4      "pages/shopping/shopping",  
5      "pages/order-list/order-list"  
6    ],  
7    "tabBar": {  
8      "color": "#8a8a8a",  
9      "selectedColor": "#e40030",  
10     "borderStyle": "white",  
11     "list": [  
12       {  
13         "pagePath": "pages/index/index",  
14         "text": "首页",  
15         "iconPath": "assets/tab/index.png",  
16         "selectedIconPath": "assets/tab/index-active.png"  
17       },  
18       {  
19         "pagePath": "pages/shopping/shopping",  
20         "text": "点单",  
21         "iconPath": "assets/tab/shopping.png",
```



```
22     "selectedIconPath": "assets/tab/shopping-active.png"
23   },
24   {
25     "pagePath": "pages/order-list/order-list",
26     "text": "订单",
27     "iconPath": "assets/tab/order.png",
28     "selectedIconPath": "assets/tab/order-active.png"
29   }
30 ]
31 },
32 "window": {
33   "navigationBarTextStyle": "black",
34   "navigationBarBackgroundColor": "#ffffff",
35   "navigationBarTitleText": "点单小程序",
36   "backgroundColor": "#f9f9f9"
37 },
38 "resolveAlias": {
39   "@/*": "/*"
40 },
41 "style": "v2",
42 "componentFramework": "glass-easel",
43 "sitemapLocation": "sitemap.json",
44 "lazyCodeLoading": "requiredComponents"
45 }
46 }
```

2. 删除 pages 文件夹下的 logs 目录，删除后的效果如下：

注意：查看 index 页面样式文件是否为 index.scss，如果不是则重命名为此类型文件



整合第3方组件库 Vant Weapp

Vant Weapp 官网: <https://vant.pro/vant-weapp>

Vant Weapp 是一个轻量、可靠的小程序 UI 组件库。

整合步骤

步骤1: 通过 npm 安装

使用 npm 构建前, 可先阅读微信官方的 [npm 支持](#)

```
1 # cd 进入 miniprogram 项目根目录
2 cd miniprogram
3
4 # 通过 npm 安装 Vant Weapp
5 npm i @vant/weapp@1.11.6 -S --production
```

构建 调试器 问题 输出 终端 代码质量

```
mengxuegu@mengxuegu mxq-order-miniapp % cd miniprogram
mengxuegu@mengxuegu miniprogram % npm i @vant/weapp -S --production
npm WARN config production Use `--omit=dev` instead.

added 1 package in 785ms
mengxuegu@mengxuegu miniprogram %
```

执行上面命令后, 项目结构如下:

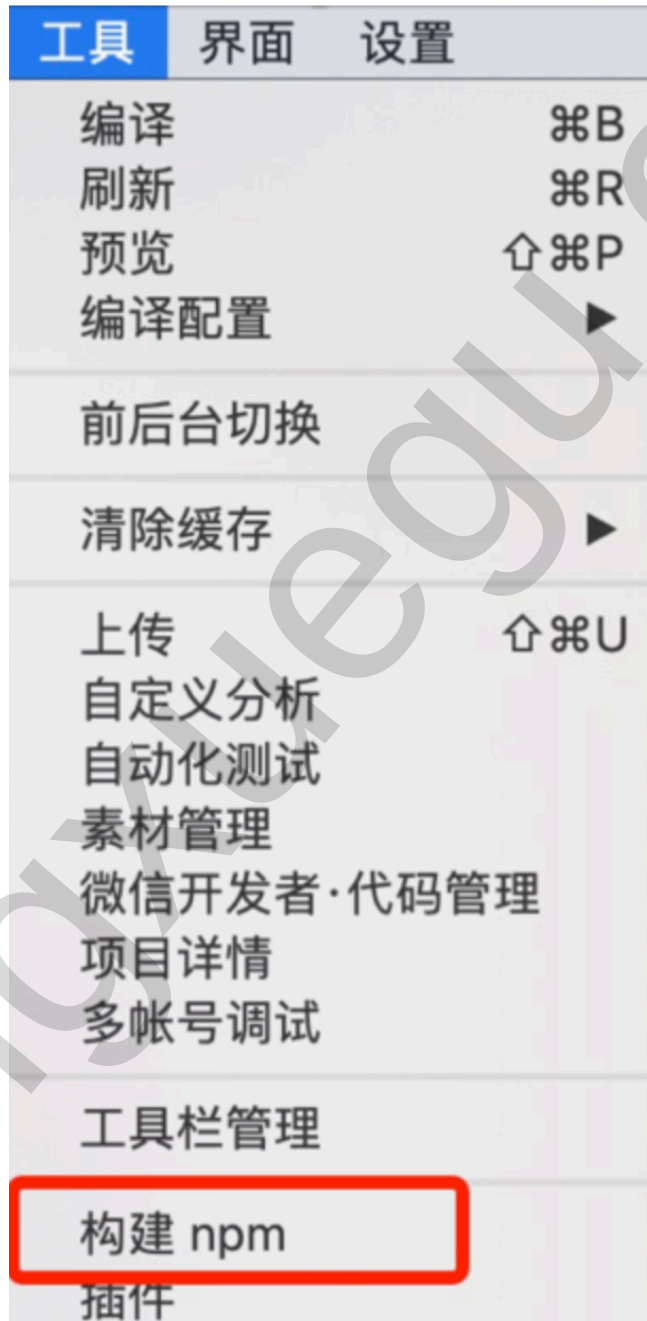
- miniprogram
 - assets
 - components
 - node_modules
 - pages
 - utils
 - app.js
 - app.json
 - app.scss
 - package-lock.json
 - package.json
 - sitemap.json
 - .eslintrc.js
 - project.config.json
 - project.private.config.json

步骤2：修改 app.json

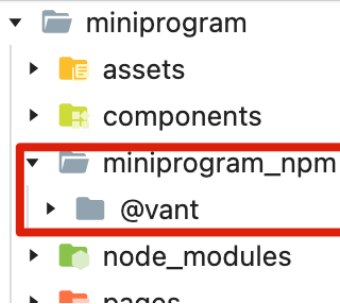
将 app.json 中的 "style": "v2" 去除，小程序的**新版基础组件**强行加上了许多样式，难以覆盖，不关闭将造成部分组件样式混乱。

步骤3：构建 npm 包

- 打开微信开发者工具，点击 工具 -> 构建 npm



- 构建成功后，在 miniprogram 下多新增一个 **miniprogram_npm** 目录存放了相关依赖的文件：



使用 Vant Weapp 组件

1. 先引入需要使用的组件

以 Button 组件为例，只需要在全局配置文件 `app.json` 或 页面配置文件 `index.json` 中配置 Button 对应的路径即可。

```
1 // app.json
2 "usingComponents": {
3   "van-button": "@vant/weapp/button/index"
4 }
```

2. 使用组件

引入组件后，可以在 `.wxml` 文件中直接使用组件

```
1 <van-button type="primary">按钮</van-button>
```

3. 缓存全部清除，再编译



第二章 首页开发

首页功能有：无导航栏(设置为自定义导航栏custom)、头部轮播图、登录用户信息区域、快捷菜单区域、热门推荐区域



首页轮播图功能开发



自定义轮播图组件

1. 创建轮播图自定义组件 `components/banner/index`
2. 轮播图模板代码实现：

```
1 <!--components/banner/index.wxml-->
2
3 <!-- 轮播图 声明css变量 --swiper-height -->
4 <swiper class="swiper-banner"
5   style="--swiper-height:{{height}};--dots-bottom:{{dotsBottom}}"
6   autoplay
7   circular
8   indicator-dots
9   indicator-color="#e7e7e7"
10  indicator-active-color="#FFF"
11  interval="5000"
12 >
13 <swiper-item wx:for="{{ list }}" wx:key="index">
14   <!-- 链接跳转 -->
15   <navigator class="nav" hover-class="none">
16     <image class="swiper-img" src="/assets/images/banner{{index}}.png" />
17   </navigator>
18 </swiper-item>
19 </swiper>
```

3. 组件样式

```
1 /* components/banner/index.wxss */
2
3 /* 轮播图, */
```

```
4 .swiper-banner {
5   // --swiper-height引用的是swiper-item中声明的css变量
6   height: var(--swiper-height, 500rpx);
7   width: 100%;
8   .nav, .swiper-img {
9     width: 100%;
10    height: 100%;
11  }
12  // 轮播图指示点样式, 要在组件 index.js 中添加 options: { styleIsolation: "shared"
13  }
14  .wx-swiper-dots {
15    left: auto !important;
16    right: -20rpx !important;
17    bottom: var(--dots-bottom, 10rpx) !important;
18  }
19  // 选中的指定点样式
20  .wx-swiper-dot-active {
21    width: 23px !important;
22    border-radius: 6px !important;
23  }
24 }
```

4. JS 逻辑代码:

- styleIsolation 指定样式隔离级别;
- properties 接收父组件传递的数据

```
1 // components/banner/index.js
2 Component({
3
4   options: {
5     // 样式隔离组件为"shared", 才可以在组件index.scss中修改轮播指示点样式
6     styleIsolation: "shared"
7   },
8
9   properties: {
10    list: { // 轮播数据
11      type: Array,
12      value: []
13    },
14    height: { // 整体高度
15      type: String,
16      value: '500rpx'
17    },
18    dotsBottom: { // 指示点到底部距离
19      type: String,
20      value: '18rpx'
21    }
22  }
23 })
```



```
22     },  
23  
24     })
```

首页引用轮播图组件

1. 在 pages/index/index.json 页面配置中

- 指定 `navigationStyle: custom` 自定义导航栏（隐藏导航）
- `usingComponents` 中注册轮播图组件 `banner`

```
1  {  
2    "navigationStyle": "custom",  
3    "usingComponents": {  
4      "banner": "/components/banner/index"  
5    }  
6  }
```

2. 在 pages/index/index.wxml 页面模板中使用 banner 组件

```
1  <view>  
2    <!-- 头部轮播图 -->  
3    <banner list="{{ bannerList }}" height="550rpx" dots-bottom="90rpx"/>  
4  </view>
```

3. 在 pages/index/index.js 文件的 data 选项中声明 `bannerList` 状态属性

```
1  // pages/index/index.js  
2  
3  Page({  
4  
5    data: {  
6      bannerList: [1, 2, 3, 4], // 头部轮播图数据  
7    },  
8  
9  })
```

真机调试问题

问题：当组件js中配置了 `styleIsolation: "shared"` 后，如下图：使用真机调试 > 二维码真机调试（1.0），扫码后真机打开的页面可能会是空白的。

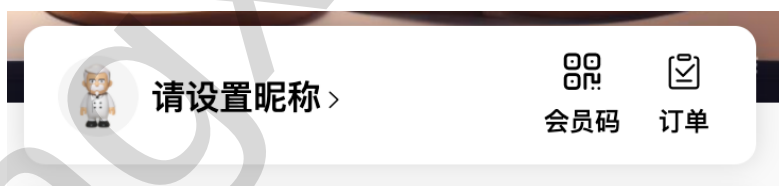
如：在 `components/banner/index.js` 中配置了 `styleIsolation: "shared"`



解决：点击左下角【切换真机调试2.0】，切换后根据自己手机选择对应的系统，然后再进行扫码即可解决。



登录用户信息功能开发



1. 模板代码实现：登录用户信息区域

```

1 <!-- pages/index/index.wxml -->
2 <view>
3   <!-- 头部轮播图 -->
4   <banner list="{{ headerAdverts }}" />
5
6   <!-- 登录用户信息区域 -->
7   <view class="index-info-wrapper">
8     <view class="index-info-user flex-space-between-center">
9       <view class="index-info-user-left flex-row-align-center"
10         bind:tap="navTo" data-logged data-url="/pages/user-info/user-info?id=
    {{member._id}}">

```

```
11      <image class="avatar-img" src="{member.avatarUrl ||  
'/assets/images/avatar.png'}"/>  
12      <text wx:if="{{ logged }}" class="nickname text-one">{{member.nickname  
|| '请设置昵称'}}<text class="iconfont icon-arrow-right"></text></text>  
13      <text wx:else class="login-nav">欢迎您，请登录</text>  
14    </view>  
15    <view class="index-info-user-right flex-row-end">  
16      <view class="flex-column-center btn-item"  
17        bind:tap="navTo" data-logged data-url="/pages/member-code/member-  
code">  
18        <text class="iconfont icon-qrcode"></text>  
19        <text>会员码</text>  
20      </view>  
21      <view class="flex-column-center btn-item"  
22        bind:tap="toOrderPage">  
23        <text class="iconfont icon-order"></text>  
24        <text>订单</text>  
25      </view>  
26    </view>  
27  </view>  
28 </view>  
29  
30 <view>
```

2. 页面样式：登录用户信息区域

```
1  /* pages/index/index.scss */  
2  
3  // 登录用户信息区域  
4  .index-info-wrapper {  
5    position: relative;  
6    top: -36px;  
7    box-shadow: var(--basic-box-shadow-light);  
8    margin: 0 30rpx;  
9    border-radius: 20rpx;  
10   background-color: #fff;  
11   overflow: hidden;  
12   .index-info-user {  
13     height: 130rpx;  
14     padding: 0 15px;  
15     font-weight: 500;  
16     .index-info-user-left {  
17       width: 300rpx;  
18       .avatar-img {  
19         width: 76rpx;  
20         height: 76rpx;  
21         margin-right: 10rpx;
```

```
22     border-radius: 50%;
23     border: 1px solid #fff;
24 }
25 .nickname {
26     flex: 1;
27     font-size: 32rpx;
28     white-space: nowrap;
29 }
30 .login-nav {
31     font-size: 30rpx;
32     color: var(--primary-color);
33 }
34 }
35 .index-info-user-right {
36     flex: 1;
37     font-size: 24rpx;
38     line-height: 46rpx;
39     .btn-item {
40         margin-left: 36rpx;
41         &:last-child {
42             margin-right: 14rpx;
43         }
44         .iconfont {
45             font-size: 46rpx;
46         }
47     }
48 }
49 }
50
51 }
```

3. JS 逻辑代码:

- 导入自定义方法 `import { navTo } from '@/utils/util.js'`

如果导入报错: module 'utils/util.js' is not defined, require args is '@/utils/util.js' Component is not found in path "wx://not-found". 解决:

1. 在 app.json 中添加 "resolveAlias": { "@/*": "/" }
2. 重启当前微信开发者工具

```
1 // pages/index/index.js
2 /*
3 如果导入报错:
4 module 'utils/util.js' is not defined, require args is '@/utils/util.js'
5 Component is not found in path "wx://not-found".
6 解决:
7 1. 在 app.json 中添加
8 "resolveAlias": {
```

```
9      "@/*": "/*"
10    }
11    2. 重启当前微信开发者工具
12    */
13    import { navTo } from '@utils/util'
14    import { toOrderPage } from '@utils/page'
15
16    Page({
17
18      data: {
19        headerAdverts: [1, 2, 3, 4], // 头部轮播图数据
20        logged: false, // 是否已登录
21        member: null, // 当前登录的会员信息
22      },
23
24      navTo, // 简化 navTo: navTo, 跳转非tabBar页面
25
26      toOrderPage, // 跳转订单列表页
27
28    })
```

快捷菜单功能开发



1. 模板代码实现：快捷菜单区域

```
1 <!-- pages/index/index.wxml -->
2 <view >
3
4   <!-- 头部轮播图 -->
5
6   <!-- 登录用户信息区域 -->
```

```
7
8      <!-- 快捷菜单区域 -->
9      <view class="index-quick-wrapper">
10         <view class="flex-row-center">
11             <view class="take-type-item" bind:tap="toShoppingPage" data-take-type="
12 {{1}}">
13                 <image class="img" src="/assets/images/store.png" />
14                 <text class="title">到店自取</text>
15             </view>
16             <view class="take-type-item" bind:tap="toShoppingPage" data-take-type="
17 {{2}}">
18                 <image class="img" src="/assets/images/rider.png" />
19                 <text class="title">外送上门</text>
20             </view>
21         </view>
22         <view class="activity-list-box flex-row-align-center">
23             <view class="activity-item flex-column-center"
24                 bind:tap="navTo" data-logged data-url="/pages/recharge/recharge">
25                 <image class="img" src="/assets/images/recharge.png" />
26                 <text class="name">账户充值</text>
27                 <text class="intro">充值更优惠</text>
28             </view>
29             <view class="activity-item flex-column-center"
30                 bind:tap="navTo" data-url="/pages/shop-list/shop-list">
31                 <image class="img" src="/assets/images/store-map.png" />
32                 <text class="name">附近门店</text>
33                 <text class="intro">找店更便捷</text>
34             </view>
35             <!-- button组件添加 open-type="share" 实现分离页面-->
36             <button class="activity-item activity-item-btn flex-column-center" open-
37 type="share">
38                 <image class="img" src="/assets/images/brand.png" />
39                 <text class="name">分享有礼</text>
40                 <text class="intro">分享更快乐</text>
41             </button>
42         </view>
43     </view>
```

2. 页面样式：快捷菜单区域

```
1  /* pages/index/index.scss */
2
3  // 登录用户信息区域
4
5  // 快捷菜单区域
```



```
6 .index-quick-wrapper {
7   margin: 0 30rpx;
8   border-radius: 20rpx;
9   background-color: #fff;
10  box-shadow: var(--basic-box-shadow-light);
11  overflow: hidden;
12  .take-type-item {
13    flex: 1;
14    display: flex;
15    flex-direction: column;
16    justify-content: center;
17    align-items: center;
18    padding: 36rpx 0;
19    position: relative;
20    &:first-child::after {
21      content: '';
22      position: absolute;
23      top: 60rpx;
24      bottom: 60rpx;
25      right: 0;
26      border-right: var(--basic-border); /* 右边框 */
27    }
28    .img {
29      width: 180rpx;
30      height: 160rpx;
31    }
32    .title {
33      margin-top: 14rpx;
34      font-weight: bold;
35      font-size: 36rpx;
36    }
37  }
38  .activity-list-box {
39    border-top: var(--basic-border);
40    margin: 0 28rpx;
41    padding: 28rpx 0;
42    .activity-item {
43      flex: 1;
44      .img {
45        width: 80rpx;
46        height: 80rpx;
47        margin-bottom: 10rpx;
48      }
49      .name {
50        color: #222;
51        font-size: 24rpx;
52        line-height: 46rpx;
53        font-weight: bold;
54      }
55    }
56  }
```

```
55     .intro {
56         font-size: 20rpx;
57         color: #555555;
58     }
59 }
60 // 分享按钮
61 .activity-item-btn {
62     padding: 0;
63     line-height: inherit;
64     background-color: inherit;
65     &::after {
66         border: 0;
67     }
68 }
69
70 }
71 }
```

3. JS 逻辑代码:

- 新增 `toShoppingPage` 方法，实现跳转点单页

```
1 // pages/index/index.js
2
3 import { toOrderPage, toShoppingPage } from '@utils/page'
4
5 Page({
6
7     // 跳转点单页
8     toShoppingPage(e) {
9         const { takeType } = e.currentTarget.dataset
10         toShoppingPage( { takeType } ) // 调用的是最上方导入的方法
11     },
12
13 })
```

新增页面配置 app.json

```
1 {
2     "pages": [
3         "pages/index/index",
4         "pages/shopping/shopping",
5         "pages/order-list/order-list",
6
7         // 新增以下页面配置
```

```
8      "pages/login/login",          // 登录页面
9      "pages/user-info/user-info",  // 个人资料页面
10     "pages/member-code/member-code", // 会员码页面
11     "pages/recharge/recharge",     // 充值页面
12     "pages/shop-list/shop-list"    // 附近门店页面
13 ],
14     .....
15 }
```

热门推荐功能开发



自定义图片水平滑动组件

1. 创建图片水平滑动组件 `components/scroll-image/index`
2. 图片水平滑动模板代码实现：

```
1  <!-- components/scroll-image/index.wxml -->
2
3  <!-- 水平滑动-如：推门推荐 -->
4  <view class="scroll-image-wrapper" wx:if="{{list && list.length>0}}">
5    <view class="title-box flex-space-between">
6      <text>{{ title }}</text>
7      <view wx:if="{{rightText}}" class="more" bind:tap="handleRight">
8        <text>{{rightText}}</text>
9        <text class="iconfont icon-arrow-right"></text>
10     </view>
11   </view>
12   <view class="scroll-parent-view">
13     <scroll-view class="scroll-view" scroll-x>
14       <view class="scroll-view-inner">
15         <navigator class="scroll-item-nav" hover-class="none"
16           wx:for="{{ list }}" wx:key="index">
17           <image class="item-img" src="/assets/images/hot{{index+1}}.png" />
18         </navigator>
19       </view>
20     </scroll-view>
21   </view>
```

22 </view>

3. 组件样式

```
1  /* components/scroll-image/index.wxss */
2
3  .scroll-image-wrapper {
4    margin-top: 30rpx;
5    .title-box {
6      margin: 0 30rpx;
7      font-size: 36rpx;
8      color: #555555;
9      font-weight: bold;
10   .more {
11     color: #a3a3a3;
12     font-weight: normal;
13     font-size: 28rpx;
14   }
15   .iconfont {
16     font-size: 28rpx;
17   }
18 }
19 .scroll-parent-view {
20   margin-top: 5rpx;
21   padding-left: 30rpx;
22   .scroll-view {
23     white-space: nowrap;
24     height: 260rpx;
25     .scroll-view-inner {
26       margin: 20rpx 0;
27       border-radius: 30rpx;
28       height: 220rpx;
29       .scroll-item-nav {
30         display: inline-block;
31         margin-right: 30rpx;
32         height: 100%;
33       }
34       .item-img {
35         width: 600rpx;
36         height: 100%;
37         border-radius: 16rpx;
38         box-shadow: 2px 2px 6px #c9c9c9;
39       }
40     }
41   }
42 }
43 }
```

4. JS 逻辑代码：

- addGlobalClass 开启引用全局样式
- properties 接收父组件传递的数据
- 添加 handleRight 方法：当点击右上角（更多）按钮，触发父组件的 right 事件

```
1 // components/scroll-image/index.js
2 Component({
3
4   options: {
5     addGlobalClass: true
6   },
7
8   /**
9    * 组件的属性列表
10   */
11   properties: {
12     title: {
13       type: String,
14       value: '热门推荐'
15     },
16     rightText: {
17       type: String,
18       value: '更多'
19     },
20     list: {
21       type: Array,
22       value: [1, 2, 3, 4]
23     }
24   },
25
26   /**
27    * 组件的方法列表
28   */
29   methods: {
30
31     // 点击右上角（更多）按钮
32     handleRight() {
33       this.triggerEvent('right') // 触发父组件的right事件
34     }
35
36   },
37
38 })
```

首页引用轮播图组件

1. 在 pages/index/index.json 页面配置中

- usingComponents 中注册组件 `scroll-image`

```
1 {
2   "navigationStyle": "custom",
3   "usingComponents": {
4     "banner": "/components/banner/index",
5     "scroll-image": "/components/scroll-image/index"
6   }
7 }
```

2. 在 pages/index/index.wxml 页面模板中使用 `scroll-image` 组件

```
1 <!-- pages/index/index.wxml -->
2
3 <view >
4   <!-- 头部轮播图 -->
5
6   <!-- 登录用户信息区域 -->
7
8   <!-- 快捷菜单区域 -->
9
10  <!-- 热门推荐 -->
11  <scroll-image list="{{ hotList }}" title="热门推荐" rightText="更多"
12    bind:right="toShoppingPage" />
13 </view>
```

3. 在 pages/index/index.js 文件的 data 选项中声明 `hotList` 状态属性

```
1 // pages/index/index.js
2
3 Page({
4
5   data: {
6     hotList: [1, 2, 3, 4], // 热门推荐数据
7   },
8
9 })
```

隐藏页面滚动条

在 `pages/index/index.wxml` 中的根组件 `<view>` 上添加 `class="none-scrollbar"`

```
1 <!-- 隐藏页面滚动条 -->
2 <view class="none-scrollbar">
3   <!-- 头部轮播图 -->
4   <!-- 登录用户信息区域 -->
5   <!-- 快捷菜单区域 -->
6   <!-- 热门推荐 -->
7 </view>
```

在 `assets/style/global.scss` 定义了隐藏页面纵向滚动条全局样式

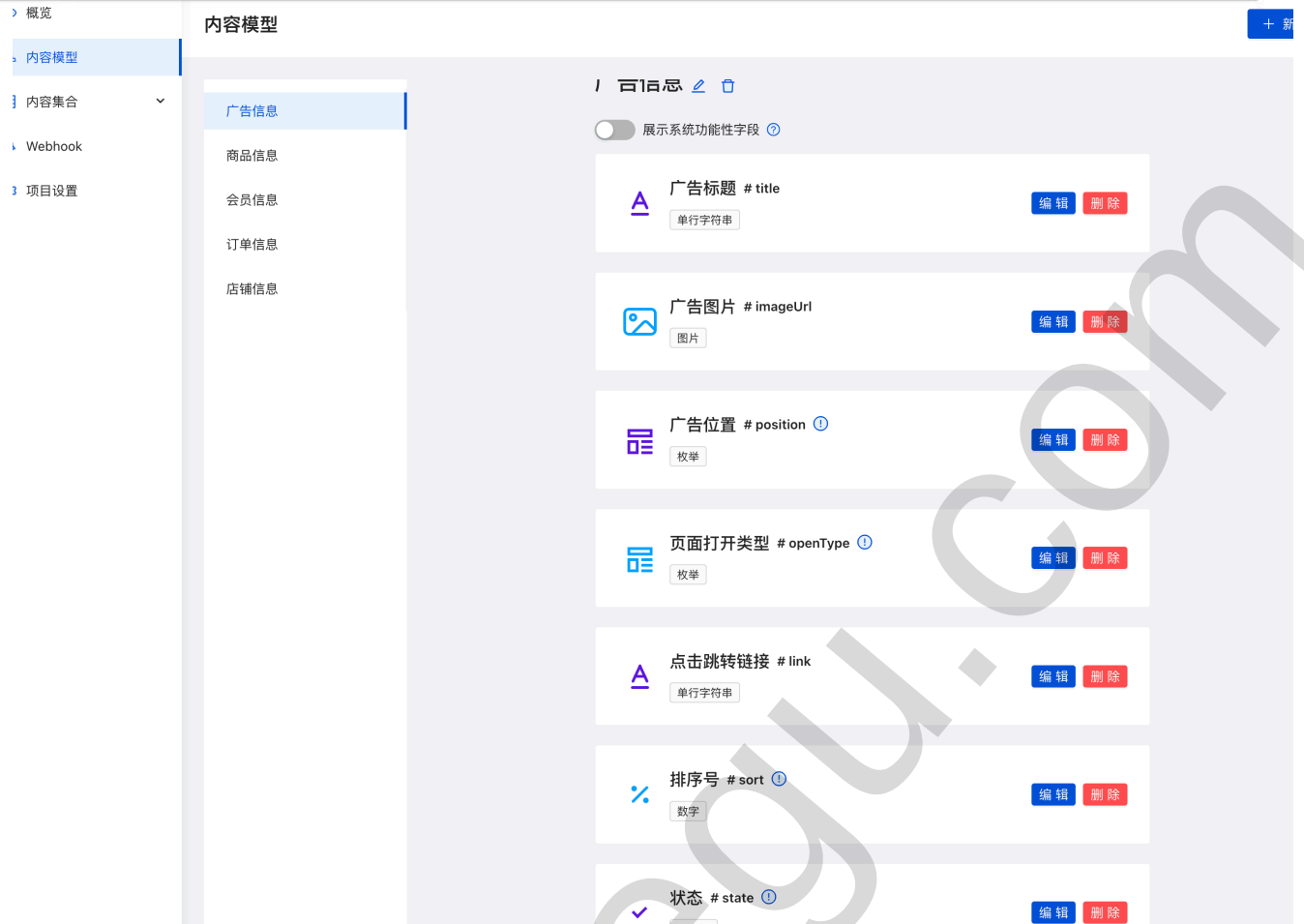
```
1 // 隐藏页面纵向滚动条
2 .none-scrollbar {
3   height: 100vh;
4   overflow-y: scroll;
5   &::-webkit-scrollbar{
6     display: none;
7   }
8 }
```

首页相关接口开发

创建广告信息模型和新增内容集合

广告信息模型：用于管理小程序中所有广告信息。

1. 创建内容模型：广告信息



2. 点击【内容集合】 - 【广告信息】 - 【+新建】添加内容集合（新增数据），如下：



在打开的窗口，填写广告相关信息：

概览

内容模型

内容集合

广告信息

商品信息

会员信息

订单信息

店铺信息

Webhook

项目设置

广告标题

1 首页轮播图-新鲜

* 广告图片

2 
点击或拖拽图片上传



14711728613205100_banner1.png



* 广告位置

3 首页头部轮播

1-首页头部轮播，2-首页热门推荐，3-点单页头部广告

页面打开类型

4 新窗口打开

新窗口打开(navigate)，当前窗口打开(redirect)，Tab页面打开 (switchTab)，关闭所有页面打开 (reLaunch)

点击跳转链接

5 /page/goods-detail/goods-detail?id=111

排序号

6 1

由小到大排序，值越小越靠前

* 状态

7 True

true正常，false禁用

头部轮播图：新增如下3条记录

文档 ID	广告标题	广告图片	广告位置	页面打开类型	点击跳转链接	排序号	状态
jRpRiz9O4eakkqw9	首页轮播图-新鲜	 下载图片 访问链接	首页头部轮播	新窗口打开	/page/goods-detail/g...	1	True
fIAIg7ITIQORdIX	咖啡	 下载图片 访问链接	首页头部轮播	新窗口打开	/page/goods-detail/g...	2	True
hn70T0agQqVpDP30	奶茶	 下载图片 访问链接	首页头部轮播	新窗口打开	/page/goods-detail/g...	3	True

热门推荐：新增如下3条记录

文档 ID	广告标题	广告图片	广告位置	页面打开类型	点击跳转链接	排序号	状态
fV3n3XXe3KmT6aYp	奶盖	 下载图片 访问链接	首页热门推荐	新窗口打开	/page/goods-detail/g...	1	True
p1UqoFxZZFeOCtnc	哇哈哈	 下载图片 访问链接	首页热门推荐	新窗口打开	/page/goods-detail/g...	2	True
ttg1kqdOVMblzSjO	水果奶茶	 下载图片 访问链接	首页热门推荐	新窗口打开	/page/goods-detail/g...	3	True

配置云函数环境

在 app.js 中的 onLaunch 应用生命周期函数中配置云函数环境

```

1 // app.js
2 App({
3
4   onLaunch() {
5
6     // 云开发初始化：用于指定使用的云开发环境
7     wx.cloud.init({
8       env: 'mxg-order-miniapp-xxxxx', // 指定云开发的环境id，后续调用云开发api时，
        所使用的环境就是此环境id，
9       traceUser: true, // 是否将用户访问日志记录到用户管理中（在开发控制台中的【运营分
        析】》【用户访问】中可见
10     })
11
12   }
13
14 })
    
```

创建云函数查询广告接口

1. 创建云函数 `getAdvertList`：查询指定位置的广告信息（'1' 首页轮播图，'2' 热门推荐）

记得修改云函数代码后，一定要上传并部署

```

1 // cloudfunctions/getAdvertList/index.js
2
3 // 云函数入口文件
4 const cloud = require('wx-server-sdk')
5
6 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
    
```

```
7
8  const db = cloud.database()
9
10 // 查询指定位置的广告信息
11 exports.main = async (event, context) => {
12
13   // position: '1' 首页轮播图, '2' 热门推荐
14   const { position } = event
15   // 查询广告信息
16   return await db.collection('mxg_advert')
17     .where({
18       position: position,
19       state: true // 正常状态
20     })
21     .orderBy("sort", "asc") // 按sort字段升序
22     .get()
23
24 }
```

小程序端封装调用云函数的API方法

1. 小程序端封装调用云函数的API方法，实现查询：首页轮播图; '2' 首页推门推荐

创建文件：miniprogram/api/advert.js

```
1  // miniprogram/api/advert.js
2
3  /**
4   * 查询指定位置的广告信息
5   * @param {string} position 位置编号: '1' 首页轮播图; '2' 首页推门推荐
6   */
7  export async function getAdvertList(position) {
8    const { result } = await wx.cloud.callFunction({
9      name: 'getAdvertList',
10     data: {
11       position
12     }
13   })
14   return result
15 }
```

页面调用API方法

在 pages/index/index.js 中调用查询广告信息接口，实现查询头部轮播图和热门推荐数据：

- 导入 showLoading, hideLoading, getAdvertList 方法,
- 将 data 选项中 bannerList 和 hotList 初始化为空数组

```
1 // pages/index/index.js
2
3 import { navTo, showLoading, hideLoading } from '@utils/util'
4 import { getAdvertList } from '@api/advert'
5
6 Page({
7
8   data: {
9     bannerList: [], // 头部轮播图数据
10    hotList: [], // 热门推荐数据
11  },
12
13  // 加载轮播图和热门推荐数据
14  async loadAdvertList(position) {
15    // 不要少了 await
16    const { data } = await getAdvertList(position)
17    if (position === '1') this.setData({ bannerList: data })
18    if (position === '2') this.setData({ hotList: data })
19  },
20
21  /** 生命周期函数--监听页面加载 */
22  async onLoad() {
23    try {
24      // 加载中
25      showLoading()
26      // 头部轮播图
27      this.loadAdvertList('1')
28      // 热门推荐, await等待它完成, 方法前要加async
29      await this.loadAdvertList('2')
30    } catch (error) {
31      console.error('首页查询数据异常', error)
32    } finally {
33      hideLoading()
34    }
35  },
36
37
38 })
```

页面渲染数据

1. 修改轮播图子组件的模板代码：

- 修改 `<swiper>` 标签上的属性 `indicator-dots="{{list.length > 1}}"` 2张图以上才显示指示点
- 修改 `<swiper>` 标签上的属性 `wx:key="_id"`
- 在 `<navigator>` 标签上添加属性 `url="{{item.link}}" open-type="{{item.openType}}"`
- 修改 `<image />` 标签上的属性 `src="{{item.imageUrl}}"`

```
1 <!--components/banner/index.wxml-->
2 <!-- 轮播图，声明css变量 --swiper-height -->
3 <swiper class="swiper-banner"
4     style="--swiper-height:{{height}};--dots-bottom:{{dotsBottom}}"
5     autoplay
6     circular
7     indicator-dots="{{list.length > 1}}"
8     indicator-color="#e7e7e7"
9     indicator-active-color="#FFF"
10    interval="5000"
11  >
12  <swiper-item wx:for="{{ list }}" wx:key="_id">
13    <!-- 链接跳转，open-type="navigate|redirect|switchTab|reLaunch|navigateBack"
-->
14    <navigator class="nav" hover-class="none" url="{{item.link}}" open-type="{{
15      item.openType }}">
16      <image class="swiper-img" src="{{ item.imageUrl }}" />
17      <!-- <image class="swiper-img" src="/assets/images/banner{{index}}.png"
-->
18    </navigator>
19  </swiper-item>
20 </swiper>
```

2. 修改热门推荐子组件的模板代码：

- 在 `<navigator>` 标签上添加属性 `wx:key="id" url="{{item.link}}" open-type="{{
15 item.openType }}"`
- 修改 `<image/>` 标签上的属性 `src="{{item.imageUrl}}"`

```
1 <!-- components/scroll-image/index.wxml -->
2
3 <!-- 水平滑动-推门推荐 -->
4 <navigator class="scroll-item-nav" hover-class="none" wx:for="{{ list }}"
  wx:key="_id"
5     url="{{item.link}}" open-type="{{item.openType}}">
6   <image class="item-img" src="{{item.imageUrl}}" />
7   <!-- <image class="item-img" src="/assets/images/hot{{index+1}}.png" /> -->
8 </navigator>
```

第三章 登录页面开发

在首页点击【欢迎您，请登录】，跳转到登录页面 `/pages/login/login`，页面效果如下：



登录页面效果实现

1. 在 `login.json` 中配置导航栏标题、禁止页面滚动 `"disableScroll":true`


```
1 {  
2   "navigationBarTitleText": "授权登录",  
3   "navigationBarTextStyle": "black",  
4   "navigationBarBackgroundColor": "#f9f9f9",  
5   "disableScroll": true,  
6   "usingComponents": {}  
7 }
```

2. 登录页面模板代码:

```
1 <!--pages/login/login.wxml-->  
2  
3 <view class="login-container flex-column-center">  
4   <view class="avatar-view flex-center">  
5     <image class="avatar-img" src="/assets/images/logo.png" />  
6   </view>  
7  
8   <text>成为会员，立享更多优惠福利</text>  
9  
10  <view class="login-btns">  
11    <button type="primary" bind:tap="loginCheck">一键登录</button>  
12  </view>  
13  
14  <view class="xieyi-box flex-row-align-center">  
15    <switch model:checked="{{agree}}" type="checkbox" class="checkbox-radius"  
16    id="xieyi" color="red" />  
17    <label for="xieyi" class="flex-row-align-center">  
18      <text>我已阅读并同意<text class="xieyi-text">《用户隐私协议》</text><text  
19      class="xieyi-text">《用户协议》</text></text>  
20    </label>  
21  </view>  
22 </view>
```

3. 登录页面样式代码:

```
1 /* pages/login/login.wxss */  
2  
3 .login-container {  
4   .avatar-view {  
5     height: 400rpx;  
6     .avatar-img {  
7       width: 230rpx;  
8       height: 230rpx;  
9       border-radius: 100%;  
10      overflow: hidden;
```

```
11     }
12 }
13
14 .tip-text {
15     font-size: 32rpx;
16 }
17
18 .login-btns {
19     width: 600rpx;
20     margin-top: 90rpx;
21     button {
22         margin-bottom: 30rpx;
23     }
24 }
25
26 .xieyi-box {
27     font-size: 26rpx;
28     color: #666666;
29
30     .xieyi-text {
31         color: var(--primary-color);
32     }
33
34     /* 修改选择框默认样式 */
35     .checkbox-radius .wx-checkbox-input {
36         /* 未选中的 背景样式 */
37         border-radius: 50%; /* 圆角 */
38         margin-right: 10rpx;
39         transform: scale(0.7); /*大小缩小*/
40         &.wx-checkbox-input-checked { /* 选中后的 背景样式 */
41             border-radius: 50%;
42             border-color: var(--primary-color) !important;
43             background-color: var(--primary-color) !important;
44             &::before{
45                 color:#fff; /* 对勾颜色 白色 */
46             }
47         }
48     }
49 }
50 }
51
52 }
```

登录逻辑实现

登录功能实现步骤

在小程序中登录功能实现步骤：

1. 一进入登录页面，就进行预登录：

预登录目的：主要判断用户之前是否登录过，来决定是快速登录，还是授权手机号登录

- 云函数实现：通过当前用户 `_openid` 查询会员(`mxg_member`)数据，如果不存在是新会员则进行注册（新增会员信息）
 - 然后判断此用户是否已绑定过手机号。（已绑定直接快速登录；未绑定点击【一键登录】发起手机号授权）
2. 点击【一键登录】按钮，判断是否已勾选协议，勾选了再进行登录。
3. 已绑定手机号直接快速登录：将会员用户信息存储到本地 Storage 中。
4. 未绑定手机号点击【一键登录】发起手机号授权（企业小程序才可用，个人不可用）：

参考：https://developers.weixin.qq.com/miniprogram/dev/devtools/minitest/phone_mock.html
<https://developers.weixin.qq.com/miniprogram/dev/framework/open-ability/getPhoneNumber.html>

- 步骤1：需要将 `button` 组件 `open-type` 的值设置为 `getPhoneNumber`，当用户点击并同意之后，通过 `bindgetphonenumber` 事件获取回调信息，接收获取手机号的授权码 `code`；

```
1 <button wx:else="{{agree}}" type="primary" open-type="getRealtimePhoneNumber" bindgetrealtimephonenumber="loginGetPhoneNumber">一键登录</button>
```

- 步骤2：将 `bindgetphonenumber` 事件回调中的动态令牌 `code` 传到云函数，并在云函数中调用微信后台提供的 `cloud.openapi.phonenumber.getPhoneNumber` 接口，传递用户的手机号授权码 `code` 换取用户手机号。每个 `code` 有效期为5分钟，且只能消费一次。

`loginGetPhoneNumber` 方法中打印日志，报错如下：没有权限，

`getPhoneNumber:fail operateWXData:fail jsapi has no permission`

原因是：不支持个人小程序，需要企业小程序才可以。

```
{type: "getrealtimephonenumber", timeStamp: 5030, target: , currentTarget: , mark: , ...}
  currentTarget: Object
  detail: Object
    errMsg: "getPhoneNumber:fail operateWXData:fail jsapi has no permission"
    errno: 102
```

预登录实现

1. 创建 `preLogin` 云函数，实现：

- 通过当前用户 `_openid` 查询会员(`mxg_member`)数据，如果不存在是新会员则进行注册（新增会员信息）
- 编写后，记得上传并部署云函数

```
1 // cloudfunctions/preLogin/index.js
2
3 // 云函数入口文件
4 const cloud = require('wx-server-sdk')
5
6 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
7
8 const db = cloud.database()
9
10 // 预登录：云函数入口函数
11 exports.main = async (event, context) => {
12   const { OPENID } = cloud.getWXContext()
13
14   // 1. 通过openId查询会员
15   const { data } = await db.collection('mxg_member')
16     .where({
17       _openid: OPENID
18     })
19     .get()
20
21   // 2. 存在，直接返回会员信息
22   if (data && data.length > 0) {
23     return data[0]
24   }
25
26   // 3. 不存在，注册新会员
27   const member = {
28     _openid: OPENID,
29     isAdmin: false, // 是否为管理员
30     cardNo: Date.now(),
31     _createTime: Date.now(),
32   }
33
34   const created = await db.collection('mxg_member').add({
35     data: member
36   })
37
38   return member
39 }
```

2. 登录页面预登录逻辑实现

```
1 // pages/login/login.js
2 import { msg, navBack, showLoading, hideLoading } from "@utils/util"
3
4 Page({
5
```

```
6   data: {
7     agree: false, // 是否同意协议
8     boundMobile: false, // 是否绑定手机号
9   },
10
11   onLoad() {
12     this.preLogin()
13   },
14
15   /**
16    * 预登录:
17    * 1、调用云函数: 通过当前用户 _openid 查询会员, 如果是新会员则注册 (新增会员信息)
18    * 2、判断是否绑定了手机号: 未绑定点击【一键登录】发起手机号授权, 已绑定直接登录成功
19    */
20   async preLogin() {
21     try {
22       showLoading()
23       // 1、调用云函数: 通过当前用户 _openid 查询会员信息, 如果是新会员则注册
24       const {result: member} = await wx.cloud.callFunction({
25         name: 'preLogin'
26       })
27
28       // 2. 判断是否绑定了手机号: 未绑定点击【一键登录】发起手机号授权, 反之直接登录成功
29       // !! 表示有值返回true, 否则返回false
30       const boundMobile = !(member && member.mobile)
31       this.setData({
32         member,
33         boundMobile
34       })
35     } catch (error) {
36       console.error('preLogin预登录失败', error)
37     } finally {
38       hideLoading()
39     }
40   },
41
42 })
```

快速登录

已绑定手机号直接快速登录: 将会员用户信息存储到本地 Storage 中。

1. 修改页面模板中的【一键登录】:

在模板中加条件判断的原因:

获取用户手机号要通过 button 组件指定 `open-type="getRealtimePhoneNumber"`, 所以要单独判断显示哪个逻辑按钮。

```
1 <view class="login-btns">
2   <!-- 未勾选同意协议：提示勾选 -->
3   <button wx:if="{{ !agree }}" type="primary" bind:tap="loginCheck">一键登
   录</button>
4   <!-- 已勾选协议且已绑定手机号：直接快速登录 -->
5   <button wx:elif="{{ boundMobile }}" type="primary" bind:tap="loginQuick">一键登
   录</button>
6 </view>
```

2. 判断是否已勾选协议，勾选了再允许进行登录。

```
1 // pages/login/login.js
2 import { msg, navBack, showLoading, hideLoading } from "@utils/util"
3
4 Page({
5
6   // 登录前检查：提示未勾选同意协议
7   loginCheck() {
8     if (!this.data.agree) {
9       return msg('请阅读并勾选下方协议')
10    }
11  },
12
13   // 快速登录：已勾选协议且已绑定手机号
14   loginQuick() {
15     try {
16       showLoading('登录中')
17       const { boundMobile } = this.data
18       // 未绑定
19       if (!boundMobile) return
20       // 已绑定：保存会员信息到本地，表示登录成功
21       this.loginSuccess()
22     } catch (error) {
23       msg('登录失败')
24       console.error('loginQuick快速登录失败', error)
25     } finally {
26       hideLoading()
27     }
28   },
29
30   // 登录成功：保存并后退
31   loginSuccess() {
32     // 本地保存新会员信息，表示登录成功。 app.js中添加storageMember方法
33     getApp().storageMember(this.data.member)
34     msg('登录成功', {icon: 'success'})
35     navBack()
36   },
37 })
```

```
37  
38  })
```

3. 在 app.js 中添加全局变量 memberInfoKey 和方法 storageMember，实现保存会员信息

```
1  // app.js  
2  App({  
3  
4    // 保存到本地Storage的key  
5    memberInfoKey: 'member_info',  
6  
7    // 保存会员信息  
8    storageMember(member) {  
9      wx.setStorageSync(this.memberInfoKey, member)  
10    },  
11  
12  })
```

4. 测试：

- 确保 preLogin云函数 已上传并部署成功。
- 手动修改预登录时新增的会员记录 (mxg_member)：添加一个 mobile 字段指定一个手机号。



- 后退再重新进入登录页，点击【一键登录】进行快速登录。

授权手机号登录

未绑定手机号点击【一键登录】发起手机号授权：

- 步骤1：需要将 `button` 组件 `open-type` 的值设置为 `getPhoneNumber`，当用户点击并同意之后，通过 `bindgetphonenumber` 事件获取回调信息，接收获取手机号的授权码 `code`

```
1  <button wx:else="{{agree}}" type="primary" open-type="getRealtimePhoneNumber"  
    bindgetrealtimephonenumber="loginGetPhoneNumber">一键登录</button>
```


- 步骤2：将 `bindgetphonenumber` 事件回调中的动态令牌 `code` 传到云函数，并在云函数中调用微信后台提供的 [cloud.openapi.phonenumber.getPhoneNumber](#) 接口，传递用户的手机号授权码 `code` 换取用户手机号。每个 `code` 有效期为5分钟，且只能消费一次。
 - 目前该接口针对非个人开发者（企业），且完成了认证的小程序开放；个人小程序不允许调用。
 - 每次调用成功收费 0.03 元，每个小程序账号将有1000次体验额度，用于开发、调试和体验
 - 扣费节点：开发者获得 `bindgetphonenumber` 事件的 `success` 回调信息时，进行扣费。

1. 在页面模板中再加一个针对 授权手机号的【一键登录】按钮：

```
1 <view class="login-btns">
2   <!-- 未勾选同意协议：提示勾选 -->
3   <button wx:if="{{!agree}}" type="primary" bind:tap="loginCheck">一键登
录</button>
4   <!-- 已勾选协议且已绑定手机号：直接快速登录 -->
5   <button wx:elif="{{boundMobile}}" type="primary" bind:tap="loginQuick">一键登
录</button>
6
7   <!-- 已勾选协议且未绑定手机号：发起授权并绑定手机号 -->
8   <button wx:else type="primary"
9     open-type="getRealtimePhoneNumber"
bindgetrealtimephonenumber="loginGetPhoneNumber">
10     一键登录
11   </button>
12 </view>
```

2. 接收用户手机号授权码

```
1 // 接收用户手机号授权码，换取手机号并绑定：已勾选协议且未绑定手机号
2 async loginGetPhoneNumber(res) {
3   // console.log('loginGetPhoneNumber', res.detail.code)
4   try {
5     showLoading('登录中')
6     // 接收用户同意的获取手机号的授权码code，通过code获取手机号
7     const { code } = res.detail
8     if (!code) return
9
10    // 调用云函数：通过授权码code换取手机号
11    const { result: member } = await wx.cloud.callFunction({
12      name: 'getPhoneNumber',
13      data: {
14        code: code // 获取手机号的授权码
15      }
16    })
17
18    // 更新状态
```



```
19     this.setData({member, boundMobile: true})
20
21     // 登录成功
22     this.loginSuccess()
23   } catch (error) {
24     msg('登录失败')
25     console.error('loginGetPhoneNumber登录失败', error)
26   } finally {
27     hideLoading()
28   }
29 },
```

3. 创建云函数 `getPhoneNumber`，并上传部署

```
1  // cloudfunctions/getPhoneNumber/index.js
2
3  // 云函数入口文件
4  const cloud = require('wx-server-sdk')
5
6  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
7
8  const db = cloud.database()
9
10 /**
11  * 获取手机号云函数入口函数，
12  *
13  * 关于 cloud.openapi.phonenumber.getPhoneNumber:
14  * 1. 参考: https://developers.weixin.qq.com/miniprogram/dev/framework/open-ability/getPhoneNumber.html
15  * 2. 目前该接口针对非个人开发者（企业），且完成了认证的小程序开放；个人小程序不允许调用
16  * 3. 每次调用成功收费 0.03 元，每个小程序账号将有1000次体验额度，用于开发、调试和体验
17  */
18 exports.main = async (event, context) => {
19   const { OPENID } = cloud.getWXContext()
20
21   // 通过code获取手机号
22   const res = await cloud.openapi.phonenumber.getPhoneNumber({
23     code: event.code
24   })
25   const { phoneNumber } = res.phoneInfo
26
27   // 更新会员：绑定手机号
28   const updated = await db.collection('mxg_member')
29     .where({
30       _openid: OPENID
31     })
32     .update({
```

```
33     data: {
34         mobile: phoneNumber
35     }
36 })
37
38 // 查询会员返回
39 const { data } = await db.collection('mxg_member')
40     .where({
41         _openid: OPENID
42     })
43     .get()
44
45 if (data && data.length > 0) {
46     return data[0]
47 } else {
48     return null
49 }
50
51 }
```

4. 测试:

- 将云数据库的 `mxg_member` 集合中的当前用户会员记录的mobile字段删除，表示未绑定过手机号
- 然后重新进行登录页，进行用户授权手机号【一键登录】

首页显示登录信息

1. 在 app.json 中新增 `current` 全局方法：获取当前会员登录信息

```
1 // app.js
2
3 import { navTo } from "@utils/util"
4
5 App({
6
7     // 保存到本地Storage的key
8     memberInfoKey: 'member_info',
9
10    // 保存会员信息
11    storageMember(member) {
12        wx.setStorageSync(this.memberInfoKey, member)
13    },
14
15    // 获取当前会员登录信息，mustLogin为true则未登录时跳转登录页
16    current(mustLogin = false) {
17        // 从本地获取保存的会员登录信息
```

```
18     const member = wx.getStorageSync(this.memberInfoKey)
19
20     // logged: 如果有手机号，表示登录成功
21     const logged = !(member && member.mobile)
22     if (mustLogin && !logged) navTo('/pages/login/login')
23     return {logged, member}
24 },
25
26 })
27
```

2. 在首页 pages/index/index.js 中的 onShow 生命钩子中调用全局 current 方法获取登录信息

```
1  /** 页面每次显示时调用 */
2  onShow() {
3      // 调用 app.json 中的current方法，获取登录用户信息
4      const {logged, member} = getApp().current()
5      this.setData({ logged, member: member || null })
6  },
```

第四章 个人资料页面开发

需求分析

在首页点击用户昵称，进入【个人资料】页面，进行修改登录用户会员信息。

15:57

100% 



个人资料



昵称 梦学谷

性别 ☒ 先生 ☐ 女士

生日 请选择日期

手机 1XXXXXXXXXX

保存

退出登录

获取用户微信头像和昵称: <https://developers.weixin.qq.com/miniprogram/dev/framework/open-ability/userProfile.html>

• 获取微信头像: `<button open-type="chooseAvatar" bind:chooseavatar="onChooseAvatar">`

• 获取微信昵称:

1. 需要使用form组件包裹住input以及button组件, form组件添加属性 bindsubmit="handleConfirm"

即: `<form bind:submit="onConfirm">`

2. input输入框组件的 type属性设置为nickname,用户点击输入框，键盘上方才会显示微信昵称。如果添加了name属性，form组件就会自动收集带有name属性的表单元素的值 即：`<input type="nickname" name="nickname" />`
3. 如果按钮组件添加 `form-type = "submit"` 属性就会变为提交按钮。在点击提交按钮时候，会触发表单的 `bind:submit` 提交事件。
即：`<button type="primary" form-type="submit">保存</button>`

个人资料页面效果实现

1. 在 user-info.json 中配置导航栏标题，和注册 van weapp 输入框和单选框组件

```
1  {
2    "navigationBarTitleText": "个人资料",
3    "navigationBarBackgroundColor": "#ffffff",
4    "navigationBarTextStyle": "black",
5    "backgroundColor": "#ffffff",
6    "backgroundColorBottom": "#f9f9f9",
7    "usingComponents": {
8      "van-field": "@vant/weapp/field/index",
9      "van-radio": "@vant/weapp/radio/index",
10     "van-radio-group": "@vant/weapp/radio-group/index"
11   }
12 }
13
```

2. 个人资料页面模板代码：

```
1  <!--pages/user-info/user-info.wxml-->
2
3  <view class="user-avatar-wrapper flex-center">
4    <button class="user-avatar-btn" open-type="chooseAvatar"
5      bind:chooseavatar="onChooseAvatar">
6      <!-- <image class="user-avatar" src="{{avatarUrl ||
7        '/assets/avatar.png'}}"></image> -->
8      <image class="user-avatar" src="{{tmpAvatarUrl || avatarUrl ||
9        '/assets/images/avatar.png'}}"></image>
10    </button>
11  </view>
12
13  <form bind:submit="submitForm">
14    <view class="user-info-wrapper">
15      <!-- 注意：真机上可正确获取用户昵称，开发工具不行 -->
16      <van-field
17        label="昵称" value="{{nickname}}"
```

```
15     type="nickname" name="nickname"
16     placeholder="请输入昵称"
17     error-message="{{ error.nickname || ' ' }}"
18   />
19   <van-field label="性别" error-message="{{ error.sex || ' ' }}">
20     <van-radio-group slot="input" model:value="{{ sex }}"
direction="horizontal">
21       <van-radio name="{{1}}" checked-color="#e40030">先生</van-radio>
22       <van-radio name="{{2}}" checked-color="#e40030">女士</van-radio>
23     </van-radio-group>
24   </van-field>
25   <van-field
26     label="生日" model:value="{{ birthday }}"
27     clearable
28     error-message="{{ error.birthday || ' ' }}"
29   >
30     <picker slot="input" class="picker-date" model:value="{{birthday}}"
mode="date">
31       <view class="picker-date-body">{{birthday || '请选择日期'}}</view>
32     </picker>
33   </van-field>
34 </view>
35
36 <van-field
37   readonly
38   label="手机" model:value="{{ mobile }}"
39   placeholder="请输入手机号码"
40   error-message="{{ error.mobile || ' ' }}"
41 >
42 </van-field>
43
44 <view class="user-confirm-btn">
45   <button type="primary" form-type="submit">保 存</button>
46 </view>
47 </form>
48
49 <view class="logout-box" bind:tap="logout">
50   <text>退出登录</text>
51 </view>
```

3. 个人资料页面样式代码：

```
1  /* pages/user-info/user-info.wxss */
2
3  .user-avatar-wrapper {
4    height: 370rpx;
5    width: 100%;
```

```
6 background-color: #fff;
7 .user-avatar-btn {
8   padding: 0;
9   border-radius: 100%;
10  background-color: inherit;
11  &::after {
12    border: 0;
13  }
14 }
15 .user-avatar {
16   display: block;
17   width: 200rpx;
18   height: 200rpx;
19 }
20 }
21
22
23 .user-info-wrapper {
24   margin: 30rpx 0;
25   .picker-date {
26     width: 100%;
27     .picker-date-body {
28       height: var(--cell-line-height, 24px);
29       width: 100%;
30     }
31   }
32 }
33
34 .user-confirm-btn {
35   margin: 80rpx 60rpx;
36 }
37
38 .logout-box {
39   color: #999999;
40   font-size: 28rpx;
41   padding: 10rpx;
42   width: 150rpx;
43   text-align: center;
44   margin: 0 auto;
45   margin-top: 180rpx;
46 }
```

4. 定义逻辑代码：

- 获取微信头像、微信昵称
- 实现退出功能

```
1 // pages/user-info/user-info.js
2
```



```
3 import { hideLoading, isNull, msg, navBack, showLoading } from "@utils/util"
4 import { toIndexPage } from "@utils/page"
5 import { getMemberById, updateMemberById } from "@api/member"
6 import { uploadFile } from "@api/file"
7
8 Page({
9
10   /**
11    * 页面的初始数据
12    */
13   data: {
14     member: {},
15     tmpAvatraUrl: '', // 临时头像url, 暂未提交上传
16     avatarUrl: '', //
17     nickname: '',
18     birthday: '',
19     sex: 1,
20     mobile: '',
21     error: {} // 表单校验错误提示信息
22   },
23
24   /**
25    * 生命周期函数--监听页面加载
26    */
27   onLoad(options) {
28     this.loadData(options.id)
29   },
30
31   // 查询个人资料信息
32   loadData(id) {
33     if (!id) return msg('未传递参数')
34     console.log('loadData', id)
35   },
36
37   // 选择头像
38   onChooseAvatar(e) {
39     const { avatarUrl } = e.detail
40     this.setData({ tmpAvatraUrl: avatarUrl })
41   },
42
43   // 提交表单保存
44   submitForm(e) {
45     console.log('submitForm', e)
46     // 获取昵称, **注意**: 模拟器测试获取不到值, 真机才获取得到值
47     const { nickname } = e.detail.value
48   },
49
50   // 退出登录
51   logout() {
```

```
52     wx.removeStorageSync(getApp().memberInfoKey)
53     toIndexPage() // 前往首页
54 },
55
56 })
```

个人资料逻辑实现

创建与部署查询和修改云函数

1. 创建与部署云函数 `getById`：通过文档 `_id` 查询指定集合的记录

```
1 // cloudfunctions/getById/index.js
2
3 // 云函数入口文件
4 const cloud = require('wx-server-sdk')
5
6 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
7
8 const db = cloud.database()
9
10 // 云函数入口函数：实现通过文档id查询指定集合的记录
11 exports.main = async (event, context) => {
12     // collectionName 集合名称, id 文档id
13     const {collectionName, id} = event
14     return await db.collection(collectionName).doc(id).get()
15 }
```

2. 创建与部署云函数 `updateById`：通过文档 `_id` 更新指定集合的记录

```
1 // cloudfunctions/updateById/index.js
2
3 // 云函数入口文件
4 const cloud = require('wx-server-sdk')
5
6 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
7
8 const db = cloud.database()
9
10 // 云函数入口函数：通过文档 _id 更新指定集合的记录
11 exports.main = async (event, context) => {
12     const {collectionName, id, data} = event
13     return await db.collection(collectionName).doc(id)
14         .update({
15             data
16         })
17 }
```

```
16     })  
17 }
```

封装调用云函数的API和上传文件API

1. 小程序端封装调用云函数的API方法，创建 `miniprogram/api/member.js` 文件，实现：

- 通过会员 id 查询会员信息 `getMemberById`
- 通过会员 id 更新会员信息 `updateMemberById`

```
1  // miniprogram/api/member.js  
2  
3  /**  
4   * 通过会员id 查询会员信息  
5   * @param id 会员id  
6   */  
7  export async function getMemberById(id) {  
8    const { result } = await wx.cloud.callFunction({  
9      name: 'getById', // 调用的云函数名  
10     data: {  
11       collectionName: 'mxg_member', // 集合名  
12       id  
13     }  
14   })  
15   return result  
16 }  
17  
18 /**  
19 * 通过会员id更新会员信息  
20 * @param id 会员id  
21 * @param data 更新的数据  
22 */  
23 export function updateMemberById(id, data) {  
24   return wx.cloud.callFunction({  
25     name: 'updateById', // 调用的云函数名  
26     data: {  
27       collectionName: 'mxg_member', // 集合名  
28       id,  
29       data  
30     }  
31   })  
32 }
```

2. 封装上传文件到云存储的API方法，创建 `miniprogram/api/file.js`

```
1  /**
```

```
2 * 上传文件：图片、视频、文档等
3 * @param {string} tempFilePath 文件本地临时路径
4 * @param {string} fileType 文件类型：image/video/file
5 */
6 export function uploadFile(tempFilePath, fileType='image') {
7   // 云存储路径，注意：ios真机上传的有 tmp_ 前缀，如果有把它替换掉
8   const cloudPath = fileType +
tempFilePath.substring(tempFilePath.lastIndexOf('/') + 1).replace('tmp_', '')
9   // console.log('cloudPath', cloudPath)
10  // 开始上传
11  return wx.cloud.uploadFile({
12    cloudPath, // 上传至云端的路径
13    filePath: tempFilePath, // 小程序临时文件路径
14  })
15
16 }
```

页面逻辑实现

1. onLoad 中接收路径参数会员id，调用查询个人资料信息API方法

```
1 // pages/user-info/user-info.js
2
3 import { hideLoading, isNull, msg, navBack, showLoading } from "@utils/util"
4 import { toIndexPage } from "@utils/page"
5 import { getMemberById, updateMemberById } from "@api/member"
6 import { uploadFile } from "@api/file"
7
8 Page({
9
10   /**
11    * 页面的初始数据
12    */
13   data: {
14     member: {},
15     tmpAvatarUrl: '', // 临时头像url，暂未提交上传
16     avatarUrl: '', //
17     nickname: '',
18     birthday: '',
19     sex: 1,
20     mobile: '',
21     error: {} // 表单校验错误提示信息
22   },
23
24   /**
25    * 生命周期函数--监听页面加载
26    */
27 })
```

```
27   onLoad(options) {
28     this.loadData(options.id)
29   },
30
31   // 查询个人资料信息
32   async loadData(id) {
33     if (!id) return msg('未传递参数')
34     try {
35       showLoading()
36       // 查询会员最新数据
37       const { data: member } = await getMemberById(id)
38       // console.log('member', member)
39
40       // 解构出会员信息属性
41       const { avatarUrl='', nickname='', sex=1, birthday='', mobile='' } =
member
42       this.setData({ member, avatarUrl, nickname, birthday, sex, mobile })
43     } catch (error) {
44       msg('个人资料获取失败')
45       console.log('个人资料获取失败', error)
46     } finally {
47       hideLoading()
48     }
49   },
50
51 })
```

2. 提交表单数据保存

- 校验用户昵称、性别 等为必填必选项
- 上传头像到云端存储
- 更新数据到云数据库

注意：获取昵称，模拟器测试获取不到值，真机才获取得到值

```
1   // 提交表单保存
2   async submitForm(e) {
3     // console.log('submitForm', e)
4     try {
5       showLoading('保存中')
6
7       // 获取昵称，**注意**：模拟器测试获取不到值，真机才获取得到值
8       const { nickname } = e.detail.value
9       // 注意：是let，不是const声明
10      let { member, tmpAvatarUrl, avatarUrl, birthday, sex } = this.data
11
12      // 1. 校验数据
13      let error = {}
14      if (isNull(nickname)) error.nickname = '昵称不能为空'
```

```
15     if (isNull(sex)) error.sex = '请选择性别'
16     this.setData({ error })
17     if (Object.keys(error).length>0) return
18
19     // 2. 上传头像到云端存储
20     if (tmpAvatraUrl) {
21         const { fileId } = await uploadFile(tmpAvatraUrl, 'avatar')
22         // 更新状态
23         this.setData({avatarUrl: fileId, tmpAvatraUrl: ''})
24         // 赋值用于更新到数据库
25         avatarUrl = fileId
26     }
27
28     // 3. 更新数据
29     const updateData = {avatarUrl, nickname, birthday, sex }
30     const { result } = await updateMemberById(member._id, updateData)
31     console.log('更新数据结果', result)
32
33     // 更新登录信息
34     getApp().storageMember({...member, avatarUrl, nickname, birthday, sex})
35     msg('保存成功')
36     setTimeout(() => {
37         navBack()
38     }, 500)
39 } catch (error) {
40     msg('保存失败')
41     console.error('更新数据异常', error)
42 } finally {
43     hideLoading()
44 }
45 },
```

第五章 会员码页面开发

需求：会员码页面根据登录会员卡号，来动态生成条形码和二维码（生成工具参见
`utils/wxbarcode/index.js`）



页面模板实现

1. 配置页面导航栏

```
1 {  
2   "navigationBarTitleText": "会员码",  
3   "navigationBarBackgroundColor": "#f9f9f9",  
4   "backgroundColor": "#f9f9f9",  
5   "usingComponents": {}  
6 }
```

2. 会员码页面模板代码:

```
1 <!--pages/member-code/member-code.wxml-->  
2 <view class="qrcode-view">  
3   <view class="header-box flex-center">  
4     <text class="iconfont icon-saoma fw6 mr5"></text>  
5     <text class="title">快速识别会员</text>  
6   </view>  
7   <view class="code-box flex-column-center">  
8     <!-- 条形码 -->
```

```
9      <canvas canvas-id="barcode" class="barcode" />
10      <!-- 会员编码值 -->
11      <text class="code-text">{{code}}</text>
12      <!-- 二维码 -->
13      <canvas canvas-id="qrcode" class="qrcode" />
14      <view class="tip-view flex-row-align-center">
15          <text class="iconfont icon-tishi"></text>
16          <text>本会员码定时更新，请勿截图，以免影响使用</text>
17      </view>
18  </view>
19 </view>
```

3. 会员码页面样式代码：

```
1  /* pages/member-code/member-code.wxss */
2
3  .qrcode-view {
4      margin: 30rpx;
5
6      .header-box {
7          background-color: var(--primary-color);
8          height: 90rpx;
9          border-radius: 20rpx 20rpx 100% 100%;
10         margin: 0 30rpx;
11         position: relative;
12         bottom: -40rpx;
13         text-align: center;
14         color: #fff;
15         font-size: 30rpx;
16         .iconfont {
17             font-weight: bold;
18             margin-right: 10rpx;
19             font-size: inherit;
20         }
21     }
22
23     .code-box {
24         background: rgba(255, 255, 255, 0.9);
25         box-shadow: 0rpx 8rpx 40rpx 0rpx rgba(0,0,0,0.08);
26         border-radius: 32rpx;
27         padding: 40rpx;
28         .barcode {
29             margin-top: 60rpx;
30             height: 200rpx;
31             width: 600rpx;
32         }
33         .qrcode {
```



```
34     height: 400rpx;
35     width: 400rpx;
36   }
37   .code-text {
38     font-size: 28rpx;
39     color: #888888;
40     margin-top: 40rpx;
41   }
42   .tip-view {
43     color: #888888;
44     font-size: 26rpx;
45     .iconfont {
46       font-size: inherit;
47       margin-right: 10rpx;
48     }
49   }
50 }
51 }
```

会员码逻辑实现

1. 会员码逻辑实现步骤：

- 导入生成条形码 barcode 和二维码 qrcode 的工具方法
- 页面显示时，`wx.getScreenBrightness` 获取屏幕亮度。用于后退恢复亮度
- 页面卸载时，`wx.setScreenBrightness` 设置屏幕亮度。恢复到初始高度

```
1  // pages/member-code/member-code.js
2  // 导入生成条形码和二维码的工具方法
3  import { barcode, qrcode } from '@/utils/wxbarcode/index'
4
5  Page({
6
7    /**
8     * 页面的初始数据
9     */
10   data: {
11     defaultLight: 1, // 屏幕默认亮度值
12     code: '' // 会员卡号
13   },
14
15   /**
16    * 生命周期函数--监听页面加载
17    */
18   onLoad() {
19     // 获取会员卡号
20     const { member } = getApp().current()
```

```
21     const code = member.cardNo || '111188899999'
22     this.setData({ code })
23
24     // 生成条形码：页面中的 Canvas id属性值，条码值，宽，高
25     barcode('barcode', code, 600, 200)
26     // 生成二维码
27     qrcode('qrcode', code, 400, 400)
28   },
29
30   /** 生命周期函数--监听页面显示 */
31   onShow() {
32     // 获取屏幕亮度，用于后退恢复亮度
33     wx.getScreenBrightness({
34       success: (res) => {
35         this.setData({defaultLight: res.value})
36         // 设置屏幕亮度，value取值范围 0 ~ 1, 0 最暗, 1 最亮
37         wx.setScreenBrightness({value: 0.7})
38       }
39     })
40   },
41
42   /** 生命周期函数--监听页面卸载 */
43   onUnload() {
44     // 设置屏幕亮度：恢复到初始高度
45     wx.setScreenBrightness({value: this.data.defaultLight})
46   },
47
48 })
```

第六章 账户充值页面+微信支付开发

账户充值页面效果，如下：



账户充值页面效果实现

1. 在 recharge.json 中配置导航栏标题

```
1 {  
2   "navigationBarTextStyle": "white",  
3   "navigationBarTitleText": "账户充值",  
4   "navigationBarBackgroundColor": "#e40030",  
5   "backgroundColor": "#e40030",  
6   "backgroundColorBottom": "#f9f9f9",  
7   "usingComponents": {}  
8 }
```

2. 帐户充值页面模板代码:

```
1 <!--pages/recharge/recharge.wxml-->  
2  
3 <view class="recharge-container">  
4   <view class="account-balance flex-column-center">  
5     <text>余额</text>  
6     <text>{{ balance || '0.00' }}元</text>  
7   </view>  
8  
9   <view class="recharge-list">  
10    <view>充值: </view>  
11    <view class="list">  
12      <view class="recharge-item {{item.id === selected.id ? 'active': ''}}"  
13        wx:for="{{rechargeList}}" wx:key="id"  
14        bind:tap="changeRecharge" data-selected="{{item}}">  
15        <view>¥{{item.payAmount}}</view>  
16        <view>到账{{item.rechargeAmount}}元</view>  
17      </view>  
18    </view>  
19  </view>  
20  
21  <view class="desc">  
22    <text>说明: </text>  
23    <text>  
24      1. 充值后不可提现、退换、转让等。  
25      2. 如遇无法充值、充值失败等问题, 请联系商家询问。  
26    </text>  
27  </view>  
28  
29  <view class="bottom center">  
30    <button class="btn" loading="{{loading}}" bind:tap="submitRecharge">  
31      立即充值  
32    </button>  
33  </view>  
34 </view>
```

3. 帐户充值页面样式代码：

```
1  /* pages/recharge/recharge.wxss */
2
3  .recharge-container {
4    .account-balance {
5      justify-content: center;
6      height: 288rpx;
7      background-color: var(--primary-color);
8      color: #FFFFFFF;
9      font-size: 88rpx;
10     text:first-child {
11       color: #e7e4e9;
12       font-size: 30rpx;
13     }
14   }
15   .recharge-list {
16     margin: 20rpx 0 0 20rpx;
17     font-size: 30rpx;
18     color: #999999;
19     .list {
20       margin-top: 20rpx;
21       text-align: center;
22       font-size: 26rpx;
23       .recharge-item {
24         float: left;
25         width: 225rpx;
26         border: var(--basic-border);
27         border-radius: 10rpx;
28         background-color: #FFFFFFF;
29         margin-right: 10rpx;
30         margin-bottom: 15rpx;
31         padding: 20rpx 0;
32         flex-wrap: wrap;
33         view:first-child {
34           color: var(--primary-color);
35           font-size: 32rpx;
36         }
37       }
38     }
39   }
40
41   .active {
42     box-shadow: 0 0 0 1px var(--primary-color);
43     color: var(--primary-color) !important;
44   }
45
46   .desc {
47     // 清除浮动
```

```
48     clear: both;
49     margin: 0 20rpx;
50     font-size: 30rpx;
51     line-height: 45rpx;
52     color: #6e6d70;
53     text:first-child {
54         font-weight: bold;
55         line-height: 80rpx;
56     }
57     text:last-child {
58         font-size: 26rpx;
59     }
60 }
61
62 .bottom {
63     position: fixed;
64     left: 0;
65     right: 0;
66     bottom: 0;
67     height: 130rpx;
68     background-color: #FFFFFF;
69     padding-top: 20rpx;
70     .btn {
71         width: 700rpx;
72         background-color: var(--primary-color);
73         color: #FFFFFF;
74         border-radius: 50rpx;
75         line-height: 80rpx;
76         font-size: 30rpx;
77         &::after {
78             // 加载中隐藏边框
79             border: none;
80         }
81     }
82 }
83 }
```

帐户充值逻辑

1. 帐户充值下面实现以下逻辑：

- 初始化充值项
- 切换充值项，亮度边框显示
- 查询会员当前剩余余额

```
1 // pages/recharge/recharge.js
2
```

```
3 import { getMemberById } from "@api/member"
4 import { hideLoading, msg, showLoading } from "@utils/util"
5
6 Page({
7
8   data: {
9     // 充值金额列表
10    rechargeList: [
11      // payAmount支付金额 , rechargeAmount到账金额
12      { id: 1, payAmount: 0.01, rechargeAmount: 0.01 },
13      { id: 2, payAmount: 50, rechargeAmount: 60 },
14      { id: 3, payAmount: 100, rechargeAmount: 130 },
15      { id: 4, payAmount: 300, rechargeAmount: 400 },
16      { id: 5, payAmount: 500, rechargeAmount: 680 },
17      { id: 6, payAmount: 1000, rechargeAmount: 1400 }
18    ],
19    selected: {id: 1}, // 选中充值项
20    balance: '0.00', // 账户余额
21    loading: false, // 是否充值中
22  },
23
24  onLoad() {
25    // 默认选中的充值项
26    this.setData({selected: this.data.rechargeList[0]})
27
28    // 查询会员数据
29    this.loadMemberData()
30  },
31
32  // 切换充值项
33  changeRecharge(e) {
34    const { selected } = e.currentTarget.dataset
35    this.setData({ selected })
36  },
37
38  // 查询会员数据
39  async loadMemberData() {
40    try {
41      showLoading()
42      // 获取登录会员, 要求必须选登录
43      const {member, logged} = getApp().current(true)
44      if (!logged) return
45
46      // 查询会员
47      const { data } = await getMemberById(member._id)
48
49      // 转为两位小数
```

```
51     const balance = data.balance ? parseFloat(data.balance).toFixed(2) :  
    '0.00'  
52     // 更新状态中  
53     this.setData({member: data, balance})  
54   } catch (error) {  
55     msg('查询账户余额失败')  
56     console.log('查询账户余额失败', error)  
57   } finally {  
58     hideLoading()  
59   }  
60 },  
61  
62  
63   // 提交立即充值  
64   submitRecharge() {  
65  
66   },  
67  
68  
69 })
```

2. 实现充值金额更新到会员余额中

- 创建 updateBalance 云函数更新会员余额
 - 使用 command 操作符 inc 实现累加运算

云函数文件：cloudfunctions/updateBalance/index.js

```
1  // 云函数入口文件  
2  const cloud = require('wx-server-sdk')  
3  
4  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境  
5  
6  const db = cloud.database()  
7  const _ = db.command  
8  
9  // 云函数入口函数：更新会员余额  
10 exports.main = async (event, context) => {  
11   // 会员id, 本次充值金额  
12   const {id, rechargeAmount} = event  
13  
14   // 更新会员余额  
15   return await db.collection('mxg_member')  
16     .doc(id)  
17     .update({  
18       data: {  
19         // inc函数, 将balance 字段值自增 rechargeAmount 值, 小数有精度问题,  
20         balance: _.inc(rechargeAmount) //
```



```
21     }  
22     })  
23  
24 }
```

- 封装调用云函数API方法

位于：miniprogram/api/member.js

```
1  /**  
2   * 更新会员余额  
3   * @param id 会员id  
4   * @param rechargeAmount 充值金额  
5   */  
6  export function updateBalance(id, rechargeAmount) {  
7    return wx.cloud.callFunction({  
8      name: 'updateBalance',  
9      data: {  
10       id,  
11       rechargeAmount  
12     }  
13   })  
14 }
```

- 提交立即充值：调用api方法更新会员余额，暂未对接微信支付。

```
1  import { hideLoading, msg, showLoading } from "@utils/util"  
2  import { getMemberById, updateBalance } from "@api/member"  
3  
4  Page({  
5  
6    // 提交充值  
7    async submitRecharge() {  
8      // 校验是否选择了充值项  
9      const { member, selected } = this.data  
10     if (!selected || !selected.id) {  
11       msg('请选择充值金额', {icon: 'error'})  
12       return  
13     }  
14  
15     try {  
16       showLoading('充值中', {mask: true})  
17  
18       // 获取选择的支付金额和充值金额  
19       const { payAmount, rechargeAmount } = selected  
20     }
```

```
21 // TODO 1. 调用微信支付
22
23 // 2. 支付成功：更新帐户余额。不要少了await不然下面查询不到更新后的余额
24 await updateBalance(member._id, rechargeAmount)
25
26 // 3. 查询会员最新数据：余额；
27 await this.loadMemberData()
28
29 msg('充值成功', {icon: 'success'})
30 } catch (error) {
31   console.error('充值失败:', error)
32   msg('充值失败', {icon: 'error'})
33 } finally {
34   hideLoading()
35 }
36 },
37
38 })
```

微信支付功能

介绍云调用对接微信支付

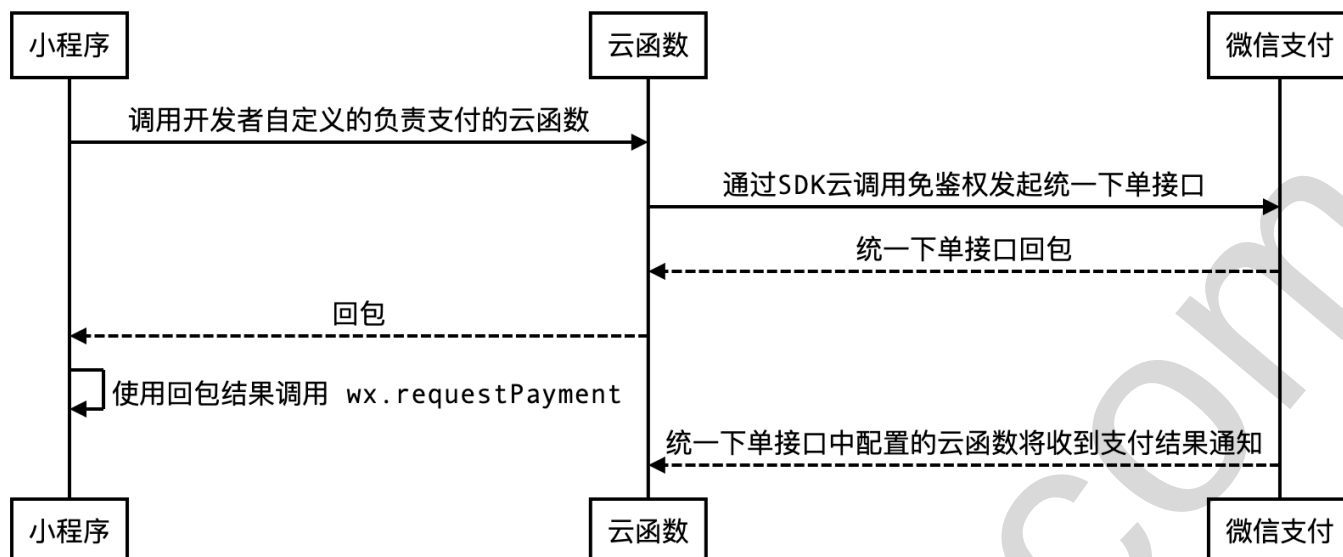
云调用对接微信支付：<https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/wechatpay/openapi.html>

使用云开发来实现相应的支付功能后，开发者无需关心证书、签名、微信支付服务器端文档，使用简单，代码较少，只需要调用相应的函数即可。此外，因为云开发基于微信私有协议实现，官方通过服务商提供支付接口对接支持，不依赖第三方模块，免去泄漏证书、支付情况等其他敏感信息的风险。同时，云开发还支持云函数接收微信支付进行支付和退款的回调，安全高效。

云开发场景下的微信支付调用方式支持 小程序/公众号 场景调用。

1. 安全：微信私有协议，无证书等信息泄露风险
2. 免签名：所有接口免签名 & 直接获取小程序 wx.requestPayment 所需参数
3. 接收回调：云函数支持接收异步支付结果回调

云调用对接微信支付流程如下：



注意：根据[交易类小程序运营规范](#)，对于小程序内提供珠宝玉石、3C 数码、盲盒、服饰内衣、海淘、美妆、酒类、家用电器、玩具、箱包皮具、鞋靴、运动户外等商品在线销售及配送服务时，不可使用云调用支付能力，请使用[云模板-微信支付模板](#)，或[使用工作流接入微信支付](#)。

开通微信支付功能

1. 需要有营业执照的企业或个体工商户，先进行开通微信支付功能。

参考[接入指引](#)，网址：https://pay.weixin.qq.com/static/applyment_guide/applyment_detail_minia_pp.shtml

登录微信支付商户平台，点击【产品中心】-【我的产品】-找到 JSPAPI 支付，



进入[JSAPI支付](#)后点击：申请开通



2. 开通微信支付后，要在微信支付商户平台要绑定小程序appid，对应小程序才可以使用微信支付功能。

绑定网址：https://pay.weixin.qq.com/index.php/extend/merchant_appid/mapay_platform/account_manage

注意：待绑定小程序AppID必须要在[小程序公众平台](#)通过微信认证，且必须是企业或个体工商户小程序。

- 点击【产品中心】 - 【AppID账号管理】 - 【我关联的AppID账号】 - 【+关联AppID】

产品大全

我的产品

我授权的产品

开发配置

AppID账号管理

AppID账号管理

服务商为我关联的AppID账号

我关联的AppID账号

① 完成关联的AppID账号，将可用于支付，营销，用户触达，支付凭证小程序等业务。 [查看指引](#)

请输入昵称/AppID

搜索

+ 关联AppID

昵称

AppID

账号类型

关联状态

操作

暂无数据

- 在打开的新增授权中，填入已微信认证的企业或个体工商户小程序AppID、AppID认证主体（如果有，则填写）

- AppID：填写的要绑定的小程序AppID。

注意：是微信开发者工具中【详情】-【基本信息】中的AppID

Appid

wx5111111111111111

修改 复制

产品大全

我的产品

我授权的产品

开发配置

AppID账号管理

新增授权

基础信息

商户号

商户简称

企业全称

授权绑定的AppID

输入要绑定的小程序AppID

AppID

可输入已通过微信认证的服务号、政府或媒体类订阅号、小程序、企业微信、移动应用。 [查看指引](#)

AppID认证主体

上面输入AppID后，会显示AppID认证主体，
是个人小程序就输入绑定的用户姓名；
是企业小程序就输入绑定的企业名称。

☒ 我已阅读并同意 [《微信支付联合运行承诺函》](#) 以及 [《微信支付商户号与AppID授权协议》](#) 的相关内容

提交

- 填入小程序的AppID后，可能会提示：AppID未认证，请通过微信认证后重试，[AppID认证指引](#)

- 原因：要是企业或个体工商户的小程序，不支持个人小程序；而且小程序要通过了微信认证。

新增授权

基础信息

商户号
商户简称
企业全称

授权绑定的AppID

AppID

w-XXXXXX

AppID未认证，请通过微信认证后重试，[AppID认证指引](#)

可输入已通过微信认证的服务号、政府或媒体类订阅号、小程序、企业微信、移动应用。 [查](#)

☐ 我已阅读并同意 [《微信支付商户号与AppID授权协议》](#) 的相关内容

提交

3. 绑定小程序AppID成功后的效果：



微信支付

商户平台

首页

交易中心

账户中心

营销中心

产品中心

数据

产品大全

我的产品

我授权的产品

开发配置

AppID账号管理

AppID账号管理

我关联的AppID账号

① 完成关联的AppID账号，将可用于支付，营销，用户触达，支付凭证小程序等业务。 [查看指引](#)

请输入昵称/AppID

搜索

+ 关联AppID

昵称	AppID	账号类型	关联状态	操作
梦学谷	WXXXXXXXXX	小程序	已关联	查看

4. 复制商户号，在小程序代码中需要使用。【帐户中心】 - 【商户信息】 - 【微信支付商户号】



1. 微信支付配置进入【云开发】窗口，然后点击【设置】-【其他设置】-【微信支付配置】-【+添加商户号】

- 当【+添加商户号】不可点击，原因是：该 AppID 无绑定的商户号，请先完成商户号绑定再设置微信支付配置。



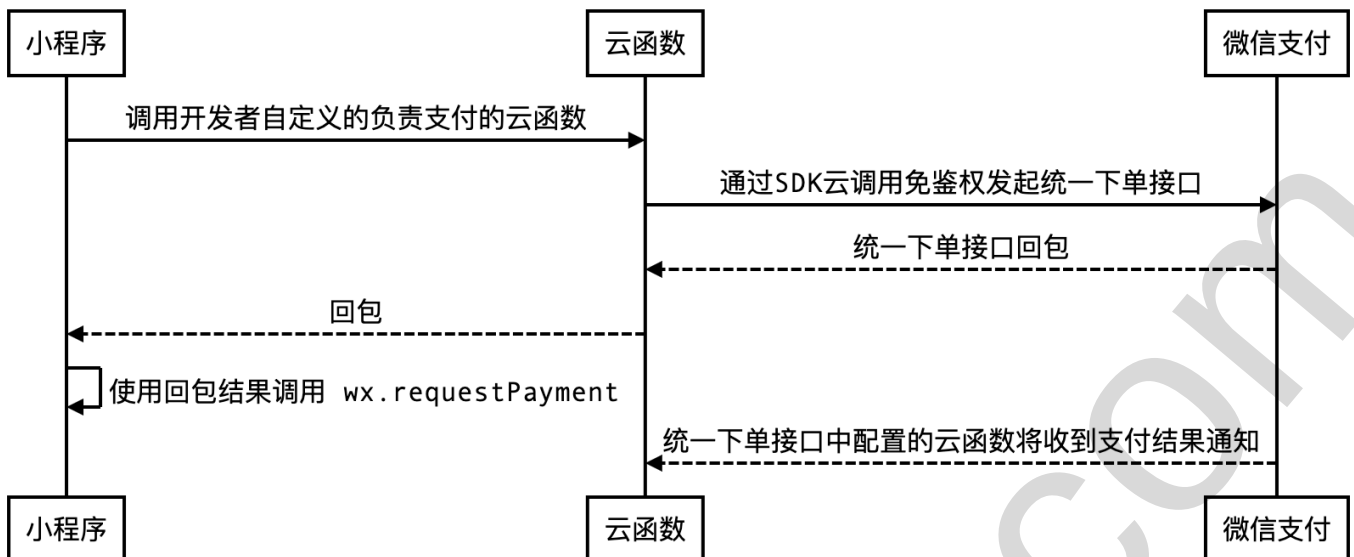


2. 添加商户号后，需要分别进行账号绑定、jsapi 和 api 退款权限授权。请注意：

- 1) 账号绑定：商户号的**超级管理员**微信会收到授权通知，点击通知进入微信支付提供的【微信支付商家助手】小程序上确认授权。
- 2) jsapi 和 api 退款权限，需要前往**微信支付商户平台**我的授权产品中进行确认授权。[说明](#)
完成授权后即可调用微信支付相关接口能力。

充值微信支付实现

云调用对接微信支付流程如下：



云函数调用统一下单接口

1. 创建 wxpayCloud 云函数：调用统一下单接口。

- 回调函数设置：envId 和 functionName 用来设置接收支付后的异步通知回调的云函数
- 考虑安全问题，subMchId 商户号 请填写自己的，
- 返回字段 payment：该对象即是在小程序端调用 wx.requestPayment 所需的信息

云函数文件：cloudfunctions/wxpayCloud/index.js

统一下单接口文档：<https://developers.weixin.qq.com/miniprogram/dev/wxcloud/reference-sdk-api/open/pay/CloudPay.unifiedOrder.html>

```

1 // cloudfunctions/wxpayCloud/index.js
2
3 // 云函数入口文件
4 const cloud = require('wx-server-sdk')
5
6 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
7
8 // 支付配置信息
9 const config = {
10   subMchId: '填写自己的', // 商户号
11   spbillCreateIp: '127.0.0.1', // IP
12 }
13
14 // 云函数入口函数：调用统一下单接口
15 exports.main = async (event, context) => {
16   const { ENV } = cloud.getWXContext()
17   const { body, orderNo, totalAmount, callFunctionName } = event
18
19   // 调用统一下单接口：在微信支付服务后台生成预支付交易单，返回正确的预支付交易后调起支付。
  
```

```
20     return await cloud.cloudPay.unifiedOrder({
21       body: body, // 商品描述
22       outTradeNo: orderNo, // 商户订单号
23       totalFee : totalAmount * 100, // 总金额，单位为【分】不能带小数； 元转分：
10.11*100=1011
24       envId: ENV, // 支付结果通知回调云函数环境ID
25       functionName: callFunctionName, // 支付结果通知回调云函数名
26       subMchId: config.subMchId, // 商户号
27       spbillCreateIp: config.spbillCreateIp, // 终端 IP
28     })
29
30 }
```

2. 创建支付回调云函数，

在发起[统一下单](#)时functionName指定的云函数会在用户完成支付后接收到支付结果回调，此时云函数的返回协议和入参协议如下：

支付结果回调的云函数必须返回如下一个对象，否则会视为回调不成功，云函数会收到重复的支付回调：

字段名	变量名	必填	类型	描述
错误码	errcode	是	Number	0
错误信息	errmsg	是	String	

支付回调云函数：cloudfunctions/wxpayCallRecharge/index.js

```
1  // cloudfunctions/wxpayCallRecharge/index.js
2
3  // 云函数入口文件
4  const cloud = require('wx-server-sdk')
5
6  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
7
8  // 云函数入口函数：账户充值微信支付回调结果云函数
9  exports.main = async (event, context) => {
10     const {returnCode} = event
11     if (returnCode == 'SUCCESS') {
12       // 支付成功，一定要返回下面对象，微信后台才不会再次回调此云函数
13       return {errcode: 0, errmsg: 'SUCCESS'}
14     } else {
15       // 支付失败，errcode返回非0则下次还是会回调
16       return {errcode: -1, errmsg: 'FAIL'}
17     }
18 }
```

小程序端封装微信支付API方法

1. 创建 `miniprogram/api/pay.js` 封装微信支付API方法

- 调用 `wxpayCloud` 云函数：微信支付统一下单
- 调用 `wx.requestPayment` 函数发起支付请求：传递统一下单接口返回的 `payment` 对象

```
1  /**
2   * 微信云调用支付:
3   * @param {string} body 支付描述
4   * @param {string|number} orderNo 订单号
5   * @param {number} totalAmount 支付金额
6   * @param {string} callFunctionName 支付结果回调云函数名
7   */
8  export async function wxpay(body, orderNo, totalAmount, callFunctionName) {
9    // 调用云函数：微信支付统一下单
10   const res = await wx.cloud.callFunction({
11     name: 'wxpayCloud',
12     data: {
13       body,
14       orderNo, // 订单号
15       totalAmount, // 支付金额
16       callFunctionName, // 支付结果回调云函数名
17     }
18   })
19   console.log('调用支付云函数响应结果', res)
20
21   // resultCode 业务结果: SUCCESS/FAIL
22   // errCodeDes 错误返回的信息描述
23   // payment 调用 wx.requestPayment 所需信息
24   const { resultCode, errCodeDes, payment } = res.result
25
26   // 统一下单失败：则返回错误信息
27   if (resultCode == 'FAIL') {
28     return Promise.reject(errCodeDes) // 交给调用方的catch捕获异常
29   }
30
31   // 2. 发起微信支付请求：传递统一下单接口返回的 payment 对象
32   return wx.requestPayment(payment)
33
34 }
```

充值业务微信支付实现

1. 在 submitRecharge 方法中，调用微信支付API方法，实现真实微信支付功能。

```
1 // pages/recharge/recharge.js
2
3 import { hideLoading, msg, showLoading } from "@utils/util"
4 import { getMemberById, updateBalance } from "@api/member"
5 import { wxpay } from "@api/pay"
6
7 Page({
8
9   // 提交充值
10  async submitRecharge() {
11    // 校验是否选择了充值项
12    const { member, selected } = this.data
13    if (!selected || !selected.id) {
14      msg('请选择充值金额', {icon: 'error'})
15      return
16    }
17
18    try {
19      showLoading('充值中', {mask: true})
20
21      // 获取选择的支付金额和充值金额
22      const { payAmount, rechargeAmount } = selected
23
24      // 1. 调用微信支付
25      const orderNo = "R" + Date.now()
26      const payRes = await wxpay('账户充值', orderNo, payAmount,
27 'wxpayCallRecharge')
28      console.log('支付成功', payRes)
29
30      // 2. 支付成功：更新帐户余额。不要少了await不然下面查询不到更新后的余额
31      await updateBalance(member._id, rechargeAmount)
32
33      // 3. 查询会员最新数据：余额
34      await this.loadMemberData()
35
36      msg('充值成功', {icon: 'success'})
37    } catch (error) {
38      console.error('充值失败:', error)
39      if ('requestPayment:fail cancel' !== error.errMsg) {
40        msg('充值失败', {icon: 'error'})
41      }
42    } finally {
43      hideLoading()
44    }
45  },
46 })
```

45

46 })

微信支付常见问题

1. 云函数调用统一支付接口，报错：受理关系不存在；

解决：点击云开发窗口右上角的【设置】-【其他设置】-【微信支付配置】-添加商户号

调用支付云函数响应结果

```
▼ {errMsg: "cloud.callFunction:ok", result: {...}, requestID: "2acf4fe3-  
  errMsg: "cloud.callFunction:ok"  
  requestID: "2acf4fe3-acb4-485e-a103-6237b8898222"  
  ▼ result: 在云开发设置中配置微信支付商户号  
    errMsg: 0  
    errMsgDes: "受理关系不存在"  
    errMsg: "cloudPay.unifiedOrder:ok"  
    nonceStr: "V8mMe0h42i40C7wN/"
```

2. 如果真机中弹出 jspapi 错误，检查是否在微信支付商户平台开通且授权了。

有其他原因，可查看 miniprogram/api/pay.js 中调用统一下单云函数打印的日志信息。

第七章 附近门店(地图)页面开发



页面配置 shop-list.json

```
1 {  
2   "disableScroll": true,  
3   "navigationBarTitleText": "门店列表",  
4   "navigationBarBackgroundColor": "#f9f9f9",  
5   "usingComponents": {  
6     "van-search": "@vant/weapp/search/index",  
7     "van-empty": "@vant/weapp/empty/index",  
8     "van-tabs": "@vant/weapp/tabs/index",  
9     "van-tab": "@vant/weapp/tab/index"  
10  }  
11 }
```

搜索区域实现

1. wxml模板代码实现

```
1 <!--pages/shop-list/shop-list.wxml-->
2
3 <!-- 搜索框区域 -->
4 <view class="shop-search-wrapper flex-row-align-center">
5   <van-search bind:tap="openChooseLocation" class="flex-1" disabled
6     placeholder="请搜索门店" background="transparent"/>
7   <view class="right-opt" bind:tap="toggleMap">
8     <text class="iconfont icon-ditu"/>
9     <text>{{showMap ? '收起地图' : '展开地图'}}</text>
10  </view>
11 </view>
```

2. css样式代码实现

```
1 /* pages/shop-list/shop-list.wxss */
2
3 /* 搜索区域 */
4 .shop-search-wrapper {
5   .van-search__content {
6     background-color: #f0f0f0;
7   }
8   .right-opt {
9     padding-right: 20rpx;
10    font-size: 28rpx;
11    color: #333;
12    .iconfont {
13      font-size: 30rpx;
14      margin-right: 6rpx;
15    }
16  }
17 }
```

3. js逻辑代码实现

- 打开地图选择位置，使用 `wx.chooseLocation(Object object)` 需要在 **app.json** 中做如下权限配置：

参考：<https://developers.weixin.qq.com/miniprogram/dev/api/location/wx.chooseLocation.html>

注意：

- 通过此方法，可以获取我们想要的地址在地图上的经纬度坐标
- 调用wx.chooseLocation时，在app.json未添加requiredPrivateInfos配置，会报如下错：


```
1 wx.chooseLocation need to be declared in the requiredPrivateInfos field
  in app.json/ext.json(env: macOS,mp,1.06.2405020; lib: 3.6.0)
2 shop-list.js? [sm]:56 chooseLocation undefined
3 TypeError: Cannot read property 'longitude' of undefined
```

```
1 "requiredPrivateInfos": [
2   "chooseLocation"
3 ]
```

o 业务逻辑代码实现

```
1 // pages/shop-list/shop-list.js
2
3 Page({
4
5   data: {
6     showMap: true, // 显示地图
7     longitude: 113.927078, // 当前位置经度
8     latitude: 22.52396, // 当前位置纬度
9   },
10
11   // 切换收起/展示地图
12   toggleMap() {
13     this.setData({ showMap: !this.data.showMap })
14   },
15
16   // 点击搜索门店：打开地图选择位置
17   async openChooseLocation() {
18     /**
19      * chooseLocation 打开地图选择位置
20      * 报错: chooseLocation:fail the api need to be declared in the
      requiredPrivateInfos field in app.json/ext.json
21      * 解决: 需要在 app.json 配置 "requiredPrivateInfos": ["chooseLocation"]
22      */
23     const res = await wx.chooseLocation()
24     console.log('chooseLocation', res)
25     // longitude经度, latitude纬度
26     const {longitude, latitude} = res
27     this.setData({longitude, latitude})
28
29     // 选择新位置后距离不一样，所以重新查询门店列表数据
30     this.loadShopList()
31   },
32
33   /**查询门店列表数据 */
34   loadShopList() {
35
36   },
```


37

38 })

地图展示区域实现

页面中的地图使用 `<map>` 组件，参考：<https://developers.weixin.qq.com/miniprogram/dev/component/map.html>

- `<map>` 组件常用属性：
 - `show-location` 显示当前定位点图标；使用 `moveToLocation` 方法将地图移动位置时，必须开启此属性
 - `longitude` 经度(浮点数，范围 -180 ~ 180)
 - `latitude` 纬度(浮点数，范围 -90 ~ 90)，不填经纬度则默认值是北京的经纬度
 - `scale` 地图缩放级别，取值范围为3-20，默认16
 - `markers` 标记点(可用于标记门店)
 - `bindmarkertap` 点击标记点时触发的事件

1. wxml模板代码实现

```
1  <!--pages/shop-list/shop-list.wxml-->
2
3  <!-- 搜索框区域 -->
4
5  <!-- 地图展示 -->
6  <view class="map-wrapper" hidden="{{!showMap}}" >
7    <!--
8      show-location 显示当前定位点图标；使用moveToLocation 方法将地图移动位置时，必须开启
      此属性
9      longitude 经度(浮点数，范围 -180 ~ 180)
10     latitude 纬度(浮点数，范围 -90 ~ 90)，不填经纬度则默认值是北京的经纬度
11     scale 地图缩放级别，取值范围为3-20，默认16
12     markers 标记点(可用于标记门店)
13     bindmarkertap 点击标记点时触发
14   -->
15   <map id="shop-map" class="map"
16     show-location
17     longitude="{{longitude}}"
18     latitude="{{latitude}}"
19     scale="{{scale}}"
20     markers="{{markers}}"
21     bindmarkertap="clickMarkert"
22   >
23   <!-- 恢复回当前位置图标，在【收藏门店】标签页时隐藏 -->
24   <view hidden="{{activeTabName == 'collect'}}" bind:tap="resetMapLocation"
25     class="map-location" hover-class="view-hover" hover-stay-time="100">
26     <text class="iconfont icon-location" />
```

```
27     </view>
28   </map>
29 </view>
30
31 <!-- open-type="openSetting"打开小程序自带的权限设置页面，设置页面后退时触发
bind:opensetting传递最新的设置信息 -->
32 <button wx:if="{{!hasUserLocation}}" open-type="openSetting"
bind:opensetting="openSettingCallback">
33   设置权限
34 </button>
```

2. CSS样式代码实现

```
1  /* pages/shop-list/shop-list.wxss */
2
3  /* 搜索区域 */
4
5  /* 地图区域 */
6  .map-wrapper {
7    position: relative;
8    .map {
9      width: 100%;
10     height: 400rpx;
11   }
12   .map-location {
13     position: absolute;
14     right: 20rpx;
15     bottom: 50rpx;
16     width: 80rpx;
17     height: 80rpx;
18     line-height: 80rpx;
19     text-align: center;
20     background-color: #fff;
21     border-radius: 100%;
22     .iconfont {
23       font-size: 60rpx;
24       color: #555;
25     }
26   }
27 }
28
```

3. JS逻辑代码实现

- 获取map组件实例 `wx.createMapContext(id属性值)`，注意：传递参数值不需要#符号，错误值#shop-map

○ 获取用户当前的地理位置

1. 获取用户当前位置 `wx.getLocation(Object obj)`

wx.getLocation参考: <https://developers.weixin.qq.com/miniprogram/dev/api/location/wx.getLocation.html>

- map 组件使用的经纬度是火星坐标系，所以调用 wx.getLocation 接口需要指定 type 为 gcj02
- 在 app.json 配置 `"requiredPrivateInfos": ["getLocation"]`
- 在 app.json 配置获取用户位置权限时，给用户的提示信息

```
1  "requiredPrivateInfos": [  
2    "chooseLocation",  
3    "getLocation"  
4  ],  
5  "permission": {  
6    "scope.userLocation": {  
7      "desc": "你的位置信息将用于小程序搜索附近门店和配送使用"  
8    }  
9  }
```

2. 如果获取当前位置成功：将位置经纬度坐标作用到地图上

3. 如果获取当前位置失败：页面中显示[设置权限]按钮，用户点击后调用 `wx.getSetting()` 打开设置权限页面

```
1  <button open-type="openSetting" bind:openSetting="openSettingCallback">  
    设置权限</button>
```



设置

允许“梦学谷”使用我的

位置消息

使用时 >

○ js代码实现：

```
1  // pages/shop-list/shop-list.js  
2  
3  import { hideLoading, showLoading } from "@utils/util"  
4  
5  // map地图实例  
6  let mpContext;  
7
```

```
8 Page({
9
10   data: {
11     showMap: true, // 显示地图
12     longitude: 113.927078, // 当前位置经度
13     latitude: 22.52396, // 当前位置纬度
14     // ++++++++以下本次添加的
15     scale: 13, // 地图缩放级别，取值范围为3-20
16     markers: [], // 门店标记点
17     hasUserLocation: true, // 是否有权限获取用户地理位置
18     activeTabName: 'near' // near附近门店, collect收藏门店
19   },
20
21   async onLoad() {
22     // map组件实例，传递map组件上的id属性值，注意：不需要#符号，错误值 #shop-map
23     mpContext = wx.createMapContext('shop-map')
24
25     // 获取当前位置经纬度
26     await this.initUserLocation()
27
28     // 等待获取完当前位置经纬度后，再查询附近门店
29     this.loadShopList()
30   },
31
32   /**查询门店列表数据 */
33   loadShopList() {
34
35   },
36
37   /**
38    * 获取用户当前的地理位置
39    */
40   async initUserLocation() {
41     try {
42       showLoading('', {mask: true})
43       // 1、获取用户当前位置
44       const res = await wx.getLocation({type: 'gcj02'})
45       console.log('initUserLocation', res)
46       // 2、获取成功，更新为当前经纬度
47       const {longitude, latitude} = res
48       this.setData({longitude, latitude, hasUserLocation: true})
49     } catch (error) {
50       console.log('获取用户当前位置失败', error)
51       // 3、获取失败：标识为false无法获取权限，展示[设置权限]按钮
52       this.setData({hasUserLocation: false})
53     } finally {
54       hideLoading()
55     }
56   },
```

```
57
58  /**
59   * 点击标记点：
60   * 默认第1个标记点中'ALWAYS'常显气泡，
61   * 当点击其他标记点时，把第1个气泡改为'ALWAYS'点击显示
62   */
63  clickMarkert(e) {
64    console.log('clickMarkert', e)
65    const { markerId } = e.detail // 标记点id
66  },
67
68  // 重置当前位置：
69  async resetMapLocation() {
70    // 重新获取当前位置坐标（因为可能通过搜索门店定位到了别的位置，所以要定位回当前位置）
71    await this.initUserLocation()
72    // 重新获取列表数据（计算当前位置到各店的距离，并移动到最近的）
73    this.loadShopList()
74  },
75
76  /**
77   * 点击按钮打开小程序设置页面，当页面后退时会回调此方法，并传递用户的最新设置结果
78   * <button open-type="openSetting" bind:opensetting="openSettingCallback">设置权限</button>
79   */
80  openSettingCallback(e) {
81    // console.log('openSettingCallback', e)
82    // authSetting 已向用户请求过的权限
83    const { authSetting } = e.detail
84    // 获取最新的获取当前地理位置权限：true用户已授权过，false用户拒绝授权当前位置，
    // undefined 未向用户发送过授权请求
85    const hasUserLocation = authSetting['scope.userLocation'] || false
86    // 如果有获取位置权限：重新获取用户当前位置和查询门店数据
87    if (hasUserLocation) {
88      this.resetMapLocation()
89    }
90  },
91
92  })
```

门店列表区域实现

1. wxml模板代码实现

```
1  <!-- 门店列表 -->
2  <view class="shop-list-wrapper">
3    <van-tabs id="tabs-box" active="{{ activeTabName }}" bind:change="changeTab">
4      <van-tab title="附近门店" name="near" >
```

```
5      <!-- 有获取用户位置权限时展示 -->
6      <list wx:if="{{ hasUserLocation }}" id="near" tab-name="near" tabs-top="{{
7      tabsTop }}"
8      longitude="{{longitude}}" latitude="{{latitude}}"
9      bind:updateMapMarkers="updateMapMarkers"/>
10     <!-- 没有获取用户位置权限时展示 -->
11     <van-empty wx:else description="请确认手机定位和授权已开启"
12     image="/assets/images/store-map.png">
13     <view class="flex-align-center">
14     <!-- open-type="openSetting"打开小程序自带的权限设置页面，
15     设置页面后退时触发bind:openSetting传递最新的设置信息 -->
16     <button open-type="openSetting"
17     bind:openSetting="openSettingCallback"
18     type="primary" class="setting-btn">设置权限</button>
19     <button bind:tap="resetMapLocation" class="setting-btn">刷新权
20     限</button>
21     </view>
22     </van-empty>
23     </van-tab>
24     <van-tab title="收藏门店" name="collect">
25     <list id="collect" tab-name="collect" tabs-top="{{tabsTop}}"
26     longitude="{{longitude}}" latitude="{{latitude}}"
27     bind:updateMapMarkers="updateMapMarkers"/>
28     </van-tab>
29     </van-tabs>
30   </view>
```

2. CSS样式代码实现

```
1  /* pages/shop-list/shop-list.wxss */
2
3  /* 搜索区域 */
4
5  /* 地图区域 */
6
7  /* 门店列表区域 */
8  .shop-list-wrapper {
9    /* 标签栏背景色*/
10    --tabs-nav-background-color: transparent;
11
12    .setting-btn {
13      width: 280rpx;
14      margin-bottom: 20rpx;
15    }
16
17  }
```

3. JS逻辑代码实现

- data选项添加一个 `tabsTop` 属性：标签到顶部的距离，作用于门店列表组件中指定 `<scroll-view>` 的滚动高度
- 通过 `wx.createSelectorQuery().boundingClientRect(选择器)` 获取标签页元素到顶部的距离。
在自定义组件中，应使用 `this.createSelectorQuery()` 来代替
在 `onLoad` 和 `toggleMap` 函数中调用获取标签页元素到顶部的距离。
- 定义一个 `changeTab` 方法，切换标签页触发此方法，调用查询对应类型的门店列表数据。
- 定义一个 `updateMapMarkers` 方法，接收列表子组件触发此事件函数：更新地图上的门店标记点

```
1 // pages/shop-list/shop-list.js
2 import { hideLoading, showLoading } from "@utils/util"
3
4 // map地图实例
5 let mpContext;
6
7 Page({
8
9   data: {
10     showMap: true, // 显示地图
11     longitude: 113.927078, // 当前位置经度
12     latitude: 22.52396, // 当前位置纬度
13     scale: 13, // 地图缩放级别，取值范围为3-20
14     markers: [], // 门店标记点
15     hasUserLocation: true, // 是否有权获取用户地理位置
16     activeTabName: 'near', // near附近门店, collect收藏门店
17
18     tabsTop: 0, // 标签到顶部的距离， ++++++
19   },
20
21   async onLoad() {
22     // map组件实例，传递map组件上的id属性值，注意：不需要#符号，错误值 #shop-map
23
24     // ++++ 获取标签栏到顶部距离：用于滚动列表 +++++
25     this.queryTabsSelector()
26
27     // 获取当前位置经纬度
28
29     // 等待获取完当前位置经纬度后，再查询附近门店
30   },
31
32
33   // 切换收起/展示地图，++++ 参数2，渲染后调用 this.queryTabsSelector 不要写括号()
34   toggleMap() {
35     this.setData({ showMap: !this.data.showMap }, this.queryTabsSelector)
36   },
```



```
37
38 // 获取标签页到顶部距离：用于指定门店列表的滚动高度
39 queryTabsSelector() {
40   wx.createSelectorQuery().select('#tabs-box').boundingClientRect((res) => {
41     // console.log('tabs-box', res)
42     this.setData({tabsTop: res.top})
43   }).exec() // 最后不要少了.exec()
44 },
45
46 /**切换标签页 */
47 changeTab(e) {
48   // console.log('changeTab', e.detail)
49   const { name } = e.detail
50   this.setData({activeTabName: name})
51
52   // 查询门店列表数据
53   this.loadShopList()
54 },
55
56 /** 更新地图上的门店标记点，列表子组件会触发此事件函数 */
57 updateMapMarkers(e) {
58
59 },
60
61 })
```

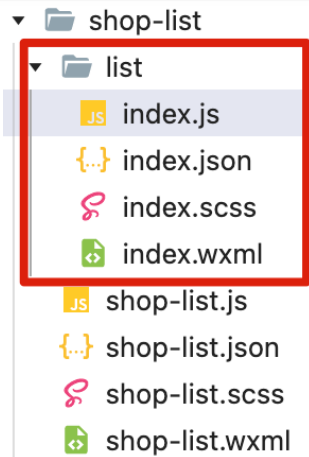
门店列表自定义组件实现

门店列表涉及功能：查询用户已收藏的门店，查询门店列表数据，收藏门店，打开导航，打电话，点击门店跳转到下单页

涉及的集合：mxg_shop、mxg_member

创建门店列表子组件

1. 创建列表子组件：因为门店列表自定义组件只在当前 shop-list 页面中使用，所以可以直接在 shop-list 目录下创建一个子组件 list，如下图：



2. 子组件声明接收父页面组件传递的数据，并开启使用全局样式

```
1 // pages/shop-list/list/index.ts
2 import { getMemberById } from '@api/member'
3
4 Component({
5
6   options: {
7     // 使用全局样式，不然图标等全局样式无效，或配置 styleIsolation: 'apply-shared'
8     addGlobalClass: true,
9   },
10
11   /** 组件的属性列表 */
12   properties: {
13     tabName: String, //当前所在标签名: 'near'附近门店, 'collect'收藏门店
14     longitude: Number, // 地理位置经度
15     latitude: Number, // 地理位置纬度
16     tabsTop: { // 标签栏到顶部距离
17       type: Number,
18       value: 260,
19     }
20   },
21
22 })
```

3. 在 shop-list/shop-list.json 配置文件中注册子组件: `"list": "list/index"`

```
1 {
2   "disableScroll": true,
3   "navigationBarTitleText": "门店列表",
4   "navigationBarBackgroundColor": "#f9f9f9",
5   "usingComponents": {
6     "van-search": "@vant/weapp/search/index",
7     "van-empty": "@vant/weapp/empty/index",
8     "van-tabs": "@vant/weapp/tabs/index",
9     "van-tab": "@vant/weapp/tab/index",
10    "list": "list/index"
11  }
12 }
```

获取当前用户已收藏的门店

查询当前用户收藏的所有店铺id：用于高亮显示收藏图标

- 当前用户已收藏的所有门店ID 保存在 `mxg_member` 集合中的 `collectShopIds` 数组字段中，所以直接调用 `ap/member.js` 中的 `getMemberById` 方法查询即可获取

```
1 // pages/shop-list/list/index.ts
2 import { msg, hideLoading, showLoading } from "@utils/util"
3 import { getMemberById } from '@api/member'
4
5 Component({
6
7   /** 组件的初始数据 */
8   data: {
9     loading: false, // 是否加载中
10    collectShopIds: [], // 所有收藏的店铺ids
11    shopList: [], // 列表数据
12  },
13
14
15   /** 组件的方法列表 */
16   methods: {
17
18     /** 获取当前用户收藏的所有门店id */
19     async getCollectShopIds() {
20       const { member } = getApp().current()
21       if (!member || !member._id) return
22       // 查询会员信息, data.collectShopIds 是用户收藏的所有门店id
23       const { data } = await getMemberById(member._id)
24       this.setData({ collectShopIds: data.collectShopIds || [] })
25       // console.log('收藏门店ids', data)
26     },
27   },
28 })
```

```
27
28  /**
29   * 查询门店列表：
30   * 1、查询当前用户收藏的所有店铺id：用于高亮显示收藏图标
31   * 2、根据当前用户位置，查询距离由近到远的门店列表数据
32   * 3、更新地图上的门店标记点：触发父组件事件
33   */
34  async loadShopList() {
35    try {
36      this.setData({loading: true})
37
38      // 1、查询当前用户收藏的所有店铺id
39      await this.getCollectShopIds()
40
41      // 2、根据当前用户位置，查询距离由近到远的门店列表数据
42
43      // 3、更新地图上的门店标记点：触发父组件事件
44
45    } catch (error) {
46      console.log('查询门店列表异常', error)
47      msg('门店查询失败', {icon: 'error'})
48    } finally {
49      this.setData({loading: false})
50    }
51  },
52
53 }
```

- 在父组件页面 `pages/shop-list/shop-list.js` 中的 `loadShopList` 中调用列表子组件的方法 `loadShopList` 方法

```
1  /**查询门店列表数据 */
2  loadShopList() {
3    const {activeTabName} = this.data
4    // 获取列表子组件实例
5    const listComponent = this.selectComponent('#' + activeTabName)
6    // 调用子组件方法查询列表数据
7    listComponent?.loadShopList()
8  },
```

使用地理位置查询功能--查询附近门店列表数据

- 创建云函数 `getNearShopList`，实现根据当前用户位置，查询距离由近到远的门店列表数据
 - 使用 `aggregate().geoNear(options)` 方法，按从近到远的顺序，找出字段值在给定点的附近记录
 - 参考：<https://developers.weixin.qq.com/miniprogram/dev/wxcloudservice/wxcloud/reference-sdk-api/database/aggregate/Aggregate.geoNear.html>

要使用地理位置查询功能时，必须建立地理位置索引，如下在云控制台建立索引：



添加索引

索引名称:

索引属性:
☐ 唯一
☒ 非唯一

索引字段: 地理位置 +

取消 确定

代码实现如下：

```

1  // 云函数入口文件
2  const cloud = require('wx-server-sdk')
3
4  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6  const db = cloud.database()
7  const _ = db.command
8
9  /**
10   * 云函数入口函数：查询用户的附近门店
11   * 使用 aggregate().geoNear(options)方法，按从近到远的顺序，找出字段值在给定点的附近记录
12   * 参考：
13   * https://developers.weixin.qq.com/miniprogram/dev/wxcloudservice/wxcloud/reference-sdk-api/database/aggregate/Aggregate.geoNear.html
14   */
15  exports.main = async (event, context) => {
16      // longitude经度, latitude纬度
17      const {longitude, latitude, shopIds} = event
18      // 封装查询条件
19      const query = {}
20      if (shopIds && shopIds.length>0) query._id = _.in(shopIds)
    
```

```
20
21   const res = await db.collection('mxg_shop').aggregate()
22     .geoNear({
23       spherical: true, // 必填, 值为 true
24       // 用于判断距离的点: db.Geo.Point(longitude经度, latitude纬度) 构造一个地理位置 "点", 参数不要错了
25       near: db.Geo.Point(longitude, latitude),
26       distanceField: 'distance', // 存放距离计算结果的字段名, 结果单位: 米
27       key: 'location', // 指定地理位置索引字段名, 若集合只有一个`location`地理位置索引的字段, 则不需填
28       includeLocs: 'location', // 若只有 location 一个是地理位置字段, 则不需填
29       query: query // 条件查询, 语法同where
30     })
31     .end() // 不要少了
32
33   return res
34 }
```

2. 封装调用云函数API方法

```
1  //   miniprogram/api/shop.js
2
3  /**
4   * 查询附近的门店列表
5   * @param {number} longitude 经度
6   * @param {number} latitude 纬度
7   * @param {Array} shopIds 收藏门店ids数组
8   */
9  export async function getNearShopList(longitude, latitude, shopIds) {
10    const { result } = await wx.cloud.callFunction({
11      name: 'getNearShopList',
12      data: {
13        longitude,
14        latitude,
15        shopIds
16      }
17    })
18    return result
19  }
20
```

3. 调用查询列表数据API方法

```
1  //   pages/shop-list/list/index.ts
2  import { msg, hideLoading, showLoading } from '@/utils/util'
3  import { getMemberById } from '@/api/member'
4  import { getNearShopList } from '@/api/shop'
5
```

```
6 Component({
7
8   options: {
9     // 使用全局样式，不然图标等全局样式无效，或配置 styleIsolation: 'apply-shared'
10    addGlobalClass: true,
11  },
12
13  /** 组件的属性列表*/
14  properties: {
15    tabName: String, //当前所在标签名: 'near'附近门店, 'collect'收藏门店
16    longitude: Number, // 地理位置经度
17    latitude: Number, // 地理位置纬度
18    tabsTop: { // 标签栏到顶部距离
19      type: Number,
20      value: 260,
21    }
22  },
23
24  /** 组件的初始数据 */
25  data: {
26    loading: false, // 是否加载中
27    collectShopIds: [], // 所有收藏的店铺ids
28    shopList: [], // 列表数据
29  },
30
31
32  /** 组件的方法列表*/
33  methods: {
34
35    /** 获取当前用户收藏的所有门店id */
36    async getCollectShopIds() {
37      const { member } = getApp().current()
38      if (!member || !member._id) return
39      // 查询会员信息, data.collectShopIds 是用户收藏的所有门店id
40      const { data } = await getMemberById(member._id)
41      this.setData({ collectShopIds: data.collectShopIds || [] })
42      console.log('收藏门店ids', data)
43    },
44
45    /**
46     * 查询门店列表:
47     * 1、查询当前用户收藏的所有店铺id: 用于高亮显示收藏图标
48     * 2、根据当前用户位置, 查询距离由近到远的门店列表数据
49     * 3、更新地图标记点
50     */
51    async loadShopList() {
52      try {
53        this.setData({loading: true})
54      }
55    }
56  }
57 }
```

```
55      // 1、查询当前用户收藏的所有店铺id
56      await this.getCollectShopIds()
57
58      // 2、查询门店列表数据
59      const { tabName, longitude = 10, latitude = 10, collectShopIds } =
this.data
60      let shopList = [] // 门店列表数据
61      if (tabName == 'near') {
62          // 附近门店
63          const {list} = await getNearShopList(longitude, latitude)
64          shopList = list
65      } else if (collectShopIds && collectShopIds.length > 0) {
66          // 收藏门店：有收藏门店ids，则加上收藏门店ids作为查询条件
67          const {list} = await getNearShopList(longitude, latitude,
collectShopIds)
68          shopList = list
69      }
70
71      // 数据格式化：单位转换、添加 collected 收藏属性亮度用
72      shopList.forEach(item => {
73          // 距离单位转换，单位默认是 米，四舍五入
74          const distance = Math.round(item.distance || 0)
75          item.distanceStr = distance < 1000 ? distance + 'm' :
(distance / 1000).toFixed(2) + 'km'
76          // 添加 collected 收藏属性，存在则true表示收藏
77          item.collected = collectShopIds.includes(item._id)
78      })
79      // console.log('门店列表', shopList)
80
81      // 更新状态数据
82      this.setData({shopList})
83
84      // 3、更新地图上的门店标记点：触发父组件事件
85      this.triggerEvent('updateMapMarkers', {shopList})
86      } catch (error) {
87          console.log('查询门店列表异常', error)
88          msg('门店查询失败', {icon: 'error'})
89      } finally {
90          this.setData({loading: false})
91      }
92  },
93
94  }
95  })
```


更新地图上的门店标记点

在 shop-list.js 中接收传递的门店列表数据：更新地图标记点 markers

```
1  /** 更新地图上的门店标记点 */
2  updateMapMarkers(e) {
3    const {shopList} = e.detail
4    const markers = shopList.map((item, index) => {
5      return {
6        id: index,
7        longitude: item.location.coordinates[0],
8        latitude: item.location.coordinates[1],
9        width: '80rpx',
10       height: '80rpx',
11       iconPath: '/assets/images/location.png',
12       callout: { // 标记点上方的气泡窗口，点击显示气泡在真机上才有用
13         content: item.name + '\n' + item.address,
14         display: index == 0 ? 'ALWAYS': 'BYCLICK', // 'BYCLICK':点击显示;
15         'ALWAYS':常显，默认第1个标记点常显气泡，当点击其他标记点时，把第1个改为'ALWAYS'点击显示
16         borderRadius: 5,
17         padding: 5,
18         textAlign: 'center',
19         bgColor: '#000000b3',
20         color: '#fff'
21       }
22     })
23
24     // scale 将地图缩放改为初始值
25     this.setData({ markers, scale: 13 })
26
27     // 将地图中心移置第一个标记点，moveToLocation 方法使用必须在map组件使用show-location
28     setTimeout(() => { // 解决真机上不会移动：在触发moveToLocation方法上给一个延时处理
29       mpContext?.moveToLocation({
30         longitude: shopList[0]?.location.coordinates[0],
31         latitude: shopList[0]?.location.coordinates[1]
32       })
33     }, 300)
34   },
35
36   /**
37    * 点击标记点：
38    * 默认第1个标记点中'ALWAYS'常显气泡，
39    * 当点击其他标记点时，把第1个气泡改为'ALWAYS'点击显示
40    */
41   clickMarkert(e) {
42     // console.log('clickMarkert', e)
43     const { markerId } = e.detail
44     // 点击第1个标记是不处理，
```



```
45 // 点击非第1个标记，且第1个是'ALWAYS'点击显示，则改为'BYCLICK'点击显示
46 if (markerId !== 0 && this.data.markers[0].callout.display === 'ALWAYS') {
47   this.setData({
48     'markers[0].callout.display': 'BYCLICK'
49   })
50 }
51 },
52
```

列表组件效果实现



1. 组件配置中注册 `van-empty` 组件

```
1 {
2   "component": true,
3   "usingComponents": {
4     "van-empty": "@vant/weapp/empty/index"
5   }
6 }
```

2. 列表组件模板代码:

```
1 <!-- pages/shop-list/list/index.wxml -->
2 <scroll-view scroll-y style="height: calc(100vh - {{tabsTop}}px);">
3   <view class="shop-list-box">
4     <!-- 加载中且没有列表数据时显示 -->
5     <view wx:if="{{loading && shopList.length <= 0}}" class="loading">加载中...
6   </view>
```

```
6      <view class="shop-item flex-row-align-center" wx:for="{{ shopList }}"
wx:key="index">
7          <view class="shop-item-left flex-1" bind:tap="toShoppingPage" data-
shop="{{item}}">
8              <view >
9                  <!-- <text class="shop-name">肯德基（万象广场店） </text> -->
10                 <text class="shop-name">{{item.name}}</text>
11             </view>
12             <view wx:if="{{item.businessTimes && item.businessTimes.length ==
2}}">
13                 <!-- <text>营业时间：09:00-23:00</text> -->
14                 <text>营业时间：{{item.businessTimes[0]}}-{{item.businessTimes[1]}}
</text>
15             </view>
16             <view>
17                 <text>直线距离{{item.distanceStr || '未知'}} | </text>
18                 <text>{{item.address}}</text>
19             </view>
20         </view>
21         <view class="shop-item-right">
22             <view class="shop-like-view" bind:tap="handleCollect" data-item="{{
item}}">
23                 <text class="iconfont {{ item.collected ? 'icon-like_fill': 'icon-
like'}}"></text>
24             </view>
25             <view class="go-order-view" bind:tap="toShoppingPage">
26                 <text>去下单</text>
27             </view>
28             <view class="other-opt-icons">
29                 <text class="iconfont icon-daohang" bind:tap="handleLocation" data-
item="{{item}}"></text>
30                 <text class="iconfont icon-dianhua" bind:tap="handleCall" data-
item="{{item}}"></text>
31             </view>
32         </view>
33     </view>
34     <!-- 未加载中且没有列表数据时显示 -->
35     <van-empty wx:if="{{!loading && shopList.length <= 0}}" description="暂无
门店"></van-empty>
36 </view>
37 </scroll-view>
```

3. 列表组件样式代码：

```
1 .shop-list-box {
2     padding: 20rpx 26rpx 200rpx 26rpx;
3     .loading {
```

```
4     color: #555;
5     font-size: 26rpx;
6     text-align: center;
7     margin-bottom: 20rpx;
8 }
9 .shop-item {
10     background-color: #fff;
11     border-radius: 20rpx;
12     padding: 26rpx;
13     box-shadow: 0 0 .3rem #e8e8e8;
14     margin-bottom: 20rpx;
15 }
16 .shop-item-left {
17     padding-right: 26rpx;
18     border-right: var(--basic-border);
19     color: #666;
20     font-size: 24rpx;
21     line-height: 40rpx;
22     .shop-name {
23         font-weight: bold;
24         color: #222;
25         font-size: 30rpx;
26     }
27 }
28 .shop-item-right {
29     padding-left: 26rpx;
30     text-align: center;
31     .shop-like-view {
32         .iconfont {
33             font-size: 50rpx;
34             color: #555;
35         }
36         .icon-like_fill {
37             color: red;
38         }
39     }
40     .go-order-view {
41         font-size: 24rpx;
42         color: #555;
43         padding: 10rpx 0;
44     }
45     .other-opt-icons {
46         .iconfont {
47             padding: 14rpx;
48             font-size: 30rpx;
49             line-height: 70rpx;
50             background-color: #f0f0f0;
51             color: red;
52             border-radius: 100%;
```

```
53      &:nth-child(2n) {  
54          margin-left: 26rpx;  
55      }  
56  }  
57  }  
58  }  
59  }
```

收藏门店功能实现

点击收藏图标：添加收藏或取消收藏



1. 创建云函数：实现更新会员的收藏门店ID数据

- 添加收藏：使用 `db.command.addToSet` 添加元素到数组中：添加店铺id到数组中
- 取消收藏：使用 `db.command.pull` 从数组中移除指定元素：从数组移除店铺id

```
1  // 云函数入口文件  
2  const cloud = require('wx-server-sdk')  
3  
4  cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境  
5  
6  const db = cloud.database()  
7  const _ = db.command  
8  
9  // 云函数入口函数：更新会员的收藏店铺ids  
10 exports.main = async (event, context) => {  
11     const {memberId, shopId, collected} = event  
12  
13     // 不要少了 await  
14     return await db.collection('mxg_member')  
15         .doc(memberId)  
16         .update({  
17             data:{
```

```
18      // 添加收藏：添加店铺id到数组中（addToSet 添加元素到数组中），
19      // 取消收藏：从数组移除（pull 从数组中移除指定元素）
20      collectShopIds: collected ? _.addToSet(shopId) : _.pull(shopId)
21    }
22  })
23
24 }
```

2. 封装调用云函数的API方法

```
1  // miniprogram/api/shop.js
2
3  /**
4   * 更新会员的收藏店铺ids
5   * @param memberId 会员id
6   * @param shopId 添加或取消收藏的店铺id
7   * @param collected true添加收藏，false取消收藏
8   */
9  export function updateCollectShopIds(memberId, shopId, collected) {
10    return wx.cloud.callFunction({
11      name: 'updateCollectShopIds',
12      data: {
13        memberId,
14        shopId,
15        collected
16      }
17    })
18  }
```

3. 列表组件调用API方法：

```
1  // pages/shop-list/list/index.ts
2  import { msg, hideLoading, showLoading } from '@utils/util'
3  import { getMemberById } from '@api/member'
4  import { getNearShopList, updateCollectShopIds } from '@api/shop'
5
6  Component({
7
8    /** 组件的方法列表*/
9    methods: {
10
11      // 点击收藏图标
12      async handleCollect(e) {
13        try {
14          showLoading()
15          // 未登录，则跳转登录页
```

```
16     const { logged, member } = getApp().current(true)
17     if (!logged) return
18
19     const { item } = e.currentTarget.dataset
20     // 1、取反：原来是收藏则变为取消收藏
21     const collected = !item.collected
22     // 2、更新会员收藏的门店id
23     await updateCollectShopIds(member._id, item._id, collected)
24     // 3、重新刷新列表数据
25     await this.loadShopList()
26     msg(collected ? '收藏成功' : '已取消收藏', {icon: 'success'})
27   } catch (error) {
28     console.error('收藏失败', error)
29     msg('收藏失败', {icon: 'error'})
30   } finally {
31     hideLoading()
32   }
33 },
34
35 }
36 })
```

导航定位/打电话/前往下单页

1. 在列表组件JS文件中, 实现以下功能:

- 打开导航定位地图: `wx.openLocation`
- 调用打电话API: `wx.makePhoneCall`
- 点击某个门店前往下单页: `toShoppingPage`

```
1 // pages/shop-list/list/index.ts
2
3 Component({
4
5   methods: {
6
7     // 打开导航定位
8     handleLocation(e) {
9       const { name, address, location } = e.currentTarget.dataset.item
10      wx.openLocation({
11        name,
12        address,
13        longitude: location.coordinates[0],
14        latitude: location.coordinates[1],
15      })
16    },
17  },
18 })
```

```
18 // 打电话
19 handleCall(e) {
20   const { contactPhone } = e.currentTarget.dataset.item
21   if (!contactPhone) return msg('无联系电话')
22   // 调用打电话
23   wx.makePhoneCall({
24     phoneNumber: contactPhone
25   })
26 },
27
28
29 // 前往下单页面
30 toShoppingPage (e) {
31   const { shop } = e.currentTarget.dataset
32   toShoppingPage({shop: shop})
33 },
34
35 }
36
37 })
```

2. 在下单页面 `shopping.js` 中的 `onShow` 生命周期函数中，接收传递的门店数据

```
1 // pages/shopping/shopping.js
2
3 Page({
4
5   data: {
6     shop: null, // 下单店铺
7     takeType: 1, // 取单类型：1自取，2外送
8   },
9
10  onShow() {
11    // shop: 选择的门店
12    const { shop } = getApp().pageData || {}
13    if (shop) this.setData({ shop })
14    // 获取后清空
15    getApp().pageData = null
16  },
17
18 })
```

第八章 购物页面和收货地址页面开发

点单页面: pages/shopping/shopping

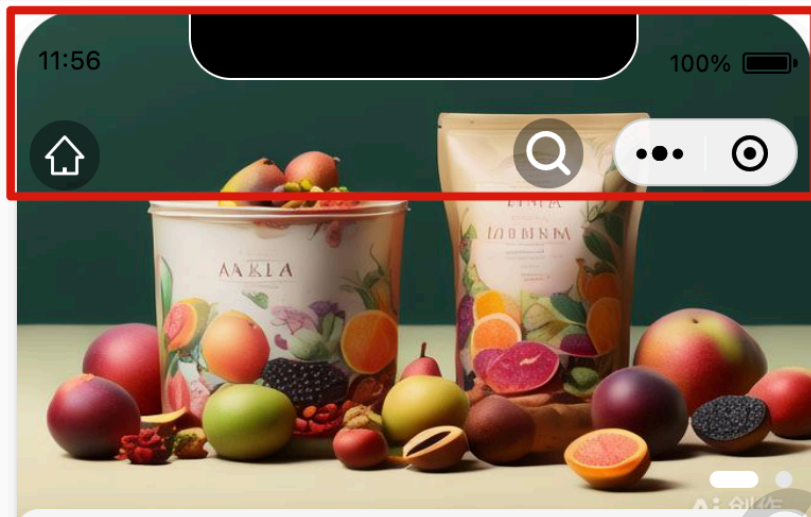


页面自定义导航栏

自定义导航栏效果1: 有背景色, 有搜索框



自定义导航栏效果2: 无背景色, 有搜索图标



实现自定义导航栏组件

1. 创建导航栏自定义组件 components/navbar/index，模板代码如下：

```
1 <!-- components/navbar/index.wxml -->
2
3 <!-- 顶部导航栏：fixed 是否固定到顶部，showSearchInput 不显示搜索框时背景色透明 -->
4 <view id="navbar" class="navbar-container {{fixed ? 'navbar--fixed': ''}}
5   {{showSearchInput ? '' : 'navbar--transparent'}}"
6   style="padding-top: {{buttonTop}}px; background: {{background}};">
7   <view class="navbar-content flex-row-align-center">
8     <view bind:tap="handleBack" class="navbar-left" hover-class="view-hover"
9       hover-stay-time="800">
10       <!-- 后退图标 -->
11       <text wx:if="{{haveBack}}" class="iconfont icon-arrow-left"></text>
12       <!-- 首页图标 -->
13       <text wx:else class="iconfont icon-home"></text>
14     </view>
15     <!-- 搜索区域展示 -->
16     <view wx:if="{{showSearch}}" class="navbar-center-input flex-1"
17       style="margin-right: {{buttonRight}}px;">
18       <!-- 吸顶时显示：输入框 -->
19       <view wx:if="{{showSearchInput}}" class="center-input"
20         bind:tap="handleDisabled">
21         <text class="iconfont icon-search"></text>
22         <input model:value="{{keyword}}" catch:confirm="confirmInput"
23           disabled="{{disabled}}" focus="{{focus}}" class="input"
24           placeholder-class="input-placeholder" placeholder="请输入搜索关键
25         词" />
26       <!-- 清空x图标 -->
27       <text wx:if="{{keyword && keyword.length > 0}}"
28         catch:tap="handleClear"
```

```
25         class="iconfont icon-delete_fill"></text>
26         <view wx:if="{{showSearchBtn}}" catch:tap="confirmInput"
class="search-btn"
27             hover-class="view-hover" hover-stay-time="500">搜索</view>
28     </view>
29     <!-- 未吸顶显示：搜索图标 -->
30     <view wx:else class="navbar-right" hover-class="view-hover"
31         hover-stay-time="800" bind:tap="handleDisabled">
32         <text class="iconfont icon-search"></text>
33     </view>
34 </view>
35
36 <!-- 导航标题 -->
37 <view wx:else class="navbar-center-title">
38     <text>{{ title }}</text>
39 </view>
40 </view>
41 </view>
42
43 <!-- 固定顶部时，占高 -->
44 <view wx:if="{{fixedFullHeight}}" style="height: calc(var(--navbar-height,
35px) + {{buttonTop}}px);"></view>
```

2. 样式文件

```
1 .navbar-container {
2     background-color: var(--navbar-background-color, #fff);
3     height: var(--navbar-height, 35px);
4     line-height: var(--navbar-height, 35px);
5     text-align: center;
6     z-index: 100;
7     position: relative;
8
9     // 固定到顶部
10    &.navbar--fixed {
11        position: fixed;
12        left: 0;
13        top: 0;
14        width: 100%;
15    }
16
17    // 透明背景
18    &.navbar--transparent {
19        background-color: transparent !important;
20        .navbar-left, .navbar-right {
21            border-radius: 100%;
22            background-color: rgba(0, 0, 0, 0.4);
23            color: #fff !important;
24            z-index: 100;
```

```
25     .iconfont {
26         color: #fff;
27     }
28 }
29 }
30
31 .navbar-content {
32     height: 100%;
33     padding: 0 6px;
34     position: relative;
35 }
36
37 .navbar-center-title {
38     position: absolute;
39     height: inherit;
40     width: 100%;
41     text-align: center;
42     // 解决上面使用position: absolute;后，图标按钮无法点击
43     z-index: 1;
44     pointer-events: none;
45 }
46
47 .navbar-left, .navbar-right {
48     width: 66rpx;
49     height: 66rpx;
50     display: flex;
51     align-items: center;
52     justify-content: center;
53     float: right;
54     // 解决标题使用position: absolute;后，图标按钮无法点击
55     z-index: 10;
56     pointer-events: auto;
57     .iconfont {
58         font-weight: bold;
59         font-size: 46rpx;
60     }
61     .icon-home {
62         font-weight: normal;
63     }
64 }
65
66 .navbar-center-input {
67     height: inherit;
68     font-size: 28rpx;
69     margin-left: 10rpx;
70     .center-input {
71         display: flex;
72         align-items: center;
73         height: inherit;
```

```
74     text-align: left;
75     position: relative;
76     background-color: #eeeeee;
77     border-radius: 10rpx;
78     overflow: hidden;
79     .iconfont {
80         color: #8d8d8d;
81         font-size: 30rpx;
82         padding: 0 16rpx;
83     }
84     .icon-delete_fill {
85         font-size: 40rpx;
86     }
87     .input {
88         height: inherit;
89         width: 100%;
90         font-size: 30rpx;
91         margin-right: 4rpx;
92     }
93     .input-placeholder {
94         color: #8d8d8d;
95         font-size: 30rpx;
96     }
97     .search-btn {
98         white-space: nowrap;
99         padding: 0 16rpx;
100        background-color: var(--primary-color);
101        color: #fff;
102        font-size: 28rpx;
103    }
104    }
105    .navbar-icon-search {
106        text-align: right;
107    }
108    }
109 }
```

3. js逻辑文件

```
1  // components/navbar/navbar.js
2
3  import { navBack } from '@/utils/util'
4  import { toIndexPage } from '@/utils/page'
5
6  Component({
7
8      options: {
```

```
9      // 使用全局样式，不然图标无效，或配置 styleIsolation: 'apply-shared'
10      addGlobalClass: true,
11    },
12
13    properties: {
14      fixed: { // 固定到顶部
15        type: Boolean,
16        value: true
17      },
18      fixedFullHeight: { // 固定到顶部时，是否占满高度
19        type: Boolean,
20        value: true
21      },
22      showSearch: { // 是否显示搜索框
23        type: Boolean,
24        value: true
25      },
26      showSearchInput: { // 是否显示输入框，反之显示搜索图标
27        type: Boolean,
28        value: true
29      },
30      showSearchBtn: { // 是否显示搜索按钮
31        type: Boolean,
32        value: false
33      },
34      background: { // 导航栏背景色
35        type: String,
36        value: '#f9f9f9'
37      },
38      disabled: { // 输入框禁用
39        type: Boolean,
40        value: true
41      },
42      focus: Boolean, // 是否自动获取焦点
43      title: String, // 导航栏标题
44      keyword: String, // 输入框内容
45      height: Number, // 导航栏高度，使用双向绑定 model:height 获取更新的值
46    },
47
48    /**
49     * 组件的初始数据
50     */
51    data: {
52      haveBack: true, // 是否有返回按钮，true 有 false 没有 若从分享页进入则为 false
53      buttonTop: 0, // 胶囊到顶部距离
54      buttonRight: 0, // 胶囊占宽
55    },
56
57    lifetimes: {
```

```
58
59  /** 在组件完全初始化完毕、进入页面节点树后触发*/
60  attached() {
61    // 大于1个页面，则有后退按钮
62    const haveBack = getCurrentPages().length > 1
63
64    // 右上角胶囊
65    const menu = wx getMenuButtonBoundingClientRect()
66    this.setData({
67      haveBack,
68      buttonTop: menu.top,
69      buttonRight: menu.width + 15, // 胶囊占宽
70    })
71
72    // 获取状态栏+导航栏高度，在自定义组件应使用 `this.createSelectorQuery`，页面中
    使用 `wx.createSelectorQuery`
73    this.createSelectorQuery().select('#navbar').boundingClientRect((res) =>
    {
74      this.setData({height: res.height})
75    }).exec() // 最后不要少了.exec()
76
77  },
78
79  },
80
81  /**
82   * 组件的方法列表
83   */
84  methods: {
85
86    // 点击左上角后退或首页图标
87    handleBack(e) {
88      if (this.data.haveBack) {
89        navBack() // 后退
90      } else {
91        toIndexPage() // 首页
92      }
93    },
94
95    // 输入框不可用时，点击触发
96    handleDisabled(e) {
97      this.triggerEvent('clickDisabled', e)
98    },
99
100    // 输入框确认时触发
101    confirmInput() {
102      const {keyword} = this.data;
103      this.setData({keyword})
104      this.triggerEvent('search', {keyword: keyword.trim()})
```



```
105     },  
106  
107     // 清空图标被点击时触发  
108     handleClear() {  
109         this.setData({keyword: '', focus: true})  
110         this.triggerEvent('clear')  
111     },  
112  
113     },  
114  
115  
116 })
```

点单页面引用自定义导航栏

1. 点单页面配置自定义导航栏 和 注册自定义导航组件navbar

pages/shopping/shopping.json

```
1 {  
2   "navigationStyle": "custom",  
3   "usingComponents": {  
4     "navbar": "/components/navbar/index",  
5   }  
6 }
```

2. 在 pages/shopping/shopping.wxml 中使用 `<navbar/>` 自定义导航组件

```
1 <!--pages/shopping/shopping.wxml-->  
2  
3 <!-- 顶部导航栏 -->  
4 <navbar  
5   fixed  
6   fixedFullHeight="{{false}}"  
7   model:height="{{navHeight}}"  
8   showSearchInput="{{showSearchInput}}"  
9   bind:clickDisabled="toSearchPage"  
10 />  
11  
12 <view style="height: 1800px;">测试占用高度</view>  
13
```

3. 在 pages/shopping/shopping.js 中使用实现逻辑


```
1 // pages/shopping/shopping.js
2
3 import { navTo } from '@utils/util'
4
5 Page({
6
7   data: {
8     navHeight: 0, // 导航栏高度，用于计算分类列表滚动吸顶停留top值
9     showSearchInput: false, //是否显示搜索输入框：true显示输入框，false显示搜索图标
10   },
11
12   // 跳转搜索页面
13   toSearchPage() {
14     navTo("/pages/search/search")
15   },
16
17 })
```

4. 在app.json文件的 pages 选项中配置搜索页面

```
1 {
2   "pages": [
3     //。。。省略其他页面配置
4
5     "pages/search/search"
6   ],
7 }
```

头部轮播图广告功能



1. 在 pages/shopping/shopping.json 注册轮播图组件

```
1 {
2   "navigationStyle": "custom",
3   "usingComponents": {
4     "navbar": "/components/navbar/index",
5     "banner": "/components/banner/index"
6   }
7 }
```

2. 点单页面模板中使用轮播图组件 <banner/>

```
1 <!--pages/shopping/shopping.wxml-->
2
3 <!-- 顶部导航栏 -->
4 <navbar
5   fixed
6   fixedFullHeight="{{false}}"
7   model:height="{{navHeight}}"
8   showSearchInput="{{showSearchInput}}"
9   bind:clickDisabled="toSearchPage"
10 />
11
12 <!-- 轮播图 -->
13 <banner list="{{ bannerList }}" height="500rpx" dots-bottom="50rpx"/>
14
15
16 <view style="height: 1800px;">测试占用高度</view>
```

3. 查询头部轮播图广告数据

```
1 // pages/shopping/shopping.js
2
3 import { getAdvertList } from '@api/advert'
4
5 Page({
6
7   data: {
8     bannerList: [], // 头部轮播图
9   },
10
11   onLoad() {
12     this.loadAdvertList()
13   },
14
15 })
```

```
15 // 查询头部轮播图广告数据
16 async loadAdvertList() {
17   // 不要少了 await ; '3'代表顶部轮播图广告
18   const { data } = await getAdvertList('3')
19   this.setData({ bannerList: data })
20 },
21
22 })
```

点单信息区域效果实现

自取效果：



外送效果：



1. 在 pages/shopping/shopping.json 中注册 van-sticky 吸顶组件和 van-tag 标签组件

```
1 {
2   "navigationStyle": "custom",
3   "usingComponents": {
4     "navbar": "/components/navbar/index",
5     "banner": "/components/banner/index",
6     "van-sticky": "@vant/weapp/sticky/index",
7     "van-tag": "@vant/weapp/tag/index"
8   }
9 }
```

2. 实现模板代码：

```
1 <!--pages/shopping/shopping.wxml-->
```

```
2
3 <!-- 顶部导航栏 -->
4 <!-- 轮播图 -->
5
6 <!-- 点单信息区域 -->
7 <view id="shop-info-id" class="take-order-wrapper">
8   <!-- 吸顶 -->
9   <van-sticky offset-top="{{ navHeight }}" bind:scroll="scrollShopInfoSticky">
10     <view class="shop-tab-box" style="{{showSearchInput ? 'border-radius: 0;':''}}">
11       <view class="shop-content flex-space-between-center">
12         <view class="shop-address flex-column text-one">
13           <!-- 自取 -->
14           <block wx:if="{{takeType == 1}}" >
15             <view class="flex-row-align-center" bind:tap="toShopListPage">
16               <view class="address-img"></view>
17               <view class="flex-row-align-center">
18                 <text class="address-name text-one">{{shop.name || '云飞路餐
厅'}}</text>
19                 <text class="iconfont icon-arrow-right"></text>
20               </view>
21             </view>
22             <view class="shop-address-info flex-row-align-center text-one">
23               <text>自取</text>
24               <text space="nbsp"> | 直线距离{{shop.distanceStr || '未知'}} |
</text>
25               <text>{{shop.address || '门店位置'}}</text>
26             </view>
27           </block>
28           <!-- 外送 -->
29           <block wx:else>
30             <view class="flex-row-align-center" bind:tap="toSelectAddress">
31               <view class="address-img take-out"></view>
32               <view class="flex-row-align-center">
33                 <text class="address-name text-one">{{ address ?
address.locationName + ', ' + address.streetNumber : '请选择收货地址'}}</text>
34                 <text class="iconfont icon-arrow-right"></text>
35               </view>
36             </view>
37             <view class="shop-address-info flex-row-align-center text-one">
38               <text space="nbsp">外送 | </text>
39               <van-tag type="danger" plain bind:tap="toShopListPage">
40                 <text class="iconfont icon-qiehuan"/>
41                 {{shop.name || '请选择门店'}}
42               </van-tag>
43             </view>
44           </block>
45         </view>
46         <view class="take-type-box flex-row-align-center">
```

```
47         <view class="take-type-node {{takeType == 1 ? 'is-checked': '' }}"
48             bind:tap="changeTakeType" data-take-type="{{1}}">
49             <text>自取</text>
50         </view>
51         <view class="take-type-node {{takeType == 2 ? 'is-checked': '' }}"
52             bind:tap="changeTakeType" data-take-type="{{2}}" >
53             <text>外送</text>
54         </view>
55         <view class="take-type-bg {{takeType == 2 ? 'is-checked': '' }}">
56     </view>
57 </view>
58 <!-- 标签栏 -->
59 <view class="tab-bar flex-row">
60     <view class="tabs-box flex-row">
61         <view class="tab-item tab-item-actived">
62             <text>经典菜单</text>
63         </view>
64     </view>
65 </view>
66 </view>
67 </van-sticky>
68 </view>
69
70 <view style="height: 1800px;">测试占用高度</view>
```

3. 点单信息区域样式实现

```
1  /* pages/shopping/shopping.wxss */
2
3  /* 点单信息区域 */
4  .take-order-wrapper {
5      position: relative;
6      margin-top: -30rpx;
7      .shop-tab-box {
8          background-color: #f9f9f9;
9          width: 100%;
10         height: 200rpx;
11         border-top-left-radius: 20rpx;
12         border-top-right-radius: 20rpx;
13         border-bottom: 1rpx solid #e4e4e4;
14     }
15     .shop-content {
16         height: 140rpx;
17         padding: 0 26rpx;
18         .shop-address {
19             .address-img {
```

```
20         width: 50rpx;
21         height: 50rpx;
22         margin-right: 10rpx;
23     }
24     .address-name {
25         font-size: 40rpx;
26         font-weight: bold;
27         max-width: 380rpx;
28     }
29     .iconfont {
30         font-size: 34rpx;
31         font-weight: bold;
32     }
33 }
34 .shop-address-info {
35     font-size: 24rpx;
36     color: #999999;
37     text:first-child {
38         color: #111111;
39     }
40     .iconfont {
41         font-size: 22rpx;
42         font-weight: bold;
43         color: inherit !important;
44     }
45 }
46 }
47
48 .take-type-box {
49     background-color: #fff;
50     border-radius: 20px;
51     border: 1px solid rgba(0, 0, 0, 0.1);
52     padding: 4rpx;
53     font-size: 28rpx;
54     position: relative;
55     .take-type-node {
56         font-size: inherit;
57         width: 90rpx;
58         height: 50rpx;
59         line-height: 50rpx;
60         text-align: center;
61         z-index: 100;
62     }
63     .take-type-bg {
64         position: absolute;
65         background-color: var(--primary-color);
66         width: 90rpx;
67         height: 50rpx;
68         border-radius: 20px;
```

```
69     box-shadow: 0px 0px 6px rgba(0, 0, 0, 0.2);
70     z-index: 0;
71     transition: transform 0.3s cubic-bezier(0.3, 1.05, 0.4, 1.05);
72   }
73   .is-checked {
74     color: #fff !important;
75     font-weight: bold;
76     &.take-type-bg {
77       transform: translateX(90rpx);
78     }
79   }
80 }
81
82 .tab-bar {
83   padding: 0 26rpx;
84   .tabs-box {
85     width: 100%;
86     .tab-item {
87       height: 60rpx;
88       color: #999;
89       font-size: 30rpx;
90       margin-right: 30rpx;
91       position: relative;
92     }
93     .tab-item-actived {
94       color: #222;
95       font-weight: bold;
96       &::after {
97         position: absolute;
98         bottom: 0;
99         left: 50%;
100        transform: translateX(-50%);
101        transition: .3s;
102        content: "";
103        width: 50rpx;
104        height: 8rpx;
105        background-color: var(--primary-color);
106        border-radius: 30rpx;
107      }
108    }
109  }
110 }
111 }
```

4. 点单信息逻辑实现：

- 定义状态属性：takeType、shop、address
- 在 `onShow` 页面显示时，获取首页传递的 takeType 和 shop 属性值

- 在 `scrollShopInfoSticky` 方法中，当页面滚动时订单信息区域吸顶时触发：显示搜索框
- 在 `toShopListPage` 方法中，跳转到门店列表页面
- 在 `changeTakeType` 方法中，实现切换取单类型更新 `takeType` 属性值
- 在 `selectorQueryStickyTop` 方法中，获取分类列表滚动吸顶停留top值=导航栏高度+点单信息区域高度

然后在 `loadAdvertList` 方法中调用 `this.setData({bannerList: data}, this.selectorQueryStickyTop)`

```
1 // pages/shopping/shopping.js
2 import { navTo } from '@utils/util'
3
4 Page({
5
6   data: {
7
8     takeType: 1, // 取单类型：1自取，2外送
9     shop: null, // 下单店铺
10    address: null, // 收货地址
11  },
12
13  onShow() {
14    // takeType取单类型，shop：选择的门店
15    const { takeType, shop } = getApp().pageData || {}
16    if (takeType) this.setData({ takeType })
17    if (shop) this.setData({ shop })
18    // 清空：当前页面中切换了取单类型，再回来切换页面应该保留选择的取单类型，所以把全局变量
    takeType清空
19    getApp().pageData = null
20  },
21
22  // 当页面滚动时，点单信息区域吸顶时触发：显示搜索框
23  scrollShopInfoSticky(e) {
24    // isFixed：是否吸顶
25    const { isFixed } = e.detail
26    this.setData({showSearchInput: isFixed})
27  },
28
29  // 切换取单类型
30  changeTakeType(e) {
31    const {takeType} = e.currentTarget.dataset
32    this.setData({takeType})
33  },
34
35  // 前往门店列表页面
36  toShopListPage() {
37    navTo('/pages/shop-list/shop-list')
38  },
39
40 })
```

```
39
40 // 前往收货地址列表页面
41 toSelectAddress() {
42   const addressId = this.data.address?._id || ''
43   navTo(`/pages/address-list/address-list?addressId=${addressId}`, {
44     events: { // events 添加一个监听事件，用于获取被打开页面传送到当前页面的数据
45       updateAddress: (address) => { // address-list.js 中触发了此监听事件
46         this.setData({ address })
47       }
48     }
49   })
50 },
51
52
53 // 查询头部轮播图广告数据
54 async loadAdvertList() {
55   // 不要少了 await ; '3'代表顶部轮播图广告
56   const { data } = await getAdvertList('3')
57   // ++++++ this.selectorQueryStickyTop +++++
58   this.setData({bannerList: data}, this.selectorQueryStickyTop)
59 },
60
61 // 获取分类列表滚动吸顶停留top值=导航栏高度+点单信息区域高度
62 selectorQueryStickyTop() {
63   // #shop-info-id是点单区域
64   wx.createSelectorQuery().select('#shop-info-id').boundingClientRect((res)
=> {
65     // 导航栏高度+点单信息区域高度
66     this.setData({stickyTop: this.data.navHeight + res.height})
67   }).exec() // 最后不要少了.exec()
68 },
69
70 })
```

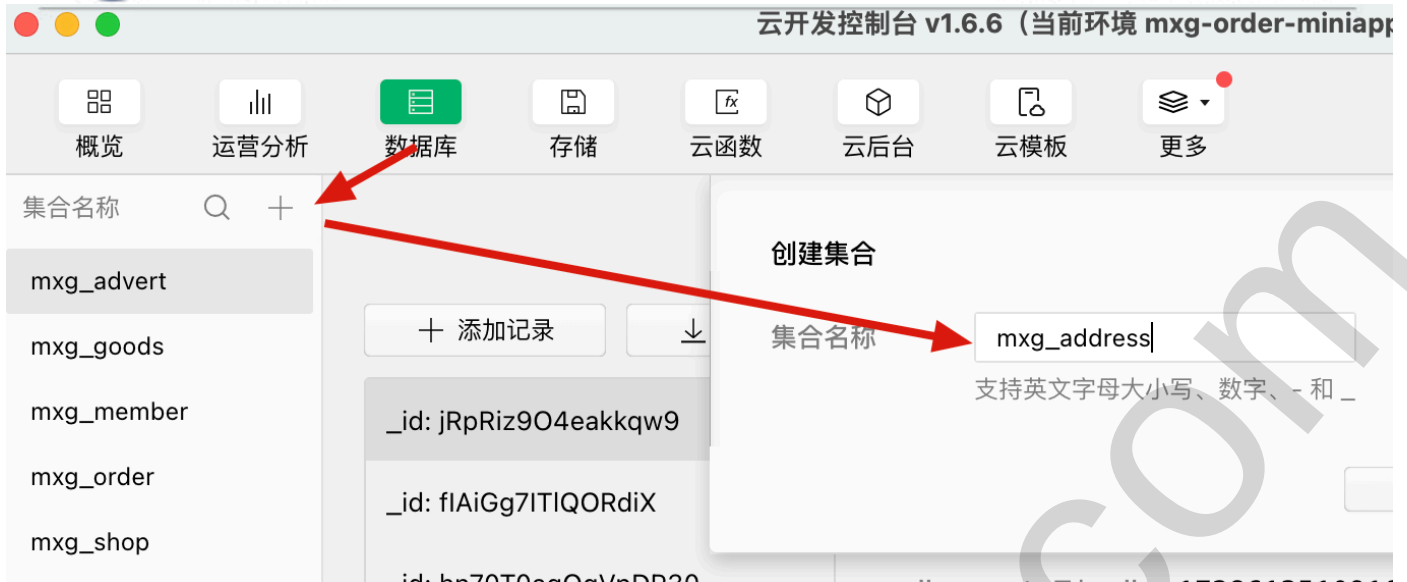
收货地址列表页面实现

收货地址列表页面实现：下拉刷新列表数据，新增地址，修改地址，删除地址，去点单：传递选中地址到下单页。



创建收货地址集合 `mxg_address`

由于内容管理系统中的内容模型，最多添加5个免费的，所以我们在【云开发控制台】的数据库中手动创建一个 `mxg_address` 集合。



地址列表页面相关配置

1. 在 app.json 中配置收货地址列表页面:

```
1 {  
2   "pages": [  
3  
4     "pages/address-list/address-list"  
5   ],  
6 }
```

2. 在 address-list.json 中配置导航栏、开启下拉刷新、注册 `van-swipe-cell` 滑动单元格组件

```
1 {  
2   "navigationBarTitleText": "选择收货地址",  
3   "navigationBarBackgroundColor": "#e40030",  
4   "navigationBarTextStyle": "white",  
5   "backgroundColor": "#e40030",  
6   "backgroundColorBottom": "#f9f9f9",  
7   "enablePullDownRefresh": true,  
8   "usingComponents": {  
9     "van-swipe-cell": "@vant/weapp/swipe-cell/index"  
10  }  
11 }  
12
```

地址列表页面效果实现

1. 地址列表页面模板代码：

```
1  <!--pages/address-list/address-list.wxml-->:
2  <view class="address-list-container">
3    <view class="address-list-box">
4      <view class="address-title-box flex-space-between">
5        <text class="address-title">选择收货地址</text>
6        <view class="add-view flex-row-align-center" hover-class="view-hover"
        hover-stay-time="100" bind:tap="navTo" data-url="/pages/address-edit/address-
        edit">
7          <text class="iconfont icon-addition"></text>
8          <text>新增地址</text>
9        </view>
10     </view>
11     <view class="address-list ">
12       <view class="address-info {{item._id == checkedId ? 'is-checked': ''}}"
13         wx:for="{{ addressList }}" wx:key="_id" >
14         <van-swipe-cell right-width="{{ 60 }}" async-close
15           bind:open="openSwipe" bind:close="closeSwipe"
16           bind:click="handleAddress" data-item="{{item}}"
17         >
18           <view class="address-item flex-column">
19             <text class="location-name">{{ item.locationName }}</text>
20             <text class="street-number">{{ item.streetNumber }}</text>
21             <view class="link-info flex-row-align-center">
22               <text>{{item.name}}</text>
23               <text class="mobile">{{item.mobile}}</text>
24               <view class="edit-view flex-row-align-center"
25                 hover-class="view-hover" hover-stay-time="100"
26                 catch:tap="navTo" data-url="/pages/address-edit/address-edit?
                id={{item._id}}">
27               >
28               <text class="iconfont icon-editor" ></text>
29               <text>编辑</text>
30             </view>
31           </view>
32           <view class="is-checked-right"><text class="iconfont icon-right">
</text></view>
33         </view>
34         <view slot="right" class="swipe-right-delete"
        catch:tap="handleDelete" data-item="{{item}}">
35           删除
36         </view>
37       </van-swipe-cell>
38     </view>
39   </view>
40 </view>
```

```
41 </view>
42
43 <view class="fixed-bottom">
44   <button type="warn" bind:tap="toShoppingPage">去点单</button>
45 </view>
```

2. 地址列表页面样式代码：

```
1  /* pages/address-list/address-list.wxss */
2
3  page {
4    background-color: #fff;
5  }
6
7  .address-list-container {
8    /* 空出底部按钮位置，列表不会被按钮挡住 */
9    padding-bottom: 200rpx;
10   .address-list-box {
11     padding: 26rpx;
12     background-color: #fff;
13     .address-title-box {
14       font-size: 32rpx;
15       .address-title {
16         color: #777;
17       }
18       .add-view {
19         font-size: 30rpx;
20         line-height: 34rpx;
21         color: #e40030;
22         .iconfont {
23           font-size: 36rpx;
24           margin-right: 6rpx;
25           font-weight: bold;
26         }
27       }
28     }
29   }
30   .address-list {
31     margin-top: 30rpx;
32     width: 100%;
33     // 左滑显示的删除按钮
34     .swipe-right-delete {
35       display: flex;
36       justify-content: center;
37       align-items: center;
38       height: 100%;
39       padding: 0 15px;
```

```
40     background-color: var(--primary-color);
41     border-radius: 0 8rpx 8rpx 0;
42     color: #fff;
43 }
44 .address-info {
45     border: 2px solid #cfcfcf;
46     border-radius: 6rpx;
47     margin-top: 20rpx;
48 }
49 .address-item {
50     padding: 20rpx;
51     overflow: hidden;
52     position: relative;
53     .location-name {
54         font-size: 32rpx;
55         font-weight: bold;
56         width: 80%;
57     }
58     .street-number, .link-info {
59         color: #999;
60         font-size: 24rpx;
61         margin-top: 20rpx;
62         width: 100%;
63         line-height: 30rpx;
64     }
65     .mobile{
66         margin-left: 20rpx;
67     }
68     .edit-view {
69         margin-left: auto;
70         color: #666;
71         padding: 0 10rpx;
72         .iconfont {
73             font-size: 30rpx;
74         }
75     }
76 }
77 .is-checked {
78     border-color: var(--primary-color);
79     .is-checked-right {
80         background-color: var(--primary-color);
81     }
82 }
83 .is-checked-right {
84     position: absolute;
85     top: 0;
86     right: 0;
87     width: 50rpx;
88     height: 50rpx;
```



```
89     line-height: 50rpx;
90     background-color: #fff;
91     text-align: center;
92     border-end-start-radius: 4rpx;
93     .iconfont {
94         color: #fff;
95         font-weight: bold;
96         font-size: 40rpx;
97     }
98 }
99 }
100 }
101 }
```

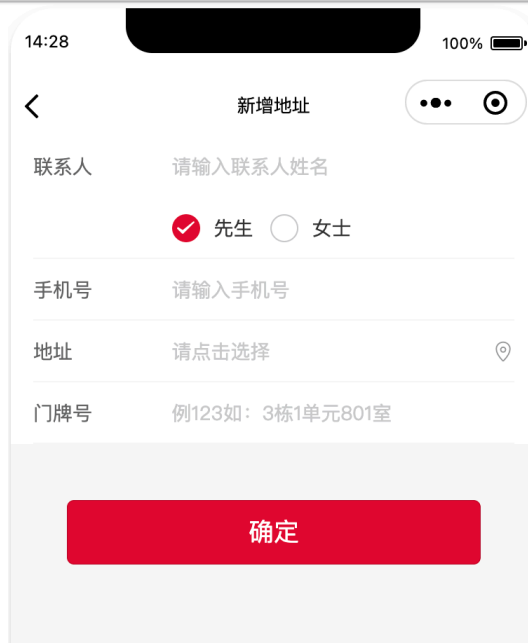
3. 地址列表页面逻辑代码:

```
1  // pages/address-list/address-list.js
2  import { msg, navTo, navBack, showLoading, hideLoading } from "@utils/util"
3
4  Page({
5
6      /**
7       * 页面的初始数据
8       */
9      data: {
10         addressList: [],
11         checkedId: '', // 选中的地址id
12         isOpen: false, // 是否打开滑动
13     },
14
15     onLoad(options) {
16         // 接收下单页面传递的地址id, 用于回显
17         this.setData({checkedId: options.addressId})
18     },
19
20     onShow() {
21         // 触发下拉刷新, 会自动调用 onPullDownRefresh 下拉刷新函数
22         wx.startPullDownRefresh()
23     },
24
25     // 下拉刷新
26     onPullDownRefresh() {
27         this.loadList()
28     },
29
30     navTo, // 跳转
31 }
```

```
32
33   async loadList () {
34     try {
35       this.setData({ checkedId: '' })
36
37       // 查询当前用户的所有收货地址（小程序端直接查询，只能查自己的）
38       const { data: addressList } = await
wx.cloud.database().collection('mxg_address').get()
39       //console.log('查询到的收货地址', addressList)
40
41       this.setData({ addressList })
42     } catch (error) {
43       console.error('查询地址信息失败', error)
44       msg('查询数据失败', {icon: 'error'})
45     } finally {
46       wx.stopPullDownRefresh()
47     }
48   },
49
50   // 打开滑动
51   openSwipe() {
52     this.setData({isOpen: true})
53   },
54
55   // 关闭滑动，组件上使用async-close手动关闭才会触发此方法
56   closeSwipe(e) {
57     this.setData({isOpen: false})
58     // 关闭滑动
59     e.detail.instance.close()
60   },
61
62   // 点击地址
63   handleAddress(e) {
64     // 如果打开滑动，则不处理让它关闭即可
65     if (this.data.isOpen) return
66     const { item } = e.currentTarget.dataset
67     this.setData({checkedId: item._id})
68   },
69
70   // 点击删除
71   handleDelete(e) {
72     const { item } = e.currentTarget.dataset
73     wx.showModal({
74       title: '您是否要删除地址?',
75       cancelText: '取消',
76       confirmText: '确认',
77       cancelColor: '#333333',
78       confirmColor: '#e40030',
79       success: async (res) => { // 不要少了async
```

```
80      // 点击【取消】不用处理
81      if (res.cancel) return
82      try {
83          showLoading('删除中', {mask: true})
84          // 点击【确认】进行删除操作（当前用户_openid下的）
85          const deleted = await
wx.cloud.database().collection('mxg_address').doc(item._id).remove()
86          msg('删除成功')
87          wx.startPullDownRefresh() // 下拉刷新
88          // 删除完成，记得把isOpen设置为false，不然点击其他地址不选中
89          this.setData({isOpen: false})
90      } catch (error) {
91          console.error('删除失败', error)
92      } finally {
93          hideLoading()
94      }
95  }
96  })
97  },
98
99  // 前往下单页面
100  toShoppingPage () {
101      const {addressList, checkedId} = this.data
102
103      // 选中的地址
104      const selectedAddress = addressList?.find(item => item._id === checkedId)
105
106      if (!selectedAddress) {
107          msg('请选择地址', {icon: 'error'})
108          return
109      }
110
111      // 触发来源页面（shopping）的 updateAddress 监听事件，传递选择的地址
112      // 在 shopping.js 中的toSelectAddress方法中定义了updateAddress监听事件
113      const eventChannel = this.getOpenerEventChannel()
114      eventChannel.emit('updateAddress', selectedAddress)
115
116      // 后退回购物页
117      navBack()
118  },
119
120  })
```

编辑收货地址页面实现



14:28 100%

< 新增地址 ...

联系人 请输入联系人姓名

☒ 先生 ☐ 女士

手机号 请输入手机号

地址 请点击选择

门牌号 例123如：3栋1单元801室

确定

编码页面相关配置

1. 在 app.json 中配置编辑收货地址页面：

```
1 {  
2   "pages": [  
3  
4     "pages/address-edit/address-edit"  
5   ],  
6 }
```

2. 在 address-edit.json 中配置导航栏（空标题，通过API动态指定标题）、注册组件

```
1 {  
2   "navigationBarTitleText": "",  
3   "backgroundColor": "#ffffff",  
4   "backgroundColorBottom": "#f9f9f9",  
5   "usingComponents": {  
6     "van-field": "@vant/weapp/field/index",  
7     "van-cell": "@vant/weapp/cell/index",  
8     "van-radio": "@vant/weapp/radio/index",  
9     "van-radio-group": "@vant/weapp/radio-group/index"  
10  }  
11 }
```

编辑页面效果实现

1. 编辑收货地址页面模板实现：

```
1 <!--pages/address-edit/address-edit.wxml-->
2
3 <view class="address-edit-container">
4   <van-field
5     model:value="{{ name }}"
6     clearable
7     label="联系人"
8     placeholder="请输入联系人姓名"
9     border="{{ false }}"
10    error-message="{{ error.name }}"
11  />
12   <van-field label=" " error-message="{{ error.sex }}">
13     <van-radio-group slot="input" model:value="{{ sex }}"
14       direction="horizontal">
15       <van-radio name="{{1}}" checked-color="#e40030">先生</van-radio>
16       <van-radio name="{{2}}" checked-color="#e40030">女士</van-radio>
17     </van-radio-group>
18   </van-field>
19   <van-field
20     model:value="{{ mobile }}"
21     clearable
22     label="手机号"
23     placeholder="请输入手机号"
24     error-message="{{ error.mobile }}"
25   />
26
27   <van-field
28     model:value="{{ locationName }}"
29     readonly
30     label="地址"
31     placeholder="请点击选择"
32     right-icon="location-o"
33     bind:click-icon="handleLocation"
34     bind:click-input="handleLocation"
35     error-message="{{ error.locationName }}"
36   >
37   </van-field>
38
39   <van-field
40     model:value="{{ streetNumber }}"
41     clearable
42     label="门牌号"
43     placeholder="例123如：3栋1单元801室"
44     error-message="{{ error.streetNumber }}"
```

```
45     />
46
47     <view class="confirm-btn">
48         <button type="primary" bind:tap="handleConfirm">确定</button>
49     </view>
50 </view>
```

2. 编辑收货地址页面样式实现：

```
1  /* pages/address-edit/address-edit.wxss */
2
3  .address-edit-container {
4      .confirm-btn {
5          margin: 80rpx;
6      }
7  }
```

3. 编辑收货地址页面逻辑代码实现：

```
1  // pages/address-edit/address-edit.js
2  import { isNull, msg, navBack, showLoading, hideLoading } from '@utils/util'
3
4  Page({
5
6      data: {
7          id: '', // 地址id
8          name: '', // 联系人姓名
9          sex: 1, // 1先生, 2女士
10         mobile: '', // 手机号
11         locationName: '', // 位置名称
12         address: '', // 完整地址
13         longitude: 0, // 位置经度
14         latitude: 0, // 位置纬度
15         streetNumber: '', // 门牌号
16         error: {}, // 表单校验错误信息
17     },
18
19     /**
20      * 生命周期函数--监听页面加载
21      */
22     onLoad(options) {
23         // 修改地址会传递id
24         const { id = '' } = options
25         this.setData( { id } )
26
27         // 设置导航栏标题
```

```
28     wx.setNavigationBarTitle({title: id ? '修改地址': '新增地址'})
29
30     // 查询要修改数据
31     this.loadData(id)
32 },
33
34 /** 查询要修改数据 */
35 async loadData(id) {
36     if (!id) return
37     // 有 id 值: 查询要修改数据
38     console.log('查询要修改数据')
39 },
40
41 // 点击选择地址框
42 async handleLocation() {
43     // 打开地图选择位置, 需要在 app.json 配置 requiredPrivateInfos
44     const {name, address, latitude, longitude} = await wx.chooseLocation()
45     this.setData({ locationName: name, address, latitude, longitude })
46 },
47
48 // 点击确定按钮触发
49 handleConfirm () {
50     const {name, sex, mobile, locationName, latitude, longitude, streetNumber}
= this.data
51     let error = {}
52     if (isNull(name)) error.name = '联系人姓名不能为空'
53     if (isNull(sex)) error.sex = '性别为必选项'
54     if (isNull(mobile)) error.mobile = '手机号不能为空'
55     if (isNull(locationName, latitude, longitude)) error.locationName = '地址不
能为空'
56     if (isNull(streetNumber)) error.streetNumber = '门牌号不能为空'
57
58     this.setData({ error })
59
60     if (Object.keys(error).length>0) return
61
62     this.addOrUpdate()
63 },
64
65 // 调用新增或更新地址接口
66 async addOrUpdate() {
67
68 }
69
70 })
```


新增和查询/修改地址接口

1. 创建通用的新增数据的云函数 `add` : `cloudfunctions/add/index.js`

```
1 // 云函数入口文件
2 const cloud = require('wx-server-sdk')
3
4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6 // 云函数入口函数：通用的新增数据
7 exports.main = async (event, context) => {
8   const { OPENID: openId } = cloud.getWXContext()
9   const { collectionName, data } = event
10
11   return await cloud.database().collection(collectionName)
12     .add({
13     data: {
14       _createTime: Date.now(),
15       _updateTime: Date.now(),
16       _openid: openId, // _openid对应用户可在小程序端直接调用查询表（不需要通过云函
17 数)
18       ...data
19     }
20   })
21 }
```

2. 封装新增地址、查询地址详情、更新地址API方法

查询地址详情调用之前创建过的 `getById` 云函数、更新地址 调用之前创建过的 `updateById` 云函数。

创建 `miniprogram/api/member.js` 文件，封装API方法：

```
1 // miniprogram/api/address.js
2
3 /**
4  * 新增收货地址信息
5  * @param {Object} data 新增的数据对象
6  */
7 export function addAddress(data) {
8   return wx.cloud.callFunction({
9     name: 'add',
10    data: {
11      collectionName: 'mxg_address',
12      data, // data: data
13    }
14  })
15 }
```

```
16
17 /**
18  * 通过地址id查询收货地址详情
19  * @param {String|Number} id 地址id
20  */
21 export function getAddressById(id) {
22   return wx.cloud.callFunction({
23     name: 'getById',
24     data: {
25       collectionName: 'mxg_address',
26       id, //id: id
27     }
28   })
29 }
30
31 /**
32  * 更新收货地址信息
33  * @param {String|Number} id 地址id
34  * @param {Object} data 更新的数据对象
35  */
36 export function updateAddress(id, data) {
37   return wx.cloud.callFunction({
38     name: 'updateById',
39     data: {
40       collectionName: 'mxg_address',
41       id, // id: id,
42       data, // data: data
43     }
44   })
45 }
```

新增和修改页面逻辑实现

1. 在address-edit.js导入api方法，并分别调用处理：

```
1 // pages/address-edit/address-edit.js
2 import { isNull, msg, navBack, showLoading, hideLoading } from '@/utils/util'
3 import { getAddressById, updateAddress, addAddress } from '@/api/address'
4
5 Page({
6
7   data: {
8     id: '', // 地址id
9     name: '', // 联系人姓名
10    sex: 1, // 1先生, 2女士
11    mobile: '', // 手机号
12    locationName: '', // 位置名称
```

```
13     address: '', // 完整地址
14     longitude: 0, // 位置经度
15     latitude: 0, // 位置纬度
16     streetNumber: '', // 门牌号
17     error: {}, // 表单校验错误信息
18   },
19
20   /**
21    * 生命周期函数--监听页面加载
22    */
23   onLoad(options) {
24     // 修改地址会传递id
25     const { id = '' } = options
26     this.setData( { id } )
27
28     // 设置导航栏标题
29     wx.setNavigationBarTitle({title: id ? '修改地址': '新增地址'})
30
31     // 查询要修改数据
32     this.loadData(id)
33   },
34
35   /** 查询要修改数据 */
36   async loadData(id) {
37     if (!id) return
38     try {
39       showLoading()
40       // 查询
41       const { result } = await getAddressById(id)
42       console.log('查询要修改数据', result.data)
43
44       const {name, sex, mobile, locationName, address, latitude, longitude,
streetNumber} = result.data
45       this.setData({name, sex, mobile, locationName, address, latitude,
longitude, streetNumber})
46     } catch (error) {
47       console.error('地址查询失败', error)
48       msg('地址查询失败', {icon: 'error'})
49     } finally {
50       hideLoading()
51     }
52   },
53
54   // 点击选择地址框
55   async handleLocation() {
56     // 打开地图选择位置，需要在 app.json 配置 requiredPrivateInfos
57     const {name, address, latitude, longitude} = await wx.chooseLocation()
58     this.setData({ locationName: name, address, latitude, longitude })
59   },
```

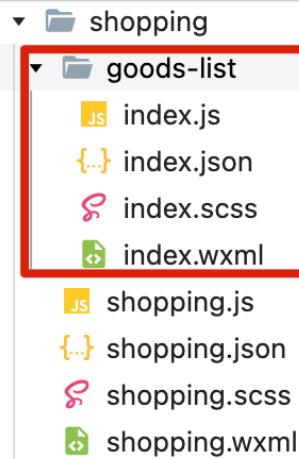
```
60
61 // 点击确定按钮触发
62 handleConfirm () {
63     const {name, sex, mobile, locationName, latitude, longitude, streetNumber}
    = this.data
64     let error = {}
65     if (isNull(name)) error.name = '联系人姓名不能为空'
66     if (isNull(sex)) error.sex = '性别为必选项'
67     if (isNull(mobile)) error.mobile = '手机号不能为空'
68     if (isNull(locationName, latitude, longitude)) error.locationName = '地址不
    能为空'
69     if (isNull(streetNumber)) error.streetNumber = '门牌号不能为空'
70
71     this.setData({error})
72
73     if (Object.keys(error).length>0) return
74
75     this.addOrUpdate()
76 },
77
78 // 调用新增或更新地址接口
79 async addOrUpdate() {
80     try {
81         showLoading('提交中')
82         const {id, name, sex, mobile, locationName,
83             address, latitude, longitude, streetNumber} = this.data
84         // 提交的数据
85         const data = { name, sex, mobile, locationName, address, latitude,
    longitude, streetNumber}
86         let res;
87         if (id) { // 修改
88             res = await updateAddress(id, data)
89         } else { // 新增
90             res = await addAddress(data)
91         }
92         console.log('数据保存结果', res.result)
93
94         msg('保存成功', {icon: 'success'})
95
96         // 后退
97         navBack()
98     } catch (error) {
99         console.error('保存地址失败', error)
100         msg('保存失败', {icon: 'error'})
101     } finally {
102         hideLoading()
103     }
104 }
105
```

分类和商品列表实现



创建并引用商品列表子组件

1. 分类和商品列表组件只在当前点单页面使用，所以我们在 `shopping` 页面目录下创建一个 `goods-list` 子组件，如下：



2. 在父组件页面配置 shopping.json 中注册子组件 `"goods-list": "../goods-list/index"`

```
1 {
2   "navigationStyle": "custom",
3   "usingComponents": {
4     "navbar": "../components/navbar/index",
5     "banner": "../components/banner/index",
6     "van-sticky": "@vant/weapp/sticky/index",
7     "van-tag": "@vant/weapp/tag/index",
8     "goods-list": "../goods-list/index"
9   }
10 }
```

3. 在父组件页面 shopping.wxml 使用商品列表子组件 `<goods-list>`

```
1 <!-- 分类+商品列表区域 -->
2 <goods-list id="goods-list-id" sticky-top="{{stickyTop}}"/>
```

查询商品分类数据

1. 创建查询商品分类数据云函数 `getCategoryList`: `cloudfunctions/getCategoryList/index.js`

- 商品分类数据: 将 `mxg_goods` 集合中的 `category` 字段值分组查询所有分类名称。

分组查询使用 `aggregate().group({_id: '$category'})`

```
1 // 云函数入口文件
2 const cloud = require('wx-server-sdk')
3
4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6 const db = cloud.database()
```



```
7
8 // 云函数入口函数：查询分类列表数据
9 exports.main = async (event, context) => {
10   // 查询所有商品分类，响应结果是 list: [{_id: "分类1"}, {_id: "分类2"}]
11   return await db.collection('mxg_goods').aggregate()
12     .group({
13       _id: '$category', // _id值指定`$分组字段名`，注意：_id不能修改
14     })
15     .sort({
16       _id: 1 // 1 代表升序排列（从小到大），-1 代表降序排列（从大到小）；
17     })
18     .end()
19 }
```

2. 封装调用云函数API方法：创建miniprogram/api/goods.js文件

```
1 // miniprogram/api/goods.js
2
3 /**
4  * 查询所有商品分类
5  */
6 export async function getCategoryList() {
7   const { result } = await wx.cloud.callFunction({
8     name: 'getCategoryList',
9   })
10   return result
11 }
```

3. goods-list 组件中调用API方法

- 在组件生命函数 `attached` 中调用 `getCategoryList` 方法查询分类
- 因为查询结果中只有分类名称，没有分类id，所以我们将分类名称编码后的值作为分类id
`encodeURIComponent(str).replaceAll('%', '')`
- 定义 `handleCategory` 方法，当点击分类时高亮显示此分类

```
1 // pages/shopping/goods-list/index.js
2
3 import { getCategoryList } from '@api/goods'
4
5 Component({
6
7   options: {
8     // 使用全局样式，不然图标等全局样式无效，或配置 styleIsolation: 'apply-shared'
9     addGlobalClass: true,
10   },
```



```
11
12  /**组件的属性列表*/
13  properties: {
14    stickyTop: { // 左侧分类吸顶top
15      type: Number,
16      value: 187
17    }
18  },
19
20  data: {
21    activatedCategoryId: -1, // 当前选中的分类id
22    categoryList: [], // 分类数据
23  },
24
25  // 组件生命周期函数
26  lifetimes: {
27    // 在组件实例进入页面节点树时执行
28    attached() {
29      this.loadCategoryList()
30    }
31  },
32
33  /**
34   * 组件的方法列表
35   */
36  methods: {
37
38    // 字符串编码,
39    encode (str) {
40      // 编码后, 将所有%替换掉
41      return encodeURIComponent(str).replaceAll('%', '')
42    },
43
44    // 查询所有分类
45    async loadCategoryList() {
46      let { list: categoryList } = await getCategoryList()
47      let activatedCategoryId = -1
48      if (categoryList && categoryList.length > 0) {
49        // 分类数据格式化: 将 分类名 转编码后,
50        // 添加到 categoryId [{categoryId: "E5BD93E5ADA3E696B0E59381",
51        categoryName: "当季新品"}]
52        categoryList = categoryList.map(item => ({ categoryId:
53          this.encode(item._id),
54
55          categoryName: item._id })
56      )
57
58      // 默认选中第1个分类
59      activatedCategoryId = categoryList[0].categoryId
60    }
61  }
62  // 更新状态属性值
```

```
57     this.setData({ categoryList, activeCategoryId })
58   },
59
60   /** 点击分类：高亮显示 */
61   handleCategory(e) {
62     const { item } = e.currentTarget.dataset
63     // 高亮显示分类
64     this.setData({ activeCategoryId: item.categoryId })
65   },
66
67 })
```

左侧商品分类效果实现

1. 左侧商品分类模板代码：

```
1  <!-- pages/shopping/goods-list/index.wxml -->
2
3  <!-- 列表区域 -->
4  <view class="main-list-wrapper flex-row">
5    <!-- 左侧分类列表 top:吸顶离顶部距离,-->
6    <view class="category-list-wrapper"
7      style="top: {{stickyTop}}px; height: calc(100vh - {{stickyTop}}px);">
8      <scroll-view class="category-scroll"
9        scroll-y
10        enable-passive
11        show-scrollbar="{{false}}"
12      >
13        <view class="category-list">
14          <view class="category-item" wx:for="{{categoryList}}" wx:key="_id"
15            bind:tap="handleCategory" data-item="{{item}}">
16            <view wx:if="{{activeCategoryId == item.categoryId}}"
17              class="flex-center category-img-view img-zoom-in">
18              <image class="category-img" src="/assets/images/category.png"
19                mode="scaleToFill" lazy-load/>
20            </view>
21            <view class="flex-center category-text-view"
22              {{activeCategoryId == item.categoryId ? 'category-text-view-actived': ''}}">
23              <text class="category-text">{{item.categoryName}}</text>
24            </view>
25          </view>
26        </scroll-view>
27      </view>
28
29    <!-- 右侧商品列表 -->
```

```
30  
31 </view>
```

2. 左侧商品分类样式代码:

```
1  /**左侧分类*/  
2  .category-list-wrapper {  
3    width: 100rpx;  
4    height: 100vh;  
5    position: sticky;  
6    .category-scroll {  
7      height: 100%;  
8      ::-webkit-scrollbar { // 隐藏滚动条 (show-scrollbar="{{false}}")无效)  
9        display: none;  
10     }  
11   }  
12   .category-list {  
13     padding-bottom: 600rpx;  
14     .category-item {  
15       transition-duration: .3s;  
16       padding-top: 10rpx;  
17       .category-img-view {  
18         position: relative;  
19         top: 16rpx;  
20         z-index: 10;  
21         .category-img {  
22           width: 50rpx;  
23           max-height: 50rpx;  
24         }  
25       }  
26       .category-text-view {  
27         padding: 16rpx 20rpx;  
28         position: relative;  
29         z-index: 1;  
30         color: #231e1f;  
31         .category-text {  
32           font-size: 24rpx;  
33           line-height: 30rpx;  
34           max-width: 2.3rem;  
35           text-align: center;  
36           white-space: pre-wrap;  
37         }  
38         &::after {  
39           position: absolute;  
40           content: "";  
41           width: 70%;  
42           height: 0;
```

```
43         border-bottom: 1px solid #e6e6e6;
44         left: 50%;
45         bottom: -3px;
46         transform: translateX(-50%);
47         transition: .3s;
48     }
49 }
50
51 .category-text-view-activated {
52     color: #fff;
53     font-weight: 600;
54     background-color: var(--primary-color);
55     border-top-right-radius: 10rpx;
56     border-bottom-right-radius: 10rpx;
57 }
58
59 .img-zoom-in {
60     animation: ZoomInOut .4s forwards;
61     animation-duration: 400ms;
62 }
63
64 }
65 }
66 }
67
68 /* 选中分类图片动画 */
69 @keyframes ZoomInOut {
70     0% {
71         transform: rotate(10deg) scale(0);
72     }
73     75% {
74         transform: rotate(-25deg) scale(1.3);
75     }
76     100% {
77         transform: rotate(0) scale(1);
78     }
79 }
80
```

分页查询商品列表数据

1. 创建分页查询商品列表数据云函数 `queryPageGoodsList`： `cloudfunctions/queryPageGoodsList/index.js`
 - 商品列表数据：分页查询 `mxg_goods` 集合中商品数据。

```
1 // 云函数入口文件
2 const cloud = require('wx-server-sdk')
```

```
3
4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6 const db = cloud.database()
7
8 // 云函数入口函数：分页查询商品列表
9 exports.main = async (event, context) => {
10   const { keyword, pageNum=1, pageSize=100 } = event
11
12   // 封装模糊查询条件
13   let data = {}
14   if (keyword && keyword.trim().length > 0) {
15     data = {
16       goodsName: db.RegExp({
17         regexp: keyword.trim(), // " xxx "
18         options: 'i',
19       })
20     }
21   }
22
23   // 1、查询总记录数
24   const { total } = await db.collection('mxg_goods').where(data).count()
25
26   // 2、分页查询商品数据
27   const { data: goodsList } = await db.collection('mxg_goods')
28     .where({
29       data
30     })
31     .limit(pageSize)
32     .skip((pageNum - 1) * pageSize)
33     .orderBy('category', 'asc')
34     .get()
35
36   return {total, goodsList}
37 }
38 }
```

2. 封装调用云函数API方法：

```
1 // miniprogram/api/goods.js
2
3 /**
4  * 分页查询商品列表
5  * @param pageNum 当前页码
6  * @param pageSize 查询多少条数据
7  * @param keyword
8  */
```

```
9 export async function queryPageGoodsList(pageNum=1, pageSize=100, keyword) {
10   const { result } = await wx.cloud.callFunction({
11     name: 'queryPageGoodsList',
12     data: {
13       pageNum,
14       pageSize,
15       keyword
16     }
17   })
18   return result
19 }
```

3. goods-list 组件中调用API方法

- 在 **页面** 生命函数 `show` 中调用 `queryPageGoodsList` 方法查询商品数据

```
1 // pages/shopping/goods-list/index.js
2
3 import { getCategoryList, queryPageGoodsList } from '@api/goods'
4 import { getTotalPage } from '@utils/util'
5
6 Component({
7
8   /**
9    * 组件的初始数据
10   */
11   data: {
12     activatedCategoryId: -1, // 当前选中的分类id
13     categoryList: [], // 分类数据
14     goodsList: [], // 商品数据
15     pageNum: 1, // 当前页码
16     pageSize: 100, // 每页显示多少条
17     totalPage: 1, // 总页码
18   },
19
20   /** 页面生命周期函数 */
21   pageLifetimes: {
22     // 页面显示时执行
23     show() {
24       this.setData({pageNum: 1})
25       this.loadGoodsList()
26     },
27   },
28
29   /** 组件的方法列表 */
30   methods: {
31
32     // 查询商品列表数据
```

```
33   async loadGoodsList() {
34       let { totalPages, pageNum, pageSize } = this.data
35
36       // 待查询页码大于总页码，则不查询
37       if (pageNum !== 1 && pageNum > totalPages) {
38           return
39       }
40
41       // 发送查询请求
42       let {total, goodsList} = await queryPageGoodsList(pageNum, pageSize)
43
44       // 计算总页数
45       totalPages = getTotalPage(total, pageSize)
46
47       // 新数据处理：添加分类id属性，并追加到原goodsList数组后面
48       if (goodsList && goodsList.length > 0) {
49           // 将 ·分类名· 转编码后添加到 categoryId
50           goodsList = goodsList.map(item => ({...item, categoryId:
this.encode(item.category)}))
51           // 第1页展示本次查询结果，第2次开始则追加到原数据后面
52           if (pageNum > 1) goodsList = this.data.goodsList.concat(goodsList)
53       }
54
55       // 更新状态数据
56       this.setData({ totalPages, goodsList })
57   },
58
59   // 供父页面滚到底部触发此方法：查询下一页商品数据
60   reachBottomLoad() {
61       const { pageNum } = this.data
62       this.setData({pageNum: pageNum+1})
63       this.loadGoodsList()
64   },
65
66   },
67
68   })
69 }
```

4. 当下单页面滚到底部时，触发查询下一页商品数据：

- 在父组件页面 shopping.json 中添加 "onReachBottomDistance": 300,

当页面上拉距离底部 300px，触发页面事件函数 onReachBottom，查询下一页数据。


```
1 {
2   "navigationStyle": "custom",
3   "onReachBottomDistance": 300,
4   "usingComponents": {
5     //省略。。。
6   }
7 }
```

- 在父组件页面 shopping.js 中添加页面事件函数 `onReachBottom`，函数体实现：调用子组件中定义 `reachBottomLoad` 方法查询下一页商品数据。

```
1  /** 监听页面初次渲染完成 */
2  onReady() {
3    // 获取<goods-list id="goods-list-id"/>组件实现
4    this.goodsListComponent = this.selectComponent('#goods-list-id')
5  },
6
7  // 页面上拉触底事件的处理函数
8  onReachBottom() {
9    this.goodsListComponent?.reachBottomLoad()
10 }
```

```
1 <!-- 分类+商品列表区域 -->
2 <goods-list id="goods-list-id" sticky-top="{{stickyTop}}" />
```

右侧商品列表效果实现

1. 右侧商品列表模板代码：

```
1 <!-- pages/shopping/goods-list/index.wxml -->
2
3 <!-- 列表区域 -->
4 <view class="main-list-wrapper flex-row">
5
6   <!-- 左侧分类列表-->
7   <view class="category-list-wrapper">
8   </view>
9
10  <!-- 右侧商品列表 -->
11  <view class="goods-list-wrapper flex-1">
12    <view class="divider-box">
13      <image src="/assets/images/leaf.png"/>
14      <text>珍惜资源 按需点单</text>
15      <image src="/assets/images/leaf.png"/>
```

```
16     </view>
17     <view class="goods-item" wx:for="{{goodsList}}" wx:key="_id">
18         <view class="goods-item-inner flex-space-between">
19             <view class="goods-item-left flex-center" bind:tap="toGoodsDetail"
data-item="{{item}}">
20                 <image class="goods-item-img" lazy-load src="{{item.imageUrl}}" />
21             </view>
22             <view class="goods-content">
23                 <view class="goods-info">
24                     <text class="goods-name">{{item.goodsName}}</text>
25                     <text class="goods-unit" wx:if="{{item.unit}}">{{item.unit}}</text>
26                     <view class="goods-price flex-row-end">
27                         <text class="original-price" wx:if="{{item.discountPrice}}">¥
{{item.price}}</text>
28                         <view class="sale-price flex-center">
29                             <text class="sale-price-symbol">¥</text>
30                             <text class="sale-price-value">{{item.discountPrice ||
item.price}}</text>
31                             <text class="sale-price-word">起</text>
32                         </view>
33                     </view>
34                 </view>
35                 <view class="btn-choose-box">
36                     <view wx:if="{{item.goodsSpecs && item.goodsSpecs.length > 0}}"
class="btn-choose flex-center" hover-class="view-hover"
37                     bind:tap="toGoodsDetail" data-item="{{item}}"
38                     >
39                         <text>选规格</text>
40                         <text class="buy-num" wx:if="{{item.buyNum}}">{{item.buyNum}}
41                     </text>
42                 </view>
43                 <view wx:else class="sub-add-box flex-center">
44                     <view wx:if="{{item.buyNum > 0}}" class="flex-center">
45                         <view class="sub-view sub-add-btn flex-center"
46                         bind:tap="hanldeItemSub" data-goods="{{item}}">
47                             <text class="iconfont icon-sub"></text>
48                         </view>
49                         <view class="num-view sub-add-btn flex-center"
50                         bind:tap="handleItemNum" data-goods="{{item}}">
51                             <text >{{item.buyNum}}</text>
52                         </view>
53                     </view>
54                     <view class="add-view sub-add-btn flex-center"
55                     bind:tap="hanldeItemAdd" data-goods="{{item}}">
56                         <text class="iconfont icon-add"></text>
57                     </view>
58                 </view>
59             </view>
60         </view>
```

```
61     </view>
62     <!-- 已售罄 -->
63     <view hidden="{{item.stockNum && item.stockNum > 0}}" class="goods-item-
cover">
64         <view class="cover-left flex-center">
65             <view class="cover-btn"><text>当前售罄</text></view>
66         </view>
67     </view>
68 </view>
69 </view>
70 </view>
```

2. 右侧商品列表样式代码：

```
1  /* pages/shopping/goods-list/index.scss */
2
3  /* 左侧商品分类 */
4
5  /* 右侧商品列表 */
6  .goods-list-wrapper {
7      padding: 26rpx 26rpx 600rpx 26rpx;
8      .divider-box {
9          display: flex;
10         align-items: center;
11         color: #303030;
12         font-size: 24rpx;
13         border: 0 solid #ebedf0;
14         image {
15             &:first-child {
16                 transform: scaleX(-1);
17             }
18             width: 28rpx;
19             height: 28rpx;
20             margin: 0 12rpx;
21         }
22         &::after, &::before {
23             content: "";
24             border-color: inherit;
25             border-style: inherit;
26             border-width: 1px 0 0;
27             flex: 1;
28         }
29     }
30
31     .goods-item {
32         margin-top: 15px;
33         position: relative;
```

```
34 .goods-item-inner {
35   background-color: #fff;
36   border-radius: 10rpx;
37   border: .05rem solid hsla(0, 0%, 91%, .7);
38   overflow: hidden;
39   .goods-item-left {
40     padding: 4rpx;
41     .goods-item-img {
42       width: 300rpx;
43       height: 240rpx;
44     }
45   }
46
47   .goods-content {
48     text-align: right;
49     padding: 20rpx 20rpx 0 20rpx;
50     position: relative;
51     .goods-info {
52       &> text {
53         display: block;
54       }
55       .goods-name {
56         font-size: 26rpx;
57         color: #222222;
58         font-weight: 500;
59         -webkit-line-clamp: 2;
60         -webkit-box-orient: vertical;
61         overflow: hidden;
62         text-overflow: ellipsis;
63         display: -webkit-box;
64       }
65       .goods-unit {
66         font-size: 20rpx;
67         color: #999;
68       }
69       .goods-price {
70         align-items: flex-end;
71         margin-top: 8rpx;
72         .original-price {
73           color: #828282;
74           text-decoration: line-through;
75           font-size: 23rpx;
76           line-height: 27rpx;
77         }
78         .sale-price {
79           color: red;
80           align-items: flex-end;
81           .sale-price-symbol {
82             font-size: 24rpx;
```

```
83         line-height: 28rpx;
84     }
85     .sale-price-value {
86         font-weight: 500;
87         font-size: 40rpx;
88         line-height: 36rpx;
89     }
90     .sale-price-word {
91         font-size: 18rpx;
92     }
93 }
94 }
95 }
96 .btn-choose-box {
97     position: absolute;
98     bottom: 0;
99     right: 20rpx;
100    background-color: var(--primary-color);
101    border-radius: 10rpx 10rpx 0 0;
102    text-align: center;
103    min-width: 60rpx;
104    color: #fff;
105    font-size: 28rpx;
106    .btn-choose {
107        padding: 0 20rpx;
108        height: 60rpx;
109        font-size: 26rpx;
110        position: relative;
111        .buy-num {
112            position: absolute;
113            top: -10rpx;
114            right: -10rpx;
115            min-width: 26rpx;
116            height: 26rpx;
117            line-height: 26rpx;
118            border-radius: 100%;
119            font-size: 20rpx;
120            text-align: center;
121            font-weight: 700;
122            color: #fff;
123            background: #363535;
124            padding: 0 4rpx;
125        }
126    }
127    .sub-add-btn {
128        height: 60rpx;
129        min-width: 70rpx;
130        overflow: hidden;
131        .iconfont {
```

```
132         font-size: 30rpx;
133         font-weight: bold;
134     }
135 }
136 }
137 }
138 }
139 // 已售罄
140 .goods-item-cover {
141     background-color: rgba(255, 255, 255, 0.6); //hsla(0, 0%, 100%, .6);
142     position: absolute;
143     top: 0;
144     left: 0;
145     height: 100%;
146     width: 100%;
147     z-index: 4;
148     .cover-left {
149         width: 300rpx;
150         height: 100%;
151         .cover-btn {
152             color: #fff;
153             background-color: rgba(0, 0, 0, .5);
154             font-size: 24rpx;
155             padding: 10rpx 25rpx;
156             border-radius: 30rpx;
157         }
158     }
159 }
160 }
161 }
```

商品加入购物车

点击商品 + 将商品加入购物车；点击商品 - 将商品从购物车中减掉，

1. 在 goods-list/index.js 实现加入购物车逻辑：

- 在 properties 中声明 cartList、totalNum、totalAmount（因为购物车数据展示后面定义有父组件页面中）
- 定义 hanldeItemAdd 实现加入购物车逻辑
- 添加监听器 observers 选项，其中监听cartList变化时：
 - 统计购物总数量、总金额
 - 将购物车中的【商品数量】赋值给 列表中的对应商品，进行展示显示
- 在生命周期函数的 show 中setData更新cartList，解决从其他页面切换回此页面时，更新 cartList 购物车数据触发监听器，让它更新商品购买数量显示在商品列表中。

```
1 // pages/shopping/goods-list/index.js
2
3 Component({
4
5   /** 组件的属性列表 */
6   properties: {
7     stickyTop: { // 左侧分类吸顶top
8       type: Number,
9       value: 187
10    },
11    // ++++++
12    cartList: Array, // 购物车数据
13    totalNum: Number, // 购物车的商品总单量，双向绑定 this.setData({totalNum: xx})
14    totalAmount: Number, // 购物车的商品总金额，双向绑定
15  },
16
17
18   /** 页面生命周期函数 */
19   pageLifetimes: {
20     // 页面显示时执行
21     async show() {
22       this.setData({pageNum: 1})
23       await this.loadGoodsList()
24       // 上面加await，从其他页面切换回此页面时，更新 cartList 购物车数据触发监听器，让它更
25       // 新商品购买数量显示在商品列表中
26       this.setData({cartList: this.properties.cartList}) // ++++++
27     },
28   },
29
30   // 监听器 ++++++
31   observers: {
32
33     // 监听购物车数据变化，更新商品列表数据
34     cartList(cartList) {
35       // 统计购物车的商品总数量
36       const totalNum = cartList.reduce((prev, curr) => {
37         return prev + curr.buyNum
38       }, 0)
39
40       // 总金额
41       const totalAmount = cartList.reduce((prev, curr) => {
42         return prev + curr.salePrice * curr.buyNum
43       }, 0)
44
45       // 将购物车中的【商品数量】赋值给 列表中的对应商品
46       const { goodsList } = this.data
47       goodsList.forEach(goods => {
48         // 同一商品有多个规格情况下，累加计算到商品中
49         goods.buyNum = cartList.reduce((prev, curr) => {
```



```
49         return prev + (curr._id == goods._id ? curr.buyNum: 0)
50     }, 0)
51 })
52
53     this.setData({ totalNum, totalAmount: totalAmount.toFixed(2), goodsList
54 })
55 },
56 },
57
58 /** 组件的方法列表 */
59 methods: {
60
61     /** 点击商品加号 + */
62     hanldeItemAdd(e) {
63         const { goods } = e.currentTarget.dataset
64
65         // _id商品id, specs选择的商品规格
66         const { _id, specs } = goods
67         // 默认购买数量 1
68         const buyNum = goods.buyNum || 1
69
70         // 购物车数据
71         const { cartList = [] } = this.properties
72
73         // 获取被点击商品在购物车中的商品：匹配商品id和规格id相等
74         const target = cartList.find(item => item._id == _id
75             && (item.specsId ? item.specsId == specs._id
76 : true))
77
78         if (target) {
79             // 在购物车：累加数量
80             target.buyNum = target.buyNum + buyNum
81         } else {
82             // 不在购物车：则商品购买数量为1，加入购物车
83             const salePrice = goods.discountPrice || goods.price
84             cartList.push({...goods, buyNum, salePrice })
85         }
86         // console.log('cartList', cartList)
87
88         // 更新购物车数据
89         this.triggerEvent('updateCartList', {cartList})
90
91     },
92 },
93 })
```

2. 在父组件页面 shopping.wxml 中传递 cartList，双向绑定total-num和total-amount，并绑定 updateCartList 事件

```
1 <!-- 分类+商品列表区域 -->
2 <goods-list
3   id="goods-list-id"
4   sticky-top="{{stickyTop}}"
5   model:total-num="{{totalNum}}"
6   model:total-amount="{{totalAmount}}"
7   cart-list="{{cartList}}"
8   bind:updateCartList="updateCartList"
9 />
```

3. 在父组件页面 shopping.js 定义事件处理函数 updateCartList，更新 cartList 数据

```
1 // pages/shopping/shopping.js
2
3 Page({
4
5   data: {
6     cartList: [], // 购物车数量
7     totalNum: 0, // 总数量
8     totalAmount: 0, //总金额
9   },
10
11   // 更新购物车数据
12   updateCartList(e) {
13     const { cartList } = e.detail
14     this.setData({ cartList })
15   },
16
17 })
```

4. 实现点击商品 - 将商品从购物车中减掉：

- 在商品列表子组件中定义 hanldeItemSub 方法实现：

```
1 // pages/shopping/goods-list/index.js
2
3 Component({
4
5   /** 组件的方法列表 */
6   methods: {
7
8     /** 点击商品减号 - */
```

```
9 hanldeItemSub(e) {
10   const { goods } = e.currentTarget.dataset
11   // 购物车数据
12   const cartList = this.data.cartList || []
13
14   // 获取被点击商品在购物车中的下标
15   const targetIndex = cartList.findIndex(item => item._id === goods._id)
16   // 购物车中没有此商品，直接结束
17   if (targetIndex < 0) return
18
19   // 在购物车中有：获取购物车中商品
20   let target = cartList[targetIndex]
21   if (target.buyNum > 1) {
22     // 数量大于1，购买数量减1
23     target.buyNum--
24   } else {
25     // 购买数量小于等于1，则移除
26     cartList.splice(targetIndex, 1)
27   }
28
29   // 更新购物车数据
30   this.triggerEvent('updateCartList', {cartList})
31 },
32
33 },
34
35 })
```

修改数量对话框实现

点击商品购物数量值，调用 wx.showModal 函数弹出修改数量对话框





1. 在列表组件methods选项中定义handleItemNum方法：实现点击商品数量，弹出修改数量窗口

```
1 // pages/shopping/goods-list/index.js
2
3 import { getCategoryList, queryPageGoodsList } from '@api/goods'
4 import { getTotalPage, msg } from '@utils/util'
5
6 Component({
7
8   methods: {
9
10     // 点击商品数量，弹出修改数量窗口
11     async handleItemNum(e) {
12       const { goods } = e.currentTarget.dataset
13
14       // 弹出修改数量窗口
15       const { cancel, content } = await wx.showModal({
16         title: '请输入商品数量',
17         placeholderText: '请输入0到10000之间的数字',
18         editable: true, // 显示输入框
19         content: `${goods.buyNum} | |`, // 输入框内容：只能传字符串，传数字会报错
20       })
21       // 点击【取消】，结束不处理
22       if (cancel) return
23
24       // 点击【确定】
25       const updateNum = parseInt(content) // 字符串转数字
26       console.log('isNaN(', isNaN(updateNum), updateNum)
27       // 必须是数字且取值范围0到10000之间
28       if (isNaN(updateNum) || updateNum < 0 || updateNum > 10000) {
29         return msg('请输入有效数字', {icon: 'error'})
30       }
31     }
32   }
33 })
```

```
32     const {cartList = []} = this.properties
33     // 获取被点击商品在购物车中的商品
34     const targetIndex = cartList.findIndex(item => item._id === goods._id)
35     // 购物车中没有此商品, 直接结束
36     if (targetIndex < 0) return
37
38     const target = cartList[targetIndex]
39     if (updateNum > 0) {
40         // 大于0, 直接作为最新数量
41         target.buyNum = updateNum
42     } else {
43         // 小于等于0, 从购物车移除
44         cartList.splice(targetIndex, 1)
45     }
46
47     // 更新购物车数据
48     this.triggerEvent('updateCartList', {cartList})
49 },
50
51 }
52
53 })
```

购物车区域实现

在页面底部加上购物车展示, 效果如下:



首页



点单



订单

1. 页面购物车区域模板代码:

```
1 <!--pages/shopping/shopping.wxml-->
2
3 <!-- 顶部导航栏 -->
4 <!-- 轮播图 -->
```

```
5 <!-- 点单信息区域 -->
6 <!-- 分类+商品列表区域 -->
7
8 <!-- 底部购物车数量和价格。`showCartBtn` 为实现加购动画而生：用于获取购物车图标位置 -->
9 <view class="shopping-cart" wx:if="{{showCartBtn || (cartList &&
  cartList.length > 0) }}">
10   <view class="shopping-cart-wrapper flex-space-between">
11     <view class="flex-space-between-center" bind:tap="handleCart">
12       <view class="cart-img-view flex-center">
13         <image class="cart-img" src="/assets/images/cart.png"></image>
14         <view class="total-num">{{totalNum || 0}}</view>
15       </view>
16       <view class="center-content-box flex-1 flex-row-align-center">
17         <text class="total-amount">¥{{totalAmount || '0.00'}}</text>
18       </view>
19     </view>
20     <view class="right-opt flex-column-center" bind:tap="toOrderConfirm">
21       <text class="chinese-text">选好了</text>
22       <text class="english-text">Order</text>
23     </view>
24   </view>
25 </view>
```

2. 页面购物车区域样式代码：

```
1 /* pages/shopping/shopping.wxss */
2
3 /* 底部购物车区域 */
4 .shopping-cart {
5   position: fixed;
6   left: 0;
7   right: 0;
8   bottom: 0;
9   z-index: 110;
10  padding-bottom: calc(10px + env(safe-area-inset-bottom));
11  background: linear-gradient(to bottom, transparent, #fff);
12  .shopping-cart-wrapper {
13    margin: 0 30rpx;
14    background-color: var(--primary-color);
15    border-radius: 20rpx;
16    align-items: center;
17    color: #fff;
18    height: 100rpx;
19  }
20  .cart-img-view {
21    position: relative;
22    top: -20rpx;
```

```
23     margin: 0 40rpx;
24     width: 110rpx;
25     height: 110rpx;
26     border: 12rpx solid var(--primary-color);
27     border-radius: 50%;
28     background-color: #f9f9f9;
29     .cart-img {
30         width: 66rpx;
31         height: 66rpx;
32     }
33     .total-num {
34         position: absolute;
35         top: -4rpx;
36         right: -6rpx;
37         min-width: 38rpx;
38         text-align: center;
39         border-radius: 16rpx;
40         height: 26rpx;
41         line-height: 26rpx;
42         font-size: 20rpx;
43         font-weight: 700;
44         color: #fff;
45         background: #363535;
46         padding: 0 5rpx;
47         box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.4);
48     }
49 }
50 .center-content-box {
51     height: 100%;
52     .total-amount {
53         font-size: 48rpx;
54         font-weight: bold;
55     }
56 }
57 .right-opt {
58     height: 100%;
59     padding: 0 26rpx;
60     justify-content: center;
61     .chinese-text {
62         font-size: 36rpx;
63         font-weight: bold;
64     }
65     .english-text {
66         font-size: 26rpx;
67     }
68 }
69 }
```


3. 在页面 data 选择中定义 showCartBtn 属性，默认值 true 显示底部购物车按钮: 为实现加购动画而生，用于获取购物车图标位置

```
1 // pages/shopping/shopping.js
2
3 import { navTo } from '@utils/util'
4 import { getAdvertList } from '@api/advert'
5
6 Page({
7
8   data: {
9
10     cartList: [], // 购物车数量
11     totalNum: 0, // 总数量
12     totalAmount: 0, // 总金额
13     // ++++++
14     showCartBtn: true, // 默认显示底部购物车按钮：为实现加购动画而生，用于获取购物车图标
    位置
15
16   },
17
18 })
```

购物车明细弹窗实现



1. **shoopng.json** 页面配置中注册组件：弹出层 van-popup、步进器 van-stepper

pages/shopping/shopping.json

```
1 {
2   "navigationStyle": "custom",
3   "onReachBottomDistance": 300,
4   "usingComponents": {
5
6     "van-popup": "@vant/weapp/popup/index",
7     "van-stepper": "@vant/weapp/stepper/index"
8   }
9 }
```

2. 页面购物车明细弹窗模板代码:

```
1 <!--pages/shopping/shopping.wxml-->
2
3 <!-- 解决：打开购物车明细弹窗时，在弹窗体上下滑动会滚动页面，加上此项就不会滚动页面，
4     注意：必须加要页面的第一行代码 -->
5 <page-meta page-style="{{ showCart ? 'overflow: hidden;' : '' }}" />
6
7
8 <!-- 顶部导航栏 -->
9 <!-- 轮播图 -->
10 <!-- 点单信息区域 -->
11 <!-- 分类+商品列表区域 -->
12 <!-- 底部购物车数量和价格 -->
13
14 <!-- 购物车明细弹窗 -->
15 <van-popup
16   show="{{ showCart }}"
17   round
18   closeable
19   position="bottom"
20   close-icon="close"
21   bind:close="closeCart"
22 >
23   <view class="cart-popup-wrapper">
24     <view class="cart-title-box">
25       <view bind:tap="clearCart" class="clearall-cart-view flex-row-align-
26         center" hover-class="view-hover">
27         <text class="iconfont icon-clear"></text>
28         <text>清空购物车</text>
29       </view>
30     </view>
31     <scroll-view scroll-y style="max-height: 55vh;">
32       <view class="cart-item flex-row" wx:for="{{cartList}}" wx:key="index">
33         <view class="cart-item-left flex-center">
34           <image class="cart-item-img" src="{{item.imageUrl}}" />
35         </view>
36       </view>
37     </scroll-view>
38   </view>
39 </van-popup>
```

```
35     <view class="cart-item-right flex-1">
36         <view class="item-content">
37             <text class="item-name">{{item.goodsName}}</text>
38             <text class="item-unit">{{item.specsName || item.unit}}</text>
39         </view>
40         <view class="item-price-num flex-space-between-center">
41             <view class="item-price flex-row-align-center">
42                 <view class="sale-price flex-center">
43                     <text class="sale-price-symbol">¥</text>
44                     <text class="sale-price-value">{{item.salePrice}}</text>
45                 </view>
46                 <text class="original-price" wx:if="{{item.discountPrice}}">¥
47                 {{item.price}}</text>
48             </view>
49             <view class="item-num-box">
50                 <van-stepper value="{{ item.buyNum }}"
51                 bind:change="changeCartNum" data-index="{{index}}"
52                 disable-input min="0" max="9999" theme="round" />
53             </view>
54         </view>
55     </scroll-view>
56 </view>
57 </van-popup>
58
```

3. 页面购物车明细弹窗样式：

```
1  /* pages/shopping/shopping.wxss */
2
3  /* 底部购物车区域 */
4
5  /* 底部购物车明细弹窗 */
6  .cart-popup-wrapper {
7      padding-bottom: 160rpx;
8      .cart-title-box {
9          background-color: inherit;
10         width: 100%;
11         height: 50px;
12         // 清空购物车
13         .clearall-cart-view {
14             height: 100%;
15             width: 180rpx;
16             margin-left: 16px;
17             color: #969799;
18             font-size: 26rpx;
```

```
19     .iconfont {
20         font-size: 33rpx;
21         line-height: 33rpx;
22     }
23 }
24 }
25
26 .cart-item {
27     margin: 0 20rpx;
28     padding: 20rpx 0;
29     border-bottom: 1px solid #f9f9f9;
30     .cart-item-left {
31         .cart-item-img {
32             width: 140rpx;
33             height: 110rpx;
34         }
35     }
36     .cart-item-right {
37         margin-left: 16rpx;
38         .item-content {
39             &> text {
40                 display: block;
41             }
42             .item-name {
43                 font-size: 26rpx;
44                 color: #222222;
45                 font-weight: 500;
46             }
47             .item-unit {
48                 font-size: 20rpx;
49                 color: #999;
50             }
51         }
52         .item-price-num {
53             .item-price {
54                 .sale-price {
55                     color: red;
56                     align-items: flex-end;
57                 }
58                 .sale-price-symbol {
59                     font-size: 24rpx;
60                     line-height: 28rpx;
61                 }
62                 .sale-price-value {
63                     font-weight: 500;
64                     font-size: 40rpx;
65                     line-height: 36rpx;
66                 }
67             }
68             .original-price {
```

```
68         margin-left: 8rpx;
69         color: #828282;
70         text-decoration: line-through;
71         font-size: 23rpx;
72     }
73 }
74 }
75 }
76 }
77 }
```

4. 实现购物车中改变数量，购物车弹窗显示、隐藏，清空购物车数据，

```
1  // pages/shopping/shopping.js
2
3  import { navTo } from '@utils/util'
4  import { getAdvertList } from '@api/advert'
5
6  Page({
7
8      data: {
9          showCartBtn: true, // 默认显示底部购物车按钮：为实现加购动画而生，用于获取购物车图标
           位置
10         showCart: false, // 显示购物车明细弹窗 ++++++
11     },
12
13     // 购物车中改变数量
14     changeCartNum(e) {
15         // console.log('changeCartNum', e)
16         // 点击的最新数量
17         const { detail: buyNum } = e
18         // 点击的购物车商品下标
19         const { index } = e.currentTarget.dataset
20         // 购物车数据
21         const { cartList } = this.data
22         if (buyNum < 1) {
23             // 小于1，从购物车移除
24             cartList.splice(index, 1)
25         } else {
26             // 大于等于1，更新为新数量
27             cartList[index].buyNum = buyNum
28         }
29
30         // 购物车无数据则隐藏它，反之继续保持显示
31         const showCart = cartList && cartList.length > 0
32
33         this.setData({ cartList, showCart })
34     },
35 })
```

```
36 // 显示购物车
37 handleCart() {
38   this.setData({showCart: !this.data.showCart})
39 },
40
41 // 隐藏购物车
42 closeCart() {
43   this.setData({showCart: false})
44 },
45
46 // 清空购物车数据
47 clearCart() {
48   this.setData({cartList: [], showCart: false})
49 },
50
51 })
```

商品详情页面实现-SKU多规格商品选择

商品详情页面相关配置

1. 在 app.json 中添加页面配置：

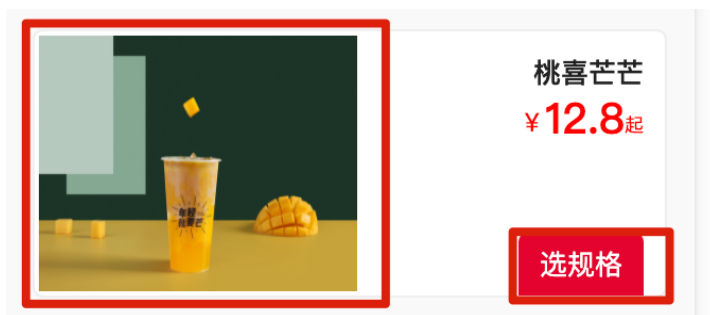
```
1 {
2   "pages": [
3
4     "pages/goods-detail/goods-detail"
5   ],
6 }
```

2. 在 goods-detail.json 中配置导航栏，注册 van-stepper 组件

```
1 {
2   "navigationBarTitleText": "",
3   "backgroundColor": "#fff",
4   "usingComponents": {
5     "van-stepper": "@vant/weapp/stepper/index"
6   }
7 }
```

点击商品跳转商品详情页

点击商品图片和选规格，跳转到商品详情页进行选择商品规格



代码实现: pages/shopping/goods-list/index.js

```
1 // 前往商品详情页，选择商品规格
2 toGoodsDetail(e) {
3   const { item } = e.currentTarget.dataset
4   navTo("/pages/goods-detail/goods-detail?goods=" + JSON.stringify(item), {
5     events: { // events 添加一个监听事件，用于获取被打开页面传送到当前页面的数据
6       addToCart: (goods) => { // goods-detail.js 中触发了此监听事件
7         // 选择的商品添加到购物车，参数 e.currentTarget.dataset.goods
8         this.hanldeItemAdd({ currentTarget: { dataset: { goods } } })
9       }
10    }
11  })
12 },
```

商品详情页效果实现



商品多规格JSON数据格式：

price是规格附加费，最终售价为：商品价格+规格附加费；cost是成本价

```
1 goodsSpecs:
2 [
3   {
4     "_id": "001", "specsData": {"份量": "中杯", "温度": "常温"}, "price": 1, "cost": 4,
    "stockNum": 1000
5   },
6   {
7     "_id": "002", "specsData": {"份量": "大杯", "温度": "少冰"}, "price": 2, "cost": 4,
    "stockNum": 200
8   }
9 ]
```

要将规格数据转换成视图展示需要的数据格式：对象形式 specsAttrs (able 属性: true 表示是否可选)

```
1 specsAttrs:
2 {
3   "份量": [
4     {attrValue: '中杯', able: true},
5     {attrValue: '大杯', able: true}
6   ],
7   "温度": [
8     {attrValue: '常温', able: true},
9     {attrValue: '少冰', able: true}
10  ]
11 }
```

1. 接收商品数据，并转换成视图展示数据。

注意：因为要使用监听器observers，Page中无法使用，所以使用 Component 组件方式构建页面

```
1 // pages/goods-detail/goods-detail.js
2 // Page({ })
3 // 使用Component构建页面，因为要使用监听器 observers
4 Component({
5
6   data: {
7     goods: {}, // 商品信息
8     specsAttrs: {}, // 规格属性，用于页面展示
9     buyNum: 1, // 购买数量
10    selectedSku: {}, // 已选择的 sku
11    targetSpecs: null, // 最终选择的商品目标规格
12    notSeletedAttrNames: [], // 未选择的规格属性名
13  },
14
15   methods: {
16
17     /** 生命周期函数 必须放在 Component 的methods选项中 */
18     onLoad(options) {
```

```
19     const goods = JSON.parse(options.goods || "{}")
20     // console.log('goods', options.goods)
21     this.setData({ goods })
22
23     // 商品名称设置为导航标题
24     wx.setNavigationBarTitle({title: goods.goodsName})
25
26     // 商品规格数据转换为页面展示所需的数据
27     this.formatSpecsData()
28   },
29
30   /**
31    * 商品规格数据转换为页面展示所需的数据
32    */
33   formatSpecsData() {
34     const { goodsSpecs } = this.data.goods
35     if (!goodsSpecs || goodsSpecs.length <= 0) return
36
37     // 存格式后的规格属性 key是规格类型, value是此类型下所有的规格名称
38     const specsAttrs = {}
39     goodsSpecs.forEach(specs => {
40       const { specsData } = specs
41       Object.keys(specsData).forEach(key => {
42         // 获取当前规格类型已有的规格名称
43         const values = specsAttrs[key] || []
44         // 是否已经存在此规格名称
45         const exist = values.some(v => v.attrValue == specsData[key])
46         if (!exist) {
47           values.push({attrValue: specsData[key], able: true})
48           specsAttrs[key] = values
49         }
50       })
51     })
52     // console.log('specsAttrs', specsAttrs)
53
54     this.setData({ specsAttrs })
55   },
56
57   /**
58    * 用户点击右上角分享
59    */
60   onShareAppMessage() {
61
62   }
63
64 },
65
66 })
67
```

2. 商品详情页面模板代码：

```
1 <!--pages/goods-detail/goods-detail.wxml-->
2
3 <view class="goods-specs-container">
4   <image class="goods-img" src="{{goods.imageUrl}}" />
5
6   <view class="basic-box">
7     <text>{{goods.goodsName}}</text>
8   </view>
9
10  <!-- 商品规格 -->
11  <view class="goods-specs">
12    <view class="specs-item flex-row"
13      wx:for="{{specsAttrs}}" wx:for-index="attrName" wx:for-item="values"
14      wx:key="attrName">
15      <text class="specs-type">{{attrName}}</text>
16      <view class="specs-values flex-row-wrap">
17        <text wx:for="{{values}}" wx:key="index"
18          bind:tap="changeSpecs" data-key="{{attrName}}"
19          data-value="{{item.attrValue}}" data-able="{{item.able}}"
20          class="specs-value
21            {{selectedSku[attrName] === item.attrValue ? 'active' :
22              ''}}
23            {{item.able ? '' : 'disabled'}}"
24          >{{item.attrValue}}</text>
25        </view>
26      </view>
27    </view>
28
29    <view class="fixed-bottom">
30      <view class="buy-info-box flex-row-align-center">
31        <view class="flex-column" hidden="{{!targetSpecs}}">
32          <!-- <text class="price">¥7.9</text>
33          <text class="specs">中怀/少冰</text> -->
34          <text class="price">¥{{(goods.discountPrice || goods.price) +
35            targetSpecs.price}}</text>
36          <text class="specs">{{targetSpecs.specsName}}</text>
37        </view>
38        <van-stepper class="buy-num" value="{{buyNum}}"
39          bind:change="changeBuyNum" disable-input min="1" max="9999"
40          theme="round" />
41      </view>
42      <button wx:if="{{notSeletedAttrNames.length>0}}" disabled type="primary">
43        请选择商品规格</button>
44      <button wx:else type="primary" bind:tap="handelAddCart">加入购物车</button>
45    </view>
```

3. 商品详情页面样式代码:

```
1  /* pages/goods-detail/goods-detail.wxss */
2  page {
3    background-color: #fff;
4  }
5
6  .goods-specs-container {
7    .goods-img {
8      width: 100%;
9      height: 500rpx;
10   }
11   .basic-box {
12     font-size: 36rpx;
13     font-weight: bold;
14     padding: 26rpx 30rpx;
15     border-bottom: var(--basic-border);
16   }
17
18   .goods-specs {
19     padding: 30rpx 30rpx 0 30rpx;
20     .specs-item {
21       font-size: 30rpx;
22       line-height: 58rpx;
23       height: 58rpx;
24       padding: 16rpx 0;
25       margin-bottom: 20rpx;
26       &.error {
27         background-color: #ffe6e6;
28       }
29       .specs-type {
30         font-size: inherit;
31         font-weight: bold;
32         white-space: nowrap;
33         min-width: 100rpx;
34       }
35       .specs-value {
36         font-size: 28rpx;
37         padding: 0 16rpx;
38         border: 1px solid #a1a1a1;
39         border-radius: 10rpx;
40         margin-left: 20rpx;
41         margin-bottom: 20rpx;
42         white-space: nowrap;
43         min-width: 130rpx;
44         color: #777777;
45       }
46     }
47   }
48 }
```

```
46     .active {
47         color: var(--primary-color);
48         border-color: var(--primary-color);
49         background-color: rgba(250, 145, 145, 0.1);
50     }
51     .disabled {
52         color: #999;
53         border: 1px dashed #ccc;
54         background-color: rgba(204, 204, 204, 0.3);
55     }
56 }
57 }
58
59 .buy-info-box {
60     margin-bottom: 20rpx;
61     .price {
62         font-size: 28rpx;
63         font-weight: bold;
64     }
65     .specs {
66         font-size: 24rpx;
67         color: #666666;
68     }
69     .buy-num {
70         margin-left: auto;
71     }
72 }
73
74 }
```

判断规格名称是否可选

1. 判断其他类型的值是否可选：可选(able: true)、不可选(able: false)

- 可选：如果已经选择过，说明再次点击，则取消选中；反之添加到选择的SKU中
- 不可选：选择的SKU对象中只保留当前的规格名称

2. 再次判断规格是否可以被选择

- 循环每个规格类型下的规格值
- 循环的每一个规格，与选中sku对象的合并，
- 查看合并的sku对象的每个属性，是否在商品规格数据中有相等的属性值的对象，且有库存，则可选

3. 根据选中的sku，查找商品目标规格对象

```
1  changeSpecs (e) {
2      const { key, value, able } = e.currentTarget.dataset
3
4      let {goods, specsAttrs, selectedSku = {}} = this.data
5  }
```

```
6 // 判断当前点击的规格值是否可选的
7 if(able) {
8   // 当前点击的是可选的：如果已存在于selectedSku说明再次点击，则取消选中，反之添加属性
9   if(selectedSku[key] == value){
10     delete selectedSku[key] // 取消选中
11   } else {
12     selectedSku[key] = value
13   }
14 } else {
15   // 当前点击的是不可选的：选择的sku对象中只保留当前的规格属性
16   selectedSku = {[key]: value}
17 }
18
19 /*
20 * 判断规格是否可以被选择
21 * 1. 循环每个规格类型下的规格值
22 * 2. 循环的每一个规格，与选中sku对象的合并，
23 * 3. 查看合并的sku对象的每个属性，是否在商品规格数据中有相等的属性值的对象，且有库存，则可
    选
24 */
25 const { goodsSpecs } = goods
26 for (let attrName in specsAttrs) {
27   // 1. 循环每个规格类型下的规格值
28   specsAttrs[attrName].forEach(attr => {
29     // 2. 循环的每一个规格，与选中sku对象的合并
30     // selectedSku: {"份量": "中杯"}
31     // everySku: {"份量": "中杯"}、{"份量": "大杯"}、 {"份量": "中杯", 温度: "常温"}、
    {"份量": "中杯", 温度: "少冰"}
32     const everySku = {...selectedSku, [attrName]: attr.attrValue}
33     // console.log('everySku', everySku)
34     // 3. 查看合并的sku对象的每个属性，是否在商品规格数据中有相等的属性值的对象，且有库
    存，则可选
35     attr.able = goodsSpecs.some(gs => {
36       return Object.keys(everySku).every(key => gs.specsData[key] ==
    everySku[key])
37       && gs.stockNum > 0 // 且商品规格对象中的库存要大于0才可选
38     })
39   })
40 }
41
42 // 根据选中的sku，查找商品目标规格对象
43 const targetSpecs = goodsSpecs.find(gs => equalsObj(selectedSku,
    gs.specsData)) || null
44 // 多个规格值使用/拼接，用于页面展示
45 if (targetSpecs) targetSpecs.specsName =
    Object.values(targetSpecs.specsData).join('/')
46
47 // 更新状态
48 this.setData({specsAttrs, selectedSku, targetSpecs})
```



```
49  
50     // console.log('selectedSku', selectedSku, targetSpecs)  
51 },
```

获取购买数量和监听选择的SKU

1. 获取购买数量

```
1 // pages/goods-detail/goods-detail.js  
2  
3 // 使用Component构建页面，因为要使用监听器 observers  
4 Component({  
5  
6   data: {  
7     buyNum: 1, // 购买数量  
8   },  
9  
10  methods: {  
11    // 购买数量  
12    changeBuyNum(e) {  
13      this.setData({buyNum: e.detail || 1})  
14    },  
15  },  
16  
17 })
```

2. 监听 selectedSku 选择的SKU变化：收集还没有被选择的规格属性名

```
1 // pages/goods-detail/goods-detail.js  
2  
3 // 使用Component构建页面，因为要使用监听器 observers  
4 Component({  
5  
6   data: {  
7     goods: {}, // 商品信息  
8     buyNum: 1, // 购买数量  
9     specsAttrs: {}, // 规格属性，用于页面展示  
10    selectedSku: {}, // 已选择的 sku  
11    targetSpecs: null, // 最终选择的商品目标规格  
12    notSeletedAttrNames: [], // 未选择的规格属性名  
13  },  
14  
15  // 监听器  
16  observers: {  
17    // 监听选择的SKU变化：收集还没有被选择的规格属性名，用于条件判断
```

```
18     selectedSku(selectedSku) {
19         const { specsAttrs } = this.data
20         // 未选择的规格属性名
21         const notSeletedAttrNames = Object.keys(specsAttrs).filter(attrName =>
22             !!!selectedSku[attrName])
23         this.setData({ notSeletedAttrNames })
24     },
25
26     methods: {
27
28     },
29
30 })
```

3. 加入购物车实现：

- 判断多规格商品是否选择了规格
- 计算：商品规格单价 = 商品单价 + 规格附加费，商品规格特价 = 商品特价 + 规格附加费
- 传递选择的传递商品规格数据，后返回购物页

```
1     methods: {
2         // 加入购物车
3         handelAddCart() {
4             const { goods, buyNum, targetSpecs, notSeletedAttrNames } = this.data
5
6             // 选择的商品信息
7             let selectedGoods = {...goods, buyNum}
8
9             // 多规格商品
10            if (goods.goodsSpecs && goods.goodsSpecs.length > 0) {
11                if (!targetSpecs || (notSeletedAttrNames &&
12                    notSeletedAttrNames.length)) {
13                    msg('请选择商品SKU')
14                    return
15                }
16
17                // 封装选择的规格商品数据
18                const { _id: specsId, price: specsPrice, specsName, cost, stockNum }
19                = targetSpecs || {}
20
21                // 商品规格单价 = 商品单价 + 规格附加费 specsPrice
22                selectedGoods.price = (selectedGoods.price || 0) + (specsPrice || 0)
23                // 商品规格特价 = 商品特价 + 规格附加费 specsPrice
24                selectedGoods.discountPrice = (selectedGoods.discountPrice || 0) +
25                (specsPrice || 0)
26
27                // 选择的商品规格信息
```

```
25         selectedGoods = { ...selectedGoods, specs: targetSpecs, specsId,  
26         specsName, cost, stockNum}  
27     }  
28     // 触发来源页面（购物页）的 addToCart 监听事件，传递商品规格数据  
29     // 在 /pages/shopping/goods-list/index.js 中的toGoodsSpecs方法中定义了  
addToCart监听事件  
30     const eventChannel = this.getOpenerEventChannel()  
31     eventChannel.emit('addToCart', selectedGoods)  
32     // 后退回购物页  
33     navBack()  
34  
35     },  
36 }
```

加入购物车动画效果

当点击单规格商品的加号 + 加入购物车时，会出现抛物线动画从点击位置开始到购物车图标位置结束



1. 购物页面的 onReady 中获取购物车图标的坐标信息

```
1 // pages/shopping/shopping.js
2
3 Page({
4
5   data: {
6     animaPosition: { // 加入购物车动画坐标信息
7       startX: 0, // 被点击的+元素坐标（抛物线开始位置）
8       startY: 0,
9       endX: 0, // 抛物线动画结束位置（购物车图标坐标）
10      endY: 0
11    }
12  },
13
```

```
14  /** 监听页面初次渲染完成 */
15  onReady() {
16    this.goodsListComponent = this.selectComponent('#goods-list-id')
17
18    // 购物车图标坐标信息 ++++
19    wx.createSelectorQuery().select('.cart-img-view').boundingClientRect((res)
=> {
20      this.setData({
21        "animaPosition.endX": res.left + res.width/3,
22        "animaPosition.endY": res.top,
23        showCartBtn: false // 获取到位置后，隐藏底部购物车按钮
24      })
25    }).exec() // 最后不要少了.exec()
26
27  },
28
29  })
```

2. 获取点击 + 位置的坐标信息和被点击的商品信息，获取后将数据传递给父组件

在列表组件 `pages/shopping/goods-list/index.js` 中的 `hanldeItemAdd` 方法中可获取：

```
1  // pages/shopping/shopping.js
2  Page({
3
4    /** 点击商品加号 + */
5    hanldeItemAdd(e) {
6      const { goods } = e.currentTarget.dataset
7      // 此处省略上面实现过的加入购物车业务代码
8
9      // 触发父组件的开始动画事件，goods 用于抛物线显示图片
10     if (e.touches) {
11       const { clientX, clientY } = e.touches[0]
12       this.triggerEvent('startAnimation', {goods, clientX, clientY})
13     }
14   },
15 })
```

3. 在购物页面模板中的 `<goods-list>` 组件标签上绑定开始动画事件

`bind:startAnimation="startAnimation"`

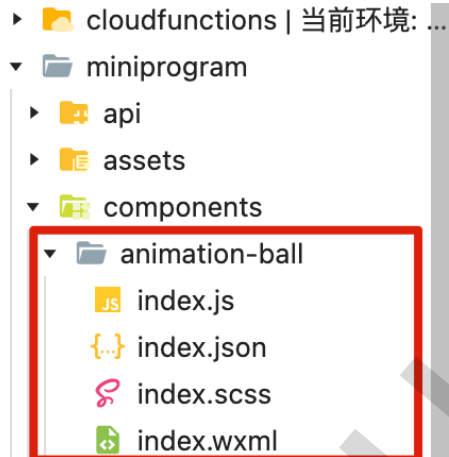
```
1 <!--pages/shopping/shopping.wxml-->
2
3 <!-- 分类+商品列表区域 -->
4 <goods-list
5   id="goods-list-id"
6   sticky-top="{{stickyTop}}"
7   model:total-num="{{totalNum}}"
8   model:total-amount="{{totalAmount}}"
9   cart-list="{{cartList}}"
10  bind:updateCartList="updateCartList"
11  bind:startAnimation="startAnimation"
12 />
```

4. 购物页面定义事件函数：接收子组件传递的点击商品和动画开始的坐标数据；加入购物车动画完成后更新startAddAnima为false

```
1 // pages/shopping/shopping.js
2 Page({
3
4   data: {
5     startAddAnima: false, // 是否开始加入购物车动画
6     animaPosition: { // 加入购物车动画坐标信息
7       startX: 0, // 被点击的+元素坐标（抛物线开始位置）
8       startY: 0,
9       endX: 0, // 抛物线结束位置（购物车图标坐标）
10      endY: 0
11    },
12    currGoods: {}, // 当前点击的商品
13  },
14
15  /** 开始动画，商品列表子组件触发调用此事件方法 */
16  startAnimation (e) {
17    // console.log('startAnimation', e)
18    const {goods, clientX, clientY} = e.detail
19    this.setData({
20      startAddAnima: true,
21      currGoods: goods,
22      "animaPosition.startX": clientX,
23      "animaPosition.startY": clientY
24    })
25  },
26
27  /** 加入购物车动画完成后触发 */
28  animationDone() {
29    this.setData({startAddAnima: false})
30  },
31
32 })
```

5. 将动画组件导入到项目中:

复制资料中的 03-配套资料/点餐小程序资料/components/animation-ball 到项目中的 components 目录下



6. 在购物页面 shopping.json中注册动画组件

```
1 {
2   "navigationStyle": "custom",
3   "onReachBottomDistance": 300,
4   "usingComponents": {
5
6     "animation-ball": "/components/animation-ball/index"
7   }
8 }
```

7. 在购物页面 shopping.wxml 中引用动画组件

```
1 <!-- 加入购物车动画组件 -->
2 <animation-ball
3   startAnima="{{startAddAnima}}"
4   imageUrl="{{currGoods.imageUrl}}"
5   animaPosition="{{animaPosition}}"
6   bind:animationDone="animationDone"
7 />
```

分类和商品滚动相互联动效果

相互联动效果需求:

1. 当点击左侧某个分类时，右侧自动滚动到所属这个分类的第1个商品处。
2. 当滚动商品列表时，左侧分类自动高亮此商品所属的分类



点击分类联动商品列表滚动

当点击左侧某个分类时，右侧自动滚动到所属这个分类的第1个商品处。

1. 在商品列表组件中的 handleCategory 方法中，实现点击分类时：页面滚动到此分类的商品处

```
1 // pages/shopping/goods-list/index.js
2
3 Component({
4
5   methods: {
6
7     /** 点击分类：高亮显示 */
8     handleCategory(e) {
9       const { item } = e.currentTarget.dataset
10      // 高亮显示分类
11      this.setData({activatedCategoryId: item.categoryId})
12
13      // 点击分类时，页面滚动到此分类的商品处
14      this.tapCategoryScrollGoodsList(item)
15    },
16
17    // 点击分类时，页面滚动到此分类的商品处
18    tapCategoryScrollGoodsList(item) {
19
20      // 标识是点击分类时，进行的页面滚动；而不是人工滑动页面滚动的
```

```

21      this.isTapCategory = true
22
23      // 将页面滚动到此分类的商品锚点处
24      wx.pageScrollTo({
25        // 在子组件使用，selector 需要使用`跨自定义组件的后代选择器` 如 #goods-list-
        id >>> .right111
26        // 注意：#goods-list-id是父页面中的组件元素id值(或class值)，>>> 左右两边有英文
        // 空格，.right111是当前组件中的元素id值（或class值）
27        selector: `#goods-list-id >>> .right${item.categoryId}`,
28        offsetTop: -this.properties.stickyTop - 15, // margin-top: 15px; 偏移
        // 距离，需要和 selector 参数搭配使用，可以滚动到 selector 加偏移距离的位置，单位 px
29        complete: () => {
30          // 滚动完成，设置为false后，如果页面滚动会调用rightScroll触发锚点
31          setTimeout(() => {
32            this.isTapCategory = false
33          }, 300)
34        }
35      })
36    },
37  }
38
39 })
  
```

2. 在 goods-list/index.wxml 中右侧商品模板代码处，添加一个 class 值 right{{item.categoryId}}，如下：

```

<!-- 右侧商品列表 -->
<view class="goods-list-wrapper flex-1">
  <view class="divider-box">
    <image src="/assets/images/leaf.png"/>
    <text>珍惜资源 按需点单</text>
    <image src="/assets/images/leaf.png"/>
  </view>
  <view class="goods-item right{{item.categoryId}}" wx:for="{{goodsList}}" wx:key="_id">
    <view class="goods-item-inner flex-space-between">
  
```

3. 核实 shopping.wxml 中是否有 <goods-list id="goods-list-id" />

```

<!-- 分类+商品列表区域 -->
<goods-list
  id="goods-list-id"
  sticky-top="{{stickyTop}}"
  model:total-num="{{totalNum}}"
  model:total-amount="{{totalAmount}}"
  cart-list="{{cartList}}"
  bind:updateCartList="updateCartList"
>
  
```

滚动商品列表联动高亮分类

当页面滚动时，也就是右侧商品列表滚动时，左侧分类自动高亮此商品所属的分类

1. 在购物页面监听页面滚动事件 `onPageScroll`

```
1 // pages/shopping/shopping.js
2
3 Page({
4
5   /** 监听页面初次渲染完成 */
6   onReady() {
7     this.goodsListComponent = this.selectComponent('#goods-list-id')
8   },
9
10
11
12   // 监听用户滑动页面事件
13   onPageScroll(e) {
14     // 调用商品列表子组件中的pageScrollChangeCategory方法实现高度对应商品分类名
15     this.goodsListComponent?.pageScrollChangeCategory(e)
16   },
17
18 })
```

2. 在商品列表子组件中定义 `pageScrollChangeCategory` 方法，实现高度对应商品分类名

```
1 // 监听页面滚动：右侧商品列表滚动时，切换对应的分类
2 pageScrollChangeCategory() {
3   // 如果是点击分类直接return，就不触发后面的滚动逻辑
4   if(this.isTapCategory) return
5
6   // 当前高亮的分类id，列表上方区域高度
7   let { activeCategoryId, stickyTop } = this.data
8
9   // 1. 获取商品列表中每个商品块到顶部的距离，
10  // 注意：`组件中`要用 `this.createSelectorQuery`，不能用
11  // `wx.createSelectorQuery`
12  this.createSelectorQuery().selectAll('.goods-item').boundingClientRect((res) => {
13    res.forEach(rect => {
14      // 2. 计算每个商品距离顶部的高度 - 列表上方区域高度 = 每个商品距离头部区域底线的
15      // 高度
16      // 差值小于等于0说明：当前展示的就是此商品的所属分类，则高亮此商品的分类
17      if (rect.top - stickyTop <= 0) {
18        activeCategoryId = rect.dataset.categoryId
19      }
20    })
21  })
```

```

18      })
19      // 更新高亮分类id
20      this.setData({ activedCategoryId })
21    }).exec()
22  },

```

3. 在 goods-list/index.wxml 中右侧商品列表代码处，

添加一个绑定分类id的属性 `data-category-id="{{item.categoryId}}"`，如下：

```

<!-- 右侧商品列表 -->
<view class="goods-list-wrapper flex-1">
  <view class="divider-box">
    <image src="/assets/images/leaf.png"/>
    <text>珍惜资源 按需点单</text>
    <image src="/assets/images/leaf.png"/>
  </view>
  <view class="goods-item right{{item.categoryId}}" wx:for="{{goodsList}}" wx:key="_id" data-category-id="{{item.categoryId}}">
    <view class="goods-item-inner flex-space-between">

```

监听分类高亮时，滑动分类滚动条位置

当分类滚动条滚动后，当前选中的高亮分类名称可能会被遮挡住，我们应该始终让当前高亮的分类展示出来。

使用监听器监听高亮分类id的变化，计算分类滚动条的位置，从而进行移动显示出来。



1. 获取左侧分类滚动条的总高度和分类项的高度 `getLeftScrollHeight`，在 `loadCategoryList` 调用

```

1  // pages/shopping/goods-list/index.js
2  Component({
3
4    data: {
5      leftScrollHeight: 0, // 左侧分类滚动条的总高度
6      leftScrollItemHeight: 0, // 分类项的高度
7    },
8
9    methods: {

```

```
10
11 // 查询所有分类
12 async loadCategoryList() {
13   // 省略查询所有分类代码
14
15   // +++ 添加 this.getLeftScrollHeight 获取左侧分类区域的高度
16   this.setData({ categoryList, activedCategoryId },
17   this.getLeftScrollHeight)
18 },
19
20 /** 获取左侧分类滚动条的总高度和分类项的高度，在查询分类数据后调用 */
21 getLeftScrollHeight() {
22   // 获取左侧分类滚动条的总高度
23   const query = this.createSelectorQuery()
24   query.select('.category-scroll').boundingClientRect(rect => {
25     this.setData({leftScrollHeight: Math.round(rect.height)})
26   }).exec()
27
28   // 获取左侧分类项的高度
29   query.select('.category-item').boundingClientRect(rect => {
30     this.setData({leftScrollItemHeight: Math.round(rect.height)})
31   }).exec()
32 },
33 }
```

2. 监听 activedCategoryId 变化：来判断是否移动左侧分类滚动条，防止分类被遮挡

```
1 // pages/shopping/goods-list/index.js
2 Component({
3
4   data: {
5     leftScrollHeight: 0, // 左侧分类滚动条的总高度
6     leftScrollItemHeight: 0, // 分类项的高度
7     leftScrollTop: 0, // 左侧分类滚动条的位置
8   },
9
10  // 监听器
11  observers: {
12
13    // 监听被选中的分类id：来判断是否移动左侧分类滚动条，防止分类被遮挡
14    activedCategoryId (activatedCategoryId) {
15      // 解构：所有分类数据，每个分类块的高度，分类区域总高度，
16      const {categoryList, leftScrollItemHeight, leftScrollHeight} = this.data
17      // 查找目标分类的下标
18      const index = categoryList.findIndex(item => item.categoryId ==
19      activedCategoryId)
```

```
19 // 目标分类距离 <scroll-view> 顶部的总高度
20 const offsetTop = index * leftScrollItemHeight
21 // 因为触发滚动的条件是，目标分类的总高度 offsetLeft，超过 <scroll-view> 总高度
   的一半之后才开始滚动，所以除以2
22 const leftScrollTop = offsetTop - leftScrollHeight/2
23 // 触发滚动
24 this.setData({ leftScrollTop })
25 }
26
27 },
28
29
30 }
```

3. 在商品列表组件模板中的 `<scroll-view>` 组件上绑定属性 `scrollTop="{{leftScrollTop}}"`

```
1 <scroll-view class="category-scroll"
2   scroll-y
3   enable-passive
4   show-scrollbar="{{false}}"
5   scrollTop="{{leftScrollTop}}"
6   >
```

选好了前往确认订单页

1. 点击 选好了，前往订单确认页 `pages/order-confirm/order-confirm`

```
1 // pages/shopping/shopping.js
2
3 import { msg, navTo } from '@utils/util'
4 Page({
5
6   // 选好了，前往订单确认页面
7   toOrderConfirm() {
8     const { shop, address, takeType, totalNum, totalAmount, cartList } =
this.data
9
10    if (!shop || !shop._id) return msg('请选择店铺')
11    if (!cartList || cartList.length<=0) return msg('购物车不能为空')
12
13    // 隐藏购物车明细窗口
14    this.setData({showCart: false})
15
16    // 前往订单确认页面，并添加一个清空购物车监听事件
17    const orderData = JSON.stringify({ shop, address, takeType, totalNum,
totalAmount, cartList })
```



```
18     navTo('/pages/order-confirm/order-confirm?orderData=' + orderData, {
19       events: { // events 添加一个监听事件，用于获取被打开页面传送到当前页面的数据
20         clearCartData: () => { // order-confirm.js 中触发了此监听事件
21           // 清空购物车
22           this.clearCart()
23         }
24       }
25     })
26
27   },
28
29 })
```

2. app.json 配置创建订单确认页面

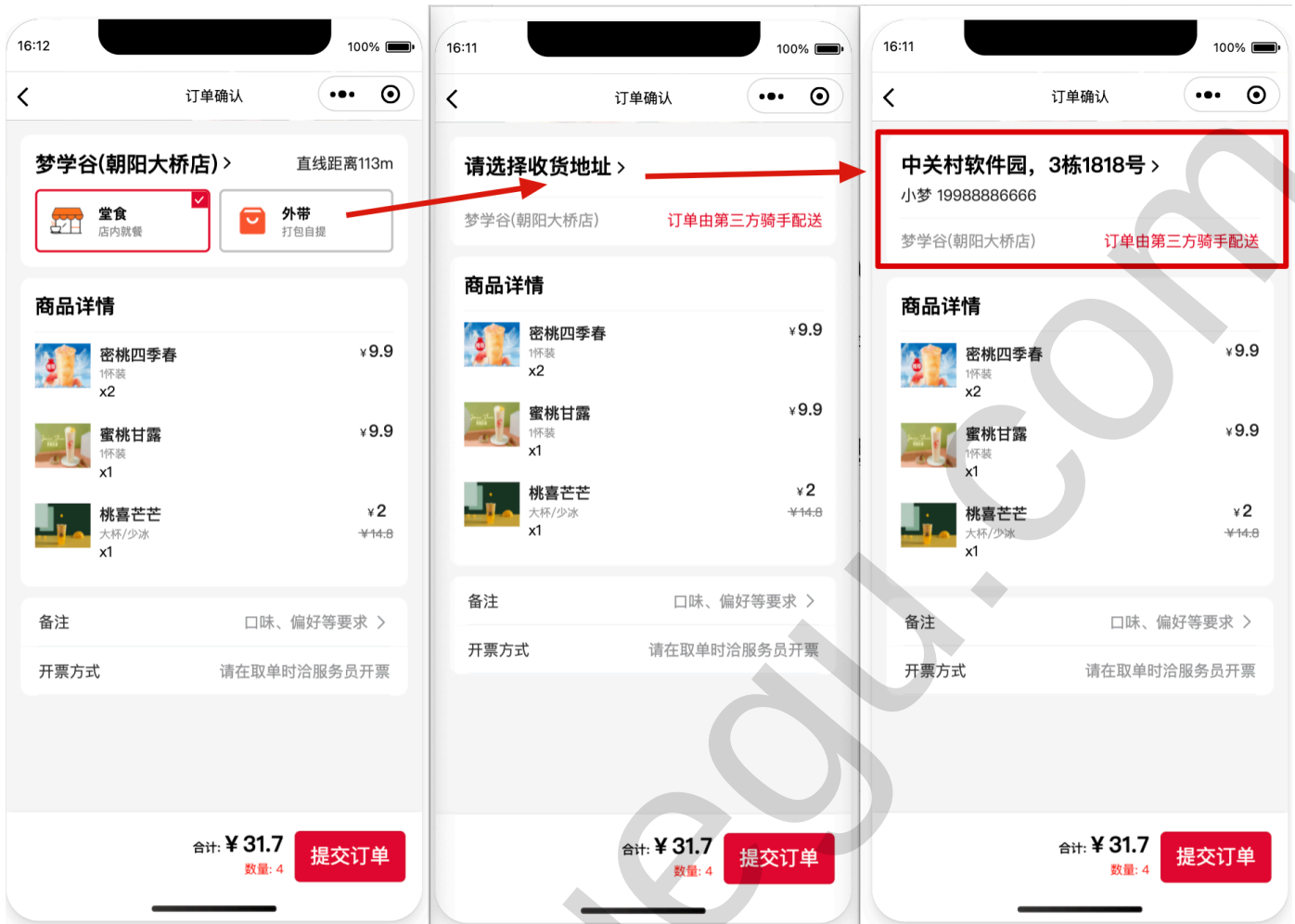
```
1  {
2    "pages": [
3
4      "pages/order-confirm/order-confirm"
5    ],
6  }
```

3. 订单确认页面接收订单数据

```
1  // pages/order-confirm/order-confirm.js
2
3  Page({
4
5    data: {
6      orderData: {}, // 订单数据
7    },
8
9    onLoad(options) {
10      //console.log('orderData', options.orderData)
11      if (options.orderData) {
12        const orderData = JSON.parse(options.orderData)
13        this.setData({ orderData })
14      }
15    },
16
17  })
```

第九章 确认订单页面开发

上面已经将确认订单页面配置好了，页面要实现的功能如下图：



配置导航栏和注册组件

1. 配置自定义导航栏和注册组件：pages/order-confirm/order-confirm.json

```
1 {
2   "navigationStyle": "custom",
3   "usingComponents": {
4     "nav-bar": "/components/nav-bar/index",
5     "van-cell": "@vant/weapp/cell/index",
6     "van-action-sheet": "@vant/weapp/action-sheet/index"
7   }
8 }
```

确认订单页面效果实现

1. 确认订单页面模板代码

```
1 <!--pages/order-confirm/order-confirm.wxml-->
```

```
2
3 <view class="none-scorll">
4   <!-- 自定义导航栏 -->
5   <navbar title="订单确认" fixed="{{ true }}" show-search="{{ false }}" />
6
7   <view class="order-confirm-container">
8     <!-- 自取 -->
9     <view wx:if="{{orderData.takeType == 1}}" class="shop-info-wrapper bg-
radius">
10       <view class="address-box flex-space-between-center">
11         <view class="flex-row-align-center">
12           <text class="address-name text-one">{{orderData.shop.name}}</text>
13           <text class="iconfont icon-arrow-right"></text>
14         </view>
15         <view class="address-distance">
16           <text>直线距离{{orderData.shop.distanceStr || '未知'}}</text>
17         </view>
18       </view>
19       <view class="type-box flex-space-between">
20         <view class="type-item flex-1 flex-row-align-center"
21           {{selfTakeType == 1 ? 'blink-animation is-checked': ''}}
22           bind:tap="changeSelfTakeType" data-value="{{1}}"
23         >
24           <image class="img" src="/assets/images/tangshi.png"></image>
25           <view class="type-content flex-column">
26             <text>堂食</text>
27             <text>店内就餐</text>
28           </view>
29           <view class="type-checked"><text class="iconfont icon-right"></text>
30         </view>
31         <view class="type-item flex-1 flex-row-align-center"
32           {{selfTakeType == 2 ? 'blink-animation is-checked': ''}}
33           bind:tap="changeSelfTakeType" data-value="{{2}}"
34         >
35           <image class="img" src="/assets/images/waidai.png"></image>
36           <view class="type-content flex-column">
37             <text>外带</text>
38             <text>打包自提</text>
39           </view>
40           <view class="type-checked"><text class="iconfont icon-right"></text>
41         </view>
42       </view>
43     </view>
44
45     <!-- 外带 -->
46     <view wx:else class="shop-info-wrapper bg-radius">
47       <view class="address-box">
```

```
48     <view class="flex-row-align-center text-one"
bind:tap="toSelectAddress">
49     <text class="address-name">{{ orderData.address ?
orderData.address.locationName + ', ' + orderData.address.streetNumber : '请选
择收货地址' }}</text>
50     <text class="iconfont icon-arrow-right"></text>
51 </view>
52 <view class="address-distance" hidden="{{!orderData.address}}">
53     <!-- <text>姓名 199999999</text> -->
54     <text>{{orderData.address.name}} {{orderData.address.mobile}}</text>
55 </view>
56 </view>
57 <view class="take-box flex-row">
58     <text class="take-shop flex-1 text-one">{{orderData.shop.name}}</text>
59     <text class="take-rider">订单由第三方骑手配送</text>
60 </view>
61 </view>
62
63 <!-- 商品详情 -->
64 <view class="order-goods-wrapper bg-radius">
65     <view class="order-title">
66         <text>商品详情</text>
67     </view>
68     <view class="goods-item flex-row" wx:for="{{orderData.cartList}}"
wx:key="index">
69         <view class="goods-item-img-view">
70             <image class="goods-item-img" src="{{item.imageUrl}}" />
71         </view>
72         <view class="goods-item-content flex-1">
73             <text class="item-name">{{item.goodsName}}</text>
74             <text class="item-unit">{{item.specsName || item.unit}}</text>
75             <text class="item-num">x{{item.buyNum}}</text>
76         </view>
77         <view class="goods-item-price">
78             <view class="sale-price flex-center">
79                 <text class="sale-price-symbol">¥</text>
80                 <text class="sale-price-value">{{item.salePrice}}</text>
81             </view>
82             <text class="original-price" wx:if="{{item.discountPrice}}">¥
{{item.price}}</text>
83         </view>
84     </view>
85 </view>
86
87 <!-- 其他输入框 -->
88 <view class="other-wrapper">
89     <van-cell title="备注" is-link value="{{remark || '口味、偏好等要求'}}"
bind:tap="openRemark"/>
90     <van-cell title="开票方式" value="请在取单时洽服务员开票" />
```

```
91     </view>
92
93     <view class="submit-wrapper">
94         <view class="submit-box">
95             <view class="submit-text flex-1">
96                 <view class="submit-text-total">
97                     <text class="total-title">合计:</text>
98                     <text class="total-price">¥{{orderData.totalAmount}}</text>
99                 </view>
100                <view class="num-info">
101                    <text>数量: {{orderData.totalNum}}</text>
102                </view>
103            </view>
104            <button type="primary" bind:tap="submitOrder">
105                提交订单
106            </button>
107        </view>
108    </view>
109 </view>
110 </view>
111
112 <!-- 备注 -->
113 <van-action-sheet show="{{ showRemark }}" title="备注信息"
114 bind:close="closeRemark">
115     <form bindsubmit="confirmRemark">
116         <view class="remark-popup-wrapper">
117             <view class="textarea-view">
118                 <!-- 解决输入框被键盘遮挡, 使用`cursor-spacing`设置光标与键盘的距离 -->
119                 <textarea value="{{remark}}" name="remark" class="textarea" cursor-
120 spacing="150" placeholder="我们会视情况尽力响应, 若无法完全满足也请您谅解" />
121             </view>
122             <view>
123                 <button form-type="submit" type="primary">确定</button>
124             </view>
125         </form>
126     </van-action-sheet>
```

2. 确认订单样式代码

```
1  /* pages/order-confirm/order-confirm.wxss */
2
3  .order-confirm-container {
4      padding: 26rpx;
5      padding-bottom: 300rpx; /*空出底部按钮区域*/
6      .bg-radius {
```

```
7     background-color: #fff;
8     border-radius: 18rpx;
9     padding: 26rpx;
10    margin-bottom: 20rpx;
11  }
12 }
13
14 .shop-info-wrapper {
15   .address-box {
16     margin-bottom: 20rpx;
17     .address-name {
18       font-size: 38rpx;
19       color: #000;
20       font-weight: bold;
21       max-width: 400rpx;
22     }
23     .iconfont {
24       margin-left: 2rpx;
25       color: #000;
26       font-size: 30rpx;
27       font-weight: bold;
28     }
29     .address-distance {
30       font-size: 28rpx;
31       color: #222;
32     }
33   }
34   .type-box {
35     .type-item {
36       border: 4rpx solid #c3c3c3;
37       border-radius: 10rpx;
38       padding: 18rpx 20rpx;
39       overflow: hidden;
40       position: relative;
41       &:nth-child(2) {
42         margin-left: 16rpx;
43       }
44       .img {
45         width: 70rpx;
46         height: 70rpx;
47         border-radius: 10rpx;
48       }
49       .type-content {
50         margin-left: 20rpx;
51         font-size: 26rpx;
52         font-weight: bold;
53         text {
54           &:last-child {
55             font-size: 20rpx;
```

```
56         color: #898989;
57         font-weight: normal;
58     }
59 }
60 }
61 &.is-checked {
62     border: 4rpx solid var(--primary-color);
63     .type-checked {
64         background-color: var(--primary-color);
65     }
66 }
67 .type-checked {
68     position: absolute;
69     top: 0;
70     right: 0;
71     width: 30rpx;
72     height: 30rpx;
73     line-height: 26rpx;
74     background-color: inherit;
75     text-align: center;
76     .iconfont {
77         color: #fff;
78         font-weight: bold;
79     }
80 }
81 }
82 }
83
84 // 外送
85 .take-box {
86     border-top: var(--basic-border);
87     padding-top: 20rpx;
88     font-size: 28rpx;
89     .take-shop {
90         color: #999;
91         margin-right: 10rpx;
92     }
93     .take-rider {
94         color: var(--primary-color);
95     }
96 }
97 }
98
99 .order-goods-wrapper {
100     .order-title {
101         font-size: 36rpx;
102         font-weight: bold;
103         border-bottom: var(--basic-border);
104         padding-bottom: 20rpx;
```

```
105     }
106
107     .goods-item {
108         padding: 20rpx 0;
109         .goods-item-img-view {
110             .goods-item-img {
111                 width: 100rpx;
112                 height: 80rpx;
113             }
114         }
115         .goods-item-content {
116             margin: 0 16rpx;
117             &> text {
118                 display: block;
119             }
120             .item-name {
121                 font-size: 28rpx;
122                 color: #222222;
123                 font-weight: 500;
124             }
125             .item-unit {
126                 font-size: 20rpx;
127                 color: #999;
128             }
129             .item-num {
130                 font-size: 26rpx;
131             }
132         }
133         .goods-item-price {
134             margin-left: auto; // 整体右对齐
135             text-align: right;
136             .sale-price {
137                 color: #222;
138                 align-items: flex-end;
139                 .sale-price-symbol {
140                     font-size: 20rpx;
141                     line-height: 24rpx;
142                 }
143                 .sale-price-value {
144                     font-weight: 500;
145                     font-size: 30rpx;
146                     line-height: 28rpx;
147                 }
148             }
149             .original-price {
150                 color: #828282;
151                 text-decoration: line-through;
152                 font-size: 22rpx;
153             }
154         }
155     }
```



```
154     }
155   }
156 }
157
158 .other-wrapper {
159   border-radius: 18rpx;
160   overflow: hidden;
161 }
162
163 .submit-wrapper {
164   background-color: #fff;
165   position: fixed;
166   bottom: 0;
167   left: 0;
168   width: 100%;
169   z-index: 100;
170   box-shadow: 0 0 .3rem #e8e8e8;
171   .submit-box {
172     display: flex;
173     align-items: center;
174     justify-content: flex-end;
175     background-color: #fff;
176     font-size: 28rpx;
177     padding: 30rpx;
178     padding-bottom: env(safe-area-inset-bottom);
179   }
180   .submit-text {
181     text-align: right;
182     margin-right: 20rpx;
183     .submit-text-total {
184       .total-title {
185         font-size: 22rpx;
186         font-weight: 500;
187       }
188       .total-price {
189         font-size: 40rpx;
190         font-weight: bold;
191       }
192     }
193     .num-info{
194       color: red;
195       font-size: 22rpx;
196       margin-bottom: 10rpx;
197     }
198   }
199 }
200
201
202 /* 应用动画到想要闪烁的元素 */
```

```
203 .blink-animation {
204   animation: scaleAnimation 1s; /* 闪烁1秒 */
205 }
206
207 /* 添加动画名称和关键帧 */
208 @keyframes scaleAnimation {
209   0%, 100% {
210     transform: scale(1); /* 初始状态和结束状态，元素的大小是1 */
211   }
212   25% {
213     transform: scale(0.9); /* 第一次缩小 */
214   }
215   50% {
216     transform: scale(1); /* 第二次放大 */
217   }
218   75% {
219     transform: scale(0.9); /* 第三次缩小，准备下一次放大 */
220   }
221 }
222
223 .remark-popup-wrapper {
224   margin: 0 30rpx;
225   .textarea-view {
226     padding: 20rpx;
227     background-color: #f9f9f9;
228     border-radius: 10rpx;
229     margin-bottom: 100rpx;
230   }
231   .textarea {
232     width: 100%;
233   }
234 }
```

3. 页面逻辑实现

```
1 // pages/order-confirm/order-confirm.js
2
3 import { navTo } from "@utils/util"
4
5 Page({
6
7   data: {
8     orderData: {}, // 订单数据
9     selfTakeType: 1, // 自取类型: 1-堂食, 2-外带
10    remark: '', // 备注信息
11    showRemark: false, // 显示备注输入框
12  },
13
14  onLoad(options) {
```

```
15 //console.log('orderData', options.orderData)
16 if (options.orderData) {
17   const orderData = JSON.parse(options.orderData)
18   this.setData({ orderData })
19 }
20 },
21
22 // 切换自取类型
23 changeSelfTakeType(e) {
24   const { value } = e.currentTarget.dataset
25   this.setData({ selfTakeType: value })
26 },
27
28 // 前往选择收货地址页面
29 toSelectAddress(e) {
30   navTo('/pages/address-list/address-list', {
31     events: { // events 添加一个监听事件，用于获取被打开页面传送到当前页面的数据
32       updateAddress: (address) => { // address-list.js 中触发了此监听事件
33         // console.log('updateAddress', address)
34         this.setData({'orderData.address': address})
35       }
36     }
37   })
38 },
39
40 // 弹出备注输入框
41 openRemark() {
42   this.setData({showRemark: true})
43 },
44
45 // 关闭备注输入框
46 closeRemark() {
47   this.setData({showRemark: false})
48 },
49
50 // 确认输入的备注信息
51 confirmRemark(e) {
52   // 获取输入的备注
53   const { remark } = e.detail.value
54   this.setData({ remark })
55   this.closeRemark()
56 },
57
58 // 提交订单
59 submitOrder() {
60 }
61
62 })
```

提交订单

1. 封装新增订单数据API方法，创建api文件：miniprogram/api/order.js

新增订单数据调用前面已创建的 `add` 云函数，所以我们只需要调用云函数新增订单的API方法即可。

```
1 // miniprogram/api/order.js
2
3 /**
4  * 新增订单信息
5  * @param {Object} data 新增的订单数据
6  */
7 export function addOrder(data) {
8   return wx.cloud.callFunction({
9     name: 'add',
10    data: {
11      collectionName: 'mxg_order',
12      data, // data: data
13    }
14  })
15 }
```

2. 页面提交订单逻辑实现：调用新增订单接口，重定向到支付页面

```
1 // pages/order-confirm/order-confirm.js
2
3 import { navTo, showLoading, hideLoading, redirectTo, msg } from "@utils/util"
4
5 import { addOrder } from '@api/order'
6
7 Page({
8
9   // 提交订单
10  async submitOrder() {
11    const { address, takeType } = this.data.orderData
12    // 判断如果是`外送`，必须有收货地址
13    if (takeType == 2 && (!address || !address._id)) {
14      return msg('请选择收货地址')
15    }
16
17    showLoading('提交中', {mask: true})
18
19    try {
20      const { orderData, selfTakeType, remark } = this.data
21      // 店铺相关数据
22      const { _id: shopId, name: shopName } = orderData.shop
```

```
23
24 // 封装要保存的数据，state状态：1-待支付，2-已支付，3-已完成，4-已取消
25 const data = {...orderData, state: 1, shopId, shopName, selfTakeType,
remark}
26
27 // 调用接口新增订单数据
28 const { result } = await addOrder(data)
29 console.log('提交订单结果', result)
30
31 // 跳转到支付页面
32 redirectTo('/pages/payment/payment?orderId=' + result._id, {
33   complete: () => {
34     // 跳转完成后再隐藏
35     hideLoading()
36     // 触发来源页面（购物页）清空购物车事件
37     const eventChannel = this.getOpenerEventChannel()
38     eventChannel.emit('clearCartData')
39   }
40 })
41 } catch (error) {
42   console.error('error', error)
43   msg('提交订单失败', {icon: 'none'})
44   hideLoading()
45 }
46 },
47
48 })
```

3. app.json 配置创建支付收银台页面

```
1 {
2   "pages": [
3
4     "pages/payment/payment"
5   ],
6 }
```

4. 支付收银台页面获取订单id

```
1 // pages/payment/payment.js
2
3 Page({
4
5   data: {
6     orderId: '', // 订单主键id
7   },
8 })
```

```
9   onLoad(options) {  
10     const {orderId} = options  
11     if (!orderId) return  
12     this.setData({ orderId })  
13   },  
14  
15 })
```

第十章 支付收银台页面开发

订单支付收银台页面：可以支付微信支付和余额支付，余额支付：当会员已登录，且有足够的余额才会展示



payment.json 收银台页面配置导航标题

```
1 {  
2   "navigationBarTitleText": "收银台",  
3   "usingComponents": {}  
4 }
```

查询订单和会员数据

1. 查询订单数据


```
1 // pages/payment/payment.js
2 import { hideLoading, showLoading, msg, redirectTo } from '@utils/util'
3
4 Page({
5
6   data: {
7     orderId: '', // 订单主键id
8     order: null, // 订单数据
9     showBalancePay: false, // 是否显示余额支付，已登录且有余额时显示
10    payType: 'wxpay', // 支付类型: wxpay、balance
11  },
12
13  onLoad(options) {
14    const {orderId} = options
15    if (!orderId) return
16    this.setData({ orderId })
17    // 查询订单
18    this.loadOrder()
19  },
20
21  /** 查询订单明细 */
22  async loadOrder() {
23    try {
24      showLoading()
25
26      const {orderId} = this.data
27      // 查询当前订单id为未支付的订单信息
28      const { data } = await wx.cloud.database().collection('mxg_order')
29        .doc(orderId).get()
30
31      if (!data || !data._id) {
32        return msg("订单不存在")
33      }
34
35      // 判断状态state=1未支付状态，才可以发起支付
36      if (data.state !== 1) {
37        redirectTo('/pages/order-detail/order-detail?orderId=' + orderId)
38        return msg("当前订单不是[未支付]状态，不允许进行支付")
39      }
40      this.setData({order: data})
41      // 处理是否显示余额支付
42      this.isShowBalancePay()
43    } catch (error) {
44      console.error("查询订单信息失败", error)
45      msg('查询订单失败', {icon: 'error'})
46    } finally {
47      hideLoading()
48    }
49  }
```

```
50
51  /** 处理是否显示余额支付，因为不知道会员和订单数据哪个先查询完，所以在分别查询完都调用此方法 */
52  isShowBalancePay() {
53      const {member, order} = this.data
54      if (!member || !order || !order._id) return
55      // 判断余额是否充足，充足则显示余额支付
56      const showBalancePay = member.balance >= order.totalAmount
57      // 显示余额支付时，则默认选中：余额支付
58      const payType = showBalancePay ? 'balance' : 'wxpay'
59      this.setData({showBalancePay, payType})
60  },
61
62  })
```

2. 查询会员数据，获取会员剩余余额

```
1  // pages/payment/payment.js
2  import { getMemberById } from '@api/member'
3  import { hideLoading, showLoading } from '@utils/util'
4  Page({
5
6      data: {
7          orderId: '', // 订单主键id
8          order: null, // 订单数据
9          member: null, // 会员数据
10         showBalancePay: false, // 是否显示余额支付，已登录且有余额时显示
11         payType: 'wxpay', // 支付类型： wxpay、balance
12     },
13
14     onLoad(options) {
15         const {orderId} = options
16         if (!orderId) return
17         this.setData({ orderId })
18         // 查询订单
19         this.loadOrder()
20         // 查询会员 ++++++++
21         this.loadMember()
22     },
23
24     /** 查询当前登录会员 */
25     async loadMember() {
26         try {
27             const {member, logged} = getApp().current()
28             if (!logged) {
29                 // 未登录，隐藏余额支付
30                 this.setData({showBalancePay: false})
31             }
32         } catch {
33             // 处理异常
34         }
35     }
36 })
```

```
31     return
32   }
33
34   // 查询会员最新信息
35   const { data } = await getMemberById(member._id)
36   this.setData({member: data})
37
38   // 处理是否显示余额支付
39   this.isShowBalancePay()
40 } catch (error) {
41   console.log('查询会员失败', error)
42 } finally {
43   hideLoading()
44 }
45 },
46
47 })
```

收银台页面效果实现

1. 收银台页面模板代码

```
1  <!--pages/payment/payment.wxml-->
2
3  <view class="pay-amount-box flex-column-center">
4    <view >
5      <text class="amount-symbol">¥</text>
6      <text class="amount-value">{{order.totalAmount || "0.00"}}</text>
7    </view>
8  </view>
9
10 <!-- 如果用 model:value="payType" 真机上无法获取值，可以使用绑定事件bind:change获取选择的值 -->
11 <radio-group class="pay-type-group" model:value="{{ payType }}">
12   <view class="pay-type-item flex-space-between-center" hover-class="view-hover" hover-stay-time="100">
13     <label for="wxpay" class="pay-type-item-label flex-row-align-center">
14       <text class="iconfont icon-wxpay"></text>
15       <text class="pay-name">微信支付</text>
16     </label>
17     <radio id="wxpay" value="wxpay" color="red" checked="{{payType == 'wxpay'}}"></radio>
18   </view>
19
```

```
20 <view wx:if="{{showBalancePay}}" class="pay-type-item flex-space-between-
    center"
21     hover-class="view-hover" hover-stay-time="100">
22     <label for="balance" class="pay-type-item-label flex-row-align-center">
23         <text class="iconfont icon-vipcard"></text>
24         <text class="pay-name">余额支付</text><text class="balance-value"> (可用余
    额: ¥ {{member.balance || '0.00'}}) </text>
25     </label>
26     <radio id="balance" value="balance" color="red" checked="{{payType ==
    'balance'}}"></radio>
27 </view>
28 </radio-group>
29
30 <view class="fixed-bottom">
31     <button type="primary" bind:tap="confirmPay" disabled="{{!payType}}">
32         确认支付 ¥ {{order.totalAmount || '0.00'}}
33     </button>
34 </view>
```

2. 收银台页面样式代码

```
1  /* pages/payment/payment.wxss */
2
3  .pay-amount-box {
4      height: 200rpx;
5      justify-content: center;
6      .amount-symbol {
7          font-size: 36rpx;
8      }
9      .amount-value {
10         font-weight: bold;
11         font-size: 80rpx;
12     }
13 }
14
15
16 .pay-type-group {
17     --type-height: 120rpx;
18     .pay-type-item {
19         height: var(--type-height);
20         margin: 26rpx;
21         padding: 0 26rpx;
22         background-color: #fff;
23         border-radius: 16rpx;
24         color: #222;
25         .pay-type-item-label {
26             flex: 1;
```

```
27     line-height: var(--type-height);
28     font-size: 32rpx;
29     .pay-name {
30         white-space: nowrap;
31     }
32     .balance-value {
33         font-size: 28rpx;
34         color: #999;
35         line-height: 32rpx;
36         height: 30rpx;
37     }
38 }
39 .iconfont {
40     font-size: 68rpx;
41     margin-right: 20rpx;
42     &.icon-wxpay {
43         color: #07c160;
44     }
45     &.icon-vipcard {
46         color: #ff9a16;
47     }
48 }
49 }
50 }
```

3. 点击确认支付简单逻辑实现

```
1  // pages/payment/payment.js
2  import { hideLoading, showLoading, msg } from '@utils/util'
3
4  Page({
5
6      // 确认支付
7      confirmPay() {
8          const { payType, order } = this.data
9          if (!order || !order._id) {
10             return msg('未查询到订单信息')
11         }
12
13         // 提交支付
14         if (payType === 'wxpay') {
15             this.handleWxpay()
16         } else if (payType === 'balance') {
17             this.handleBalancePay()
18         } else {
19             msg('请选择支付方式')
20         }
21     },
22 })
```

```
23 // 微信支付
24 async handleWxpay() {
25
26 },
27
28 // 会员余额支付
29 async handleBalancePay() {
30
31 },
32 })
```

余额支付实现

余额支付要完成的操作：扣减会员余额、更新商品库存、更新订单状态：支付成功，我们通过云函数来实现

1. 创建订单余额支付云函数 `orderBalancePay`，实现：扣减会员余额、更新商品库存、更新订单状态：支付成功

```
1 // 云函数入口文件 cloudfunctions/orderBalancePay/index.js
2 const cloud = require('wx-server-sdk')
3
4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6 const db = cloud.database()
7 const _ = db.command
8
9 // 余额支付：扣减会员余额、更新库存、更新订单状态：支付成功
10 exports.main = async (event, context) => {
11   const { memberId, totalAmount, orderId } = event
12
13   // 1. 扣减会员余额，不要少了await
14   const updated = await db.collection('mxg_member')
15     .where({
16       _id: memberId,
17       // 可用余额必须大于等于支付总金额
18       balance: _.gte(totalAmount)
19     })
20     .update({
21       data: {
22         // inc是自增，所以-totalAmount变成自减，
23         // 注意：如果余额是1，减0.07，数据库更新后为0.9299999999999999，1-0.07精度有问题，可转成整数存储，将单位`元`转分`
24         balance: _.inc( -totalAmount )
25       }
26     })
27
28   // 更新失败， updated.stats.updated>0才是更新成功
```

```
29   if (!updated.stats.updated) {
30     return { success: false, error: '余额扣减失败，请核实余额是否充足' }
31   }
32
33   // 2. 扣减商品库存
34   const { result } = await cloud.callFunction({
35     name: 'updateGoodsStock',
36     data: {
37       orderId
38     }
39   })
40
41   // 扣减商品库存失败
42   if (!result.success) return result
43
44   // 3. 更新订单状态：支付成功
45   await db.collection("mxg_order")
46     .doc(orderId)
47     .update({
48       data: {
49         payType: 'balance', // 余额支付
50         state: 2, // 1-待支付, 2-已支付, 3-已完成, 4-已取消
51         payEndTime: Date.now()
52       }
53     })
54
55   // 返回成功
56   return {
57     success: true,
58     message: '余额支付成功'
59   }
60
61 }
```

2. 创建更新商品库存云函数 `updateGoodsStock`，使用云开发事务控制更新云函数

事务控制：<https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/database/transaction.html>

```
1 // 云函数入口文件
2 const cloud = require('wx-server-sdk')
3
4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6 const db = cloud.database({
7   throwOnNotFound: false, // 指定 false 后可使得 db.doc().get() 在找不到记录时不抛出异常
8 })
```



```
9
10 const _ = db.command
11
12 // 支付成功，更新订单状态和商品库存 云函数入口函数
13 exports.main = async (event, context) => {
14   const { orderId } = event
15   try {
16     // 1、查询订单
17     const { data: order } = await db.collection('mxg_order').doc(orderId).get()
18     if (!order) {
19       // 无订单直接返回成功
20       return { success: true, message: '未查询到订单'+orderId }
21     }
22
23     // 订单商品
24     const { cartList } = order
25     if (!cartList || cartList.length <= 0) {
26       // 无订单商品直接返回成功
27       return { success: true, message: '未查询到订单商品'+orderId }
28     }
29
30     const goodsList = [] // 待更新的商品库存
31     const specsList = [] // 待更新的规格库存
32
33     // 2、循环购物车数据，分别提取待更新的商品、规格
34     cartList.forEach(item => {
35       const { _id: goodsId, specsId, buyNum } = item
36       // specsId有值：多规格商品，更新规格库存
37       if (specsId) specsList.push({ goodsId, specsId, buyNum })
38       // specsId无值：单规格商品，更新商品库存
39       else goodsList.push({ goodsId, buyNum })
40     })
41
42     // 更新事务控制：
43     const result = await db.runTransaction(async transaction => {
44       // 批量更新商品库存数量
45       for (const item of goodsList) {
46         // 不要少了await
47         await transaction.collection('mxg_goods')
48           .doc(item.goodsId)
49           .update({
50             data: {
51               stockNum: _.inc( -item.buyNum ) // inc是自增，所以取负-buyNum变成自减
52             }
53           })
54       }
55     })
```

```
56 // 批量更新规格库存数量(参考:  
https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/database/query-  
array-object.html)  
57 for (const item of specsList) {  
58 // 不要少了await  
59 await transaction.collection('mxg_goods')  
60 .where({  
61   _id: item.goodsId,  
62   'goodsSpecs._id': item.specsId  
63 })  
64 .update({  
65   data: {  
66     // goodsSpecs值是对象数组, 后面跟上$, 再定位到数组中的对象的哪个属性名,  
67     // `$$`是更新第一个匹配到满足条件的, 如果是`$[]`是更新goodsSpecs数组中所有  
元素的stockNum值  
68     'goodsSpecs.$.stockNum': _.inc( -item.buyNum ) // inc是自增, 所以取  
负-buyNum变成自减  
69   }  
70 })  
71 }  
72 // 会作为 runTransaction resolve 的结果返回  
73 return { message: '扣减商品库存成功' }  
74 })  
75  
76 // 返回给调用方  
77 return {  
78   success: true,  
79   message: result.message  
80 }  
81 } catch(error) {  
82   console.error(`transaction error`, error)  
83   return { success: false, error: '扣减商品库存失败' }  
84 }  
85  
86 }
```

3. 支付收银台页面调用订单余额支付云函数

```
1 // 会员余额支付  
2 async handleBalancePay() {  
3   try {  
4     showLoading('支付中', {mask: true})  
5  
6     const {member, order} = this.data  
7  
8     // 更新会员余额  
9     const { result } = await wx.cloud.callFunction({  
10       name: 'orderBalancePay',  
11       data: {
```

```
12         memberId: member._id,  
13         totalAmount: order.totalAmount,  
14         orderId: order._id  
15     }  
16 })  
17 // console.log('余额支付结果', result)  
18  
19 if (!result.success) {  
20     return msg(result.error)  
21 }  
22 msg('支付成功', {icon: 'success'})  
23 // 支付成功: 跳转订单详情页  
24 this.toOrderDetailPage()  
25 } catch (error) {  
26     console.error('余额支付失败', error)  
27     msg('支付失败', {icon: 'error'})  
28     this.toOrderDetailPage() // 跳转订单详情页  
29 } finally {  
30     setTimeout(() => {  
31         hideLoading()  
32     }, 500);  
33 }  
34 },  
35  
36 // 跳转到订单详情页  
37 toOrderDetailPage() {  
38     redirectTo(`/pages/order-detail/order-detail?orderId=${this.data.orderId}`)  
39 },
```

微信支付实现

1. 调用封装的微信支付方法 wxpay 进行支付

- 导入wxpay微信支付方法: `import { wxpay } from '@api/pay'`
- 更新订单编号orderNo, 解决报错: 201 商户订单号重复,
 - 原因: 如果第1次发起支付, 用户没有支付, 下次订单重新进行支付时, 就会报: 201 商户订单号重复
 - 解决: 每次支付的时候, 都生成一个新的orderId传递给outTradeNo进行提交支付, 并更新订单表中的orderId `const orderNo = _id.substring(0, 10) + Date.now()`
- 调用微信支付方法 wxpay 传递相关参数

```
1 // pages/payment/payment.js  
2 import { getMemberById } from '@api/member'  
3 import { hideLoading, redirectTo, showLoading } from '@utils/util'  
4 import { wxpay } from '@api/pay'
```

```
5
6 Page({
7
8   // 微信支付
9   async handleWxpay() {
10     try {
11       showLoading('发起支付中', {mask: true})
12       const {_id, shopName, totalAmount} = this.data.order
13       /*
14        报错: 201 商户订单号重复,
15        原因: 如果第1次发起支付, 用户没有支付, 下次订单重新进行支付时, 就会报: 201 商户订单号
16        重复
17        解决: 每次支付的时候, 都生成一个新的订单Id传递给outTradeNo进行提交支付, 并更新订单表
18        中的订单Id
19        cosnt orderNo = _id.substring(0, 10) + Date.now()
20        */
21       const orderNo = _id.substring(0, 8) + Date.now()
22       // 1、更新订单编号
23       await wx.cloud.database().collection('mxg_order')
24         .doc(_id)
25         .update({
26           data: {
27             orderNo: orderNo,
28           }
29         })
30
31       // 2、调用微信支付, 支付成功回调云函数: wxpayCallOrder
32       const payRes = await wxpay(shopName || '结账支付', orderNo, totalAmount,
33         'orderWxpayCall')
34       console.log('支付成功', payRes)
35
36       msg('支付成功', {icon: 'success'})
37       // 支付成功: 跳转订单详情页
38       this.toOrderDetailPage()
39     } catch (error) {
40       console.error('支付失败', error)
41       msg('支付失败', {icon: 'error'})
42       this.toOrderDetailPage() // 跳转订单详情页
43     } finally {
44       setTimeout(() => {
45         hideLoading()
46       }, 500);
47     }
48   },
49 })
```

2. 创建订单微信支付成功回调云函数 `orderWxpayCall`

```
1 // 云函数入口文件
2 const cloud = require('wx-server-sdk')
3
4 cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV }) // 使用当前云环境
5
6 const db = cloud.database()
7
8 // 返回 errcode: 0, 表示支付通知已处理成功, 就不会再通知了; 如果返回其他值, 还会重复通知
9 const SUCCESS = { errcode: 0, errmsg: 'SUCCESS' }
10 const FAIL = { errcode: -1, errmsg: 'FAIL' }
11
12 // 云函数入口函数: 接收云支付平台的支付结果回调通知
13 exports.main = async (event, context) => {
14   // 业务订单编号, 支付结果编码, 微信支付订单号, 支付完成时间
15   const {outTradeNo, resultCode, transactionId, timeEnd} = event
16
17   // 支付失败
18   if (resultCode !== 'SUCCESS') {
19     return FAIL
20   }
21
22   // 通过业务订单id查询商品
23   const { data } = await db.collection('mxg_order')
24     .where({
25       orderNo: outTradeNo
26     })
27     .get()
28
29   if (!data || data.length <= 0) {
30     // 无商品, 直接返回成功
31     return SUCCESS
32   }
33
34   // 解构出订单id
35   const {_id: orderId} = data[0]
36
37   // 更新订单状态: 支付成功
38   const updated = await db.collection("mxg_order")
39     .doc(orderId)
40     .update({
41       data: {
42         payType: 'wxpay', // 微信支付
43         state: 2, // 1-待支付, 2-已支付, 3-已完成, 4-已取消
44         payOrderId: transactionId,
45         payEndTime: timeEnd
46       }
47     })
48 }
```

```
48
49 // 扣减商品库存
50 const { result } = await cloud.callFunction({
51   name: 'updateGoodsStock',
52   data: {
53     orderId
54   }
55 })
56
57
58 // 成功
59 if (result.success) return SUCCESS
60
61 // 失败
62 return FAIL
63
64 }
65
66 /**
67 支付结果回调数据
68  event: {
69   "appid": "wxd2d16a504f24665e", // 服务商的appid
70   "mchId": "1800008281", // 服务商的商户号
71   "bankType": "OTHERS", // 银行类型
72   "cashFee": 1, // 现金支付金额
73   "feeType": "CNY", // 货币类型
74   "nonceStr": "5ebf24442c30611e", // 随机字符串，不长于32位
75   "openid": "oPoo4456B6UvaIDDG_gtMucyWw3I", // 用户在服务商appid下的唯一标识
76   "outTradeNo": "1724153334725", // 商户系统业务订单号
77   "resultCode": "SUCCESS", // 判断交易是否成功 SUCCESS/FAIL
78   "returnCode": "SUCCESS", // SUCCESS/FAIL此字段是通信标识，非交易标识，交易是否
    成功通过resultCode来判断
79   "subAppid": "wx447a088bff5a5aca", // 子商户的小程序ID
80   "subMchId": "1608355689", // 子商户的商户号
81   "subOpenid": "oYkTM1QWJVH7GcPta_vf1vMWQsA2", // 用户在子商户下的唯一标识
82   "timeEnd": "20260820192859", // 支付完成时间
83   "totalFee": 1, // 订单总金额，单位为分
84   "tradeType": "JSAPI", // 交易类型
85   "transactionId": "4200002335202408208210907502", // 微信支付订单号
86   "isSubscribe": "N", // 已废弃，默认统一返回N
87   "subIsSubscribe": "N", // 已废弃，默认统一返回N
88   }
89  */
```

<page-container> 组件处理页面后退事件

需求：我们希望监听在支付页面的所有后退事件（如左上角<，IOS趁机向右滑动手势等），当后退时我们实现跳转到订单详情页面，而不是后退到上页。

实现：我们可能通过 `<page-container>` 容器组件来监听页面后退事件，利用 `<page-container/>` 容器组件，当用户进行返回操作时（可监听到 滑动手势、导航返回键，安卓物理返回键和调用 `navigateBack` 后退），关闭该 `<page-container/>` 容器而不关闭页面，触发 `bindbeforeleave` 事件，实现：后退操作跳转到订单详情页。

`<page-container>` 组件 <https://developers.weixin.qq.com/miniprogram/dev/component/page-container.html>

在 `payment.wxml` 最后一行添加以下代码：

```
1 <!--pages/payment/payment.wxml-->
2
3 <!-- 利用 <page-container/> 组件，当用户进行返回操作，关闭该容器不关闭页面，触发
   bindbeforeleave事件，实现后退操作跳转到订单详情页（可监听到 滑动手势、导航返回键，安卓物理返回键和
   调用 navigateBack 后退） -->
4 <page-container bind:beforeleave="toOrderDetailPage" show="{{true}}" overlay="
   {{false}}" />
```

第十一章 订单详情页面

左侧是未支付、右侧是已支付的订单详情图



订单详情页面配置

1. 在 app.json 添加页面配置，创建页面文件

```
1 {  
2   "pages": [  
3     "pages/order-detail/order-detail"  
4   ],  
5 }
```

2. 在 order-detail.json 页面配置中添加导航栏标题

```
1 {
2   "navigationBarTitleText": "订单详情",
3   "navigationBarTextStyle": "white",
4   "navigationBarBackgroundColor": "#e40030",
5   "backgroundColor": "#e40030",
6   "backgroundColorBottom": "#f9f9f9",
7   "enablePullDownRefresh": true,
8   "usingComponents": {}
9 }
```

查询订单详情数据

1. 在小程序端直接调用调用云函数api查询，因为只允许查自己的订单

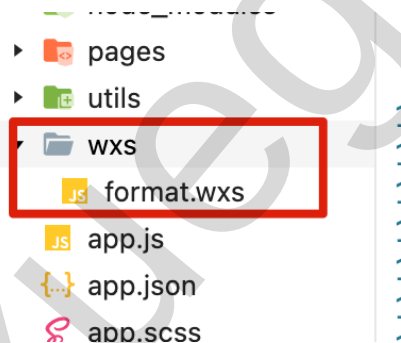
```
1 // pages/order-detail/order-detail.js
2
3 import { hideLoading, msg, navBack, navTo, showLoading } from "@utils/util"
4
5 Page({
6
7   /**
8    * 页面的初始数据
9    */
10  data: {
11    orderId: '',
12    order: {},
13  },
14  onLoad(options) {
15    const {orderId} = options
16    this.setData({ orderId })
17    this.loadOrder()
18  },
19
20  // 下拉刷新
21  onPullDownRefresh() {
22    this.loadOrder()
23  },
24
25  // 查询订单详情
26  async loadOrder() {
27    try {
28      showLoading('')
29      const { orderId } = this.data
30      const { data } = await wx.cloud.database().collection('mxg_order')
31        .doc(orderId).get()
32      this.setData({order: data})
33    } catch (e) {
34      msg(e.message)
35    }
36  }
37 })
```

```
33     } catch (e) {  
34         console.error('订单详情查询失败', e)  
35         msg('订单详情查询失败')  
36     } finally {  
37         // 停止下拉刷新  
38         wx.stopPullDownRefresh()  
39         hideLoading()  
40     }  
41 },  
42  
43  
44  
45 })
```

订单详情页面效果实现

1. 导入format.wxs 日期格式文件到项目中：

将 03-配套资料/wxs/format.wxs 复制到项目中，如下：



2. 订单详情页面模板代码：

```
1  <!--pages/order-detail/order-detail.wxml-->  
2  
3  <!-- 引入wxs模块 -->  
4  <wxs src="/wxs/format.wxs" module="format" />  
5  
6  <view class="none-scrollbar">  
7    <view class="order-detail-container">  
8      <view class="remaining-time-wrapper">  
9        <view class="bg-radius flex-space-between-center">  
10         <!-- 1-待支付, 2-已支付, 3-已完成, 4-已取消 -->  
11         <block wx:if="{{order.state == 1}}">  
12           <view class="remaining-time-text">  
13             <text>请尽快支付</text>  
14             <text>若超时未付款，订单将自动取消</text>  
15           </view>  
16         </block>
```

```
17     <view wx:else class="remaining-time-text">
18         <text>订单{{order.state == 1 ? '待支付' : order.state == 2 ? '已支付'
19 : order.state == 3 ? '已完成' : order.state == 4 ? '已取消': ''}}</text>
20         <text>期待您的再次光临! </text>
21     </view>
22 </view>
23
24 <view class="order-goods-wrapper bg-radius">
25     <view class="goods-info-header flex-space-between-center">
26         <view class="header-left flex-1 flex-row-align-center">
27             <text class="iconfont icon-coordinates_fill" />
28             <view class="shop-name text-one">{{order.shopName}}</view>
29         </view>
30         <view class="header-right flex-row">
31             <view class="right-item flex-column-center"
32 bind:tap="handleOpenLocation">
33                 <text class="iconfont icon-coordinates" />
34                 <text class="tag-text">门店指引</text>
35             </view>
36         </view>
37
38         <view class="goods-item flex-row" wx:for="{{order.cartList}}"
39 wx:key="id">
40             <view class="goods-item-img-view">
41                 <image class="goods-item-img" src="{{item.imageUrl}}" />
42             </view>
43             <view class="goods-item-content flex-1">
44                 <text class="item-name">{{item.goodsName}}</text>
45                 <text class="item-unit">{{item.specsName || item.unit}}</text>
46                 <text class="item-num">x{{item.buyNum}}</text>
47             </view>
48             <view class="goods-item-price">
49                 <view class="sale-price flex-center">
50                     <text class="sale-price-symbol">¥</text>
51                     <text class="sale-price-value">{{item.salePrice}}</text>
52                 </view>
53                 <text class="original-price" wx:if="{{item.discountPrice}}">¥
54 {{item.price}}</text>
55             </view>
56         </view>
57
58         <view class="goods-total-box">
59             <text>合计</text>
60             <text class="total-amount-value">¥{{order.totalAmount}}</text>
61         </view>
62     </view>
```

```
62 <!-- 1-待支付, 2-已支付, 3-已完成, 4-已取消 -->
63 <view class="order-info-wrapper bg-radius" wx:if="{{order.state != 1}}">
64   <view class="order-header">
65     <text>订单信息</text>
66   </view>
67   <view class="order-row-group">
68     <view class="row-cell flex-space-between-center">
69       <text class="row-title">下单时间</text>
70       <text class="row-value">{{format.formatTime(order._createTime)}}
</text>
71     </view>
72     <view class="row-cell flex-space-between-center">
73       <text class="row-title">订单号</text>
74       <text class="row-value">{{order.orderNo || order._id}}</text>
75     </view>
76     <view class="row-cell flex-space-between-center">
77       <text class="row-title">取单类型</text>
78       <text class="row-value">{{order.takeType == 1 ? (order.selfTakeType
== 1 ? '自取(堂食)': '自取(外带)') : '外送'}}</text>
79     </view>
80     <view wx:if="{{order.payType}}" class="row-cell flex-space-between-
center">
81       <text class="row-title">支付方式</text>
82       <text class="row-value">{{order.payType == 'wxpay' ? '微信支付' :
order.payType == 'balance' ? '余额支付': '未知' }}</text>
83     </view>
84     <view class="row-cell flex-space-between-center">
85       <text class="row-title">备注</text>
86       <text class="row-value">{{order.remark}}</text>
87     </view>
88   </view>
89 </view>
90
91 <view class="fixed-bottom">
92   <view class="order-detail-bottom-btns" >
93     <!--
94     客服参考:
95     https://developers.weixin.qq.com/miniprogram/dev/framework/open-
ability/customer-message/customer-message.html
96     云函数接收消息
97     https://developers.weixin.qq.com/miniprogram/dev/wxcloud/guide/functions/open
api.html -->
98     <button type="default" open-type="contact" size="mini">在线客服</button>
99     <block wx:if="{{order.state == 1}}">
100       <button type="default" size="mini" bind:tap="handleCancelOrder">取消
订单</button>
```

```
101         <button type="primary" size="mini" bind:tap="navTo" data-url="
102         {{ '/pages/payment/payment?orderId='+orderId }}">立即支付</button>
103     </block>
104 </view>
105 </view>
106 </view>
```

3. 订单详情页面样式代码：

```
1  /* pages/order-detail/order-detail.wxss */
2
3  .order-detail-container {
4      padding: 26rpx;
5      padding-bottom: 200rpx;
6      .bg-radius {
7          background-color: #fff;
8          border-radius: 18rpx;
9          padding: 30rpx;
10         margin-bottom: 20rpx;
11     }
12 }
13
14 .remaining-time-wrapper {
15     background-color: #e40030;
16     background: linear-gradient(to bottom, #e40030 0%, #e40030 50%, #f9f9f9 50%,
17     #f9f9f9 100%);
18     padding: 26rpx;
19     margin: -26rpx;
20     .remaining-time-text {
21         > text {
22             display: block;
23             &:nth-child(1) {
24                 font-size: 36rpx;
25                 font-weight: bold;
26             }
27             &:nth-child(2) {
28                 font-size: 24rpx;
29                 color: #9d9d9d;
30             }
31         }
32     }
33 }
34
35 .order-goods-wrapper {
36     .goods-info-header {
```

```
37 padding-bottom: 26rpx;
38 border-bottom: var(--basic-border);
39 .header-left {
40     font-size: 34rpx;
41     font-weight: bold;
42     .shop-name {
43         max-width: 460rpx;
44         // width: auto !important;
45     }
46     .iconfont {
47         font-size: 36rpx;
48         margin-right: 8rpx;
49     }
50 }
51 .header-right {
52     .right-item {
53         margin-left: 26rpx;
54         .iconfont {
55             font-size: 44rpx;
56         }
57         .tag-text {
58             font-size: 20rpx;
59             color: #333;
60             white-space: nowrap;
61         }
62     }
63 }
64 }
65 .goods-item {
66     padding: 20rpx 0;
67     .goods-item-img-view {
68         .goods-item-img {
69             width: 100rpx;
70             height: 80rpx;
71         }
72     }
73     .goods-item-content {
74         margin: 0 16rpx;
75         &> text {
76             display: block;
77         }
78         .item-name {
79             font-size: 28rpx;
80             color: #222222;
81             font-weight: 500;
82         }
83         .item-unit {
84             font-size: 20rpx;
85             color: #999;
```



```
86     }
87     .item-num {
88         font-size: 26rpx;
89     }
90 }
91 .goods-item-price {
92     margin-left:auto; // 整体右对齐
93     text-align: right;
94     .sale-price {
95         color: #222;
96         align-items: flex-end;
97         .sale-price-symbol {
98             font-size: 20rpx;
99             line-height: 20rpx;
100        }
101        .sale-price-value {
102            font-weight: 500;
103            font-size: 30rpx;
104            line-height: 28rpx;
105        }
106    }
107    .original-price {
108        color: #828282;
109        text-decoration: line-through;
110        font-size: 22rpx;
111    }
112 }
113 }
114 .goods-total-box {
115     position: relative;
116     border-top: 1px dashed var(--basic-border-color);
117     text-align: right;
118     padding-top: 26rpx;
119     font-size: 28rpx;
120     .total-amount-value {
121         font-weight: bold;
122         margin-left: 4rpx;
123         font-size: 32rpx;
124     }
125     &::before{
126         content: '';
127         width: 40rpx;
128         height: 40rpx;
129         border-radius: 50%;
130         background: #f9f9f9;
131         position: absolute;
132         left: -46rpx;
133         top: -20rpx;
134         z-index: 999;
```

```
135     }
136     &::after{
137         content: '';
138         width: 40rpx;
139         height: 40rpx;
140         border-radius: 50%;
141         background: #f9f9f9;
142         position: absolute;
143         right: -46rpx;
144         top: -20rpx;
145         z-index: 999;
146     }
147 }
148 }
149
150 .order-info-wrapper {
151     .order-header {
152         font-weight: bold;
153         font-size: 38rpx;
154         padding-bottom: 26rpx;
155         border-bottom: var(--basic-border);
156     }
157     .order-row-group {
158         .row-cell {
159             padding: 26rpx 0;
160             font-size: 28rpx;
161             .row-title {
162                 color: #9d9d9d;
163                 width: 200rpx;
164             }
165             .row-value {
166                 flex: 1;
167                 text-align: right;
168                 color: #222;
169             }
170         }
171     }
172 }
173 }
174
175
176 .order-detail-bottom-btns {
177     text-align: right;
178     button {
179         margin-left: 20rpx;
180         &::after { // 去除边框
181             display: none;
182         }
183     }
184 }
```

门店指引-打开地图

在页面 pages/order-detail/order-detail.js 中 定义handleOpenLocation方法中调用wx.openLocation打开地图

```
1 // pages/order-detail/order-detail.js
2 import { hideLoading, msg, navBack, navTo, showLoading } from "@utils/util"
3
4 Page({
5
6   navTo, // 页面跳转，页面中要使用
7
8   // 门店指引：打开地图
9   handleOpenLocation() {
10     const { name, address, location = {} } = this.data.order?.shop
11     // console.log('location', location)
12     const { longitude, latitude } = location
13     if (!longitude || !latitude) {
14       return msg('门店没有地图位置')
15     }
16     wx.openLocation({
17       longitude: location.longitude,
18       latitude: location.latitude,
19       name,
20       address
21     })
22   },
23
24 })
```

取消订单逻辑实现

1. 先提示是否确认取消订单，用户确认后进行更新订单状态：4-已取消

```
1 // pages/order-detail/order-detail.js
2 Page({
3
4   // 取消订单
5   handleCancelOrder () {
6     wx.showModal({
7       title: '取消订单',
8       content: '取消后，需重新选择商品下单',
9       cancelText: '取消订单',
```

```
10      confirmText: '再考虑下',
11      cancelColor: '#333333',
12      confirmColor: '#e40030',
13      success: (res) => {
14        if (res.cancel) {
15          this.updateOrderCancel()
16        }
17      }
18    })
19  },
20
21  // 更新订单状态
22  async updateOrderCancel() {
23    try {
24      showLoading('取消中', {mask: true})
25      const updated = await wx.cloud.database().collection("mxg_order")
26        .doc(this.data.orderId)
27        .update({
28          data: {
29            state: 4, // 1-待支付, 2-已支付, 3-已完成, 4-已取消
30            _updateTime: Date.now()
31          }
32        })
33      msg('订单已取消')
34      navBack()
35    } catch (error) {
36      console.error('订单取消失败', error)
37      msg('订单取消失败')
38    } finally {
39      hideLoading()
40    }
41  },
42
43  })
```

第十二章 订单列表页面开发

订单页面效果如下：



订单列表页面配置

1. app.json 配置订单列表页面，创建订单列表页面文件

```
1 {  
2   "pages": [  
3     "pages/shop-list/shop-list"  
4   ],  
5 }  
6 }
```

2. shop-list.json页面配置开启下拉刷新

```
1 {
2   "navigationBarTitleText": "我的订单",
3   "enablePullDownRefresh": true,
4   "onReachBottomDistance": 300,
5   "usingComponents": {
6     "van-tag": "@vant/weapp/tag/index",
7     "van-empty": "@vant/weapp/empty/index"
8   }
9 }
```

分页查询订单数据

实现下拉刷新查询第1页数据，上拉查询一下页数据

1. 配置开启下拉刷新

```
1 import { getTotalPage, msg, navTo } from "@/utils/util"
2 import { toShoppingPage } from "@/utils/page"
3
4 const db = wx.cloud.database()
5
6 Page({
7
8   /**
9    * 页面的初始数据
10   */
11   data: {
12     pageNum: 1, // 当前页码
13     pageSize: 20, // 每页查询多少条
14     totalPage: 0, // 总页数
15     orderList: [],
16   },
17
18   navTo,
19
20   toShoppingPage, // 跳转点单页面
21
22   onShow() {
23     // 触发下拉刷新，会自动调用 onPullDownRefresh 下拉刷新函数
24     wx.startPullDownRefresh()
25   },
26
27   // 下拉刷新
28   onPullDownRefresh() {
29     this.setData({pageNum: 1})
```

```
30     this.loadOrderList()
31   },
32
33   // 页面上拉触底事件的处理函数
34   onReachBottom() {
35     this.setData({pageNum: this.data.pageNum+1})
36     this.loadOrderList()
37   },
38
39   // 加载订单列表数据
40   async loadOrderList() {
41     try {
42       let { totalPages, pageNum, pageSize, orderList } = this.data
43       // 不是第一次加载数据（页面触底），且待查询页码大于总页码，则不查询
44       if (pageNum !== 1 && pageNum > totalPages) {
45         return
46       }
47
48       // 查询总记录数（当前用户的订单）
49       const { total } = await db.collection('mxg_order').count()
50
51       // 分页查询订单数据（当前用户的订单）
52       const { data: newOrderList } = await db.collection('mxg_order')
53         .limit(pageSize)
54         .skip((pageNum - 1) * pageSize)
55         .orderBy('_createTime', 'desc')
56         .get()
57
58       // 计算总页数
59       totalPages = getTotalPage(total, pageSize)
60
61       // 第1页展示本次查询结果，第2次开始则追加到原数据后面
62       orderList = pageNum == 1 ? newOrderList : orderList.concat(newOrderList)
63     } catch (error) {
64       console.error('查询订单列表失败', error)
65       msg('查询订单失败')
66     } finally {
67       wx.stopPullDownRefresh()
68     }
69   },
70
71   /** 用户点击右上角分享 */
72   onShareAppMessage() {
73     // ...
74   }
75 }
```


订单列表页面效果实现

1. 订单列表页面模板代码：

```
1 <!--pages/order-list/order-list.wxml-->
2
3 <!-- 导入 wxs 模块 -->
4 <wxs src="/wxs/format.wxs" module="format" />
5
6 <!-- 订单列表 -->
7 <view wx:if="{{orderList && orderList.length>0}}" class="order-list-wrapper">
8   <view class="order-item-box" wx:for="{{orderList}}" wx:key="_id"
9     bind:tap="navTo" data-url="{{ '/pages/order-detail/order-detail?
orderId='+item._id }}"
10   >
11     <view class="flex-space-between-center">
12       <view class="flex-1 text-one">
13         <view class="flex-row-align-center">
14           <van-tag class="type-tag" plain type="danger">{{item.takeType == 1 ?
'自取': '外送'}}</van-tag>
15           <text class="shop-name">{{item.shopName}}</text>
16         </view>
17         <view>
18           <text class="order-time">{{format.formatTime(item._createTime)}}
</text>
19         </view>
20       </view>
21       <view class="right-box">
22         <!-- 1-待支付, 2-已支付, 3-已完成, 4-已取消 -->
23         <text class="right-btn-text" {{item.state == 1 ? 'waiting-pay': ''}}>
{{item.state == 1 ? '待支付' : item.state == 2 ? '已支付' : item.state == 3 ?
'已完成': '已取消'}}</text>
24         <text class="iconfont icon-arrow-right"/>
25       </view>
26     </view>
27
28     <!-- 左右滑动 -->
29     <view class="order-item-goods flex-row-align-center">
30       <scroll-view class="scroll-view" scroll-x>
31         <image class="scroll-item"
32           wx:for="{{item.cartList}}"
33           wx:for-item="g" wx:for-index="gIndex" wx:key="gIndex"
34           src="{{g.imageUrl}}"
35         />
36       </scroll-view>
```

```
37     <view class="totol-box">
38         <text class="total-amount">¥{{item.totalAmount}}</text>
39         <text class="total-num">共{{item.totalNum}}件</text>
40     </view>
41 </view>
42 </view>
43 </view>
44
45 <!-- 无订单时，显示 -->
46 <van-empty wx:else description="无订单，快去选一下您喜欢的吧！" >
47     <button bind:tap="toShoppingPage" type="primary">来一单</button>
48 </van-empty>
```

2. 订单列表页面样式代码：

```
1  /* pages/order-list/order-list.wxss */
2
3  .order-list-wrapper {
4      margin: 28rpx;
5      padding-bottom: 50rpx;
6      .order-item-box {
7          background-color: #fff;
8          border-radius: 20rpx;
9          padding: 28rpx;
10         margin-bottom: 20rpx;
11         box-shadow: 0 0 .3rem #e8e8e8;
12         .type-tag {
13             line-height: 26rpx;
14         }
15         .shop-name {
16             font-size: 30rpx;
17             line-height: 30rpx;
18             font-weight: bold;
19             margin-left: 16rpx;
20         }
21         .order-time {
22             color: #999;
23             font-size: 24rpx;
24         }
25         .right-box {
26             width: 180rpx;
27             text-align: right;
28             .right-btn-text, .iconfont {
29                 color: #999;
30                 font-size: 28rpx;
31                 &.waiting-pay {
32                     font-weight: bold;
```

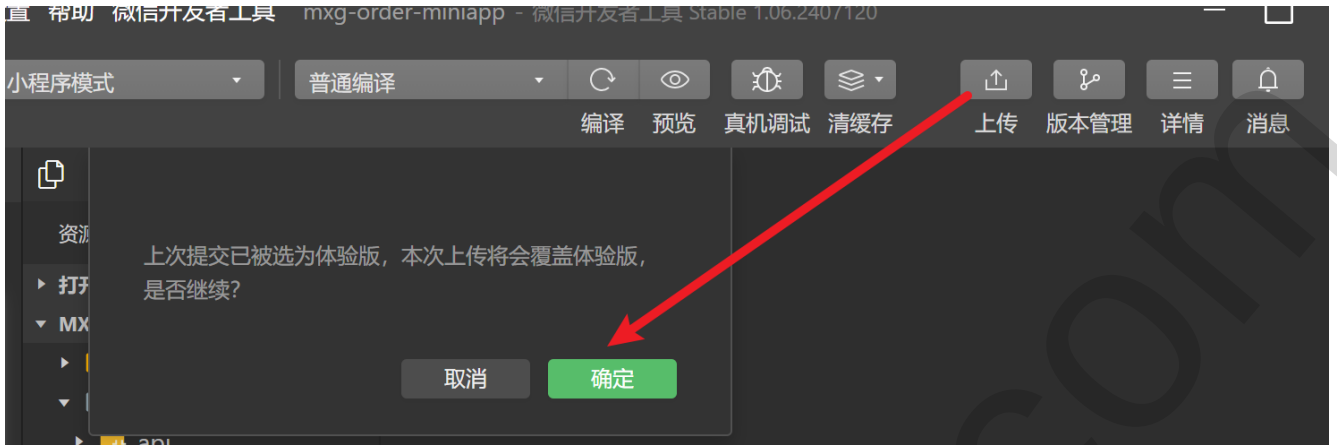
```
33         color: #e40030;
34     }
35 }
36 }
37
38 .order-item-goods {
39     margin: 26rpx 0;
40     .scroll-view {
41         white-space: nowrap;
42         overflow: hidden;
43         .scroll-item {
44             display: inline-block;
45             width: 130rpx;
46             height: 90rpx;
47             border-radius: 6rpx;
48             margin-right: 16rpx;
49         }
50     }
51     .total-box {
52         padding-left: 20rpx;
53         text-align: right;
54         white-space: nowrap;
55         .total-amount {
56             display: block;
57             font-weight: bold;
58             font-size: 32rpx;
59         }
60         .total-num {
61             display: block;
62             color: #999;
63             font-size: 24rpx;
64         }
65     }
66 }
67
68 .opt-btn {
69     margin-top: 26rpx;
70     text-align: right;
71 }
72 }
73
74 }
```

第十三章 微信小程序发布上线

小程序打包上传

1. 将小程序上传到微信小程序服务器：

点击开发工具右上角的【上传】，如下点击【确定】



2. 然后会弹出如下窗口，输入：版本号、项目备注;; 然后点击【上传】



3. 上传中，在【调试器】窗口的【构建】可看到正在编译代码。

打包可能要等待很久（遇到微信服务器问题），多重启开发工具，多试几次



4. 如下代码上传成功

编译 预览 真机调试 清缓存

✓ 代码上传成功

↓ SourceMap 文件

以下 98 文件没有被打包上传：

miniprogram_npm/@vant/weapp/action-sheet/index.d.ts
miniprogram_npm/@vant/weapp/button/index.d.ts
miniprogram_npm/@vant/weapp/area/index.d.ts
miniprogram_npm/@vant/weapp/calendar/index.d.ts
miniprogram_npm/@vant/weapp/calendar/utils.d.ts
miniprogram_npm/@vant/weapp/cascader/index.d.ts
miniprogram_npm/@vant/weapp/card/index.d.ts
miniprogram_npm/@vant/weapp/cell/index.d.ts
miniprogram_npm/@vant/weapp/cell-group/index.d.ts

New 使用多端框架，可快速生成App

去了解

的文件之间切换 Ctrl 1 ~ 9

在文件中查找 Ctrl + Shift + F

切换终端 Ctrl + `

切换面板 Ctrl + J

切换侧边栏可见性 Ctrl + Shift + Alt

构建 300 调试器 问题 输出 终端 代码质量

```
$: 正在：Seal code package
$: 编译所有文件完成，共编译 655 个文件，耗时：18827ms
$: 过滤文件
$: 过滤文件完成，耗时：8ms
$: 正在处理编译结果
$: 处理编译结果完成，耗时：1ms
$: 正在对比文件列表
$: 对比文件列表完成，耗时：440ms
$: 正在打包
$: 打包完成，耗时：11ms
$: 上传中
$: 代码质量检测完成，耗时：2260ms
$: 上传完成，耗时：2378ms
```

5. 打包成功访问 <https://mp.weixin.qq.com/> 使用小程序管理员微信扫码登录小程序管理后台
6. 登录后，点击 管理》版本管理》提交审核

首页

管理

版本管理

成员管理

开发管理

用户反馈

付费管理

微信认证

数据

基础功能

体验评价

客服

小程序插件

订阅消息

审核版本

你暂无提交审核的版本或者版本已发布上线

开发版本

版本号

1.0.0

[体验版](#)

开发者

提交时间

项目备注

客浪

2024-11-18 11:32:52

点餐小程序 客浪 在 2024年11月18日中午11点32分 提交上传

提交审核



7. 点击【去备案】去进行小程序备案，也可以点击【继续提交】

小程序备案提醒



经查，你的小程序还未履行备案手续，请尽快提交备案申请，平台将帮助你完成备案，以避免逾期未备案被相关主管部门处置，影响小程序正常运营。

去备案

继续提交

设置主营类目，具体操作查看下一节



无法提交审核

请先添加小程序服务类目，并设置**主营类目**后，再提交审核

去设置

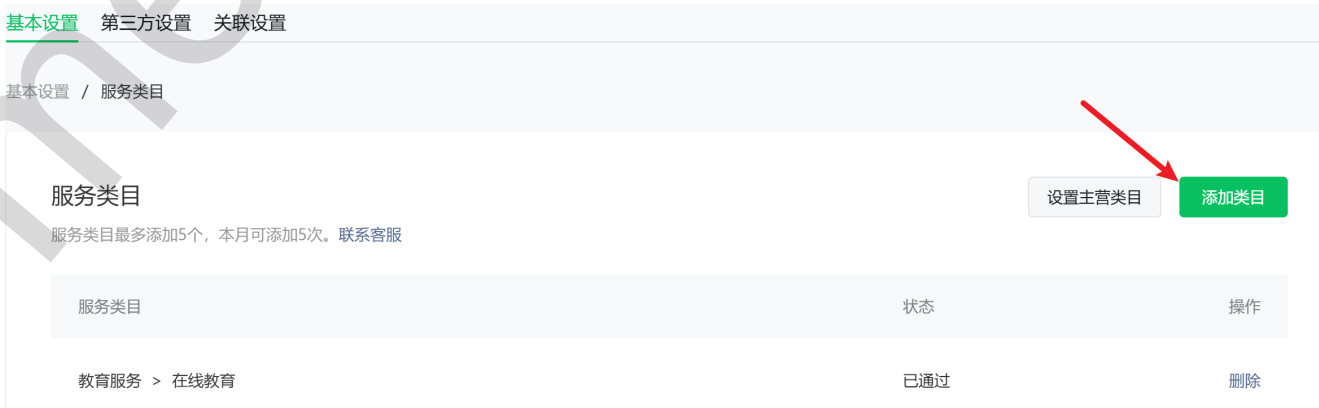
取消

添加服务分类

1. 如下图，找到服务分类，点击详情



2. 点击【详情】，进入后【添加类目】，添加：生活服务 > 家政服务



3. 再设置主营类目：生活服务 > 家政服务

设置

基本设置 第三方设置 关联设置

基本设置 / 服务类目

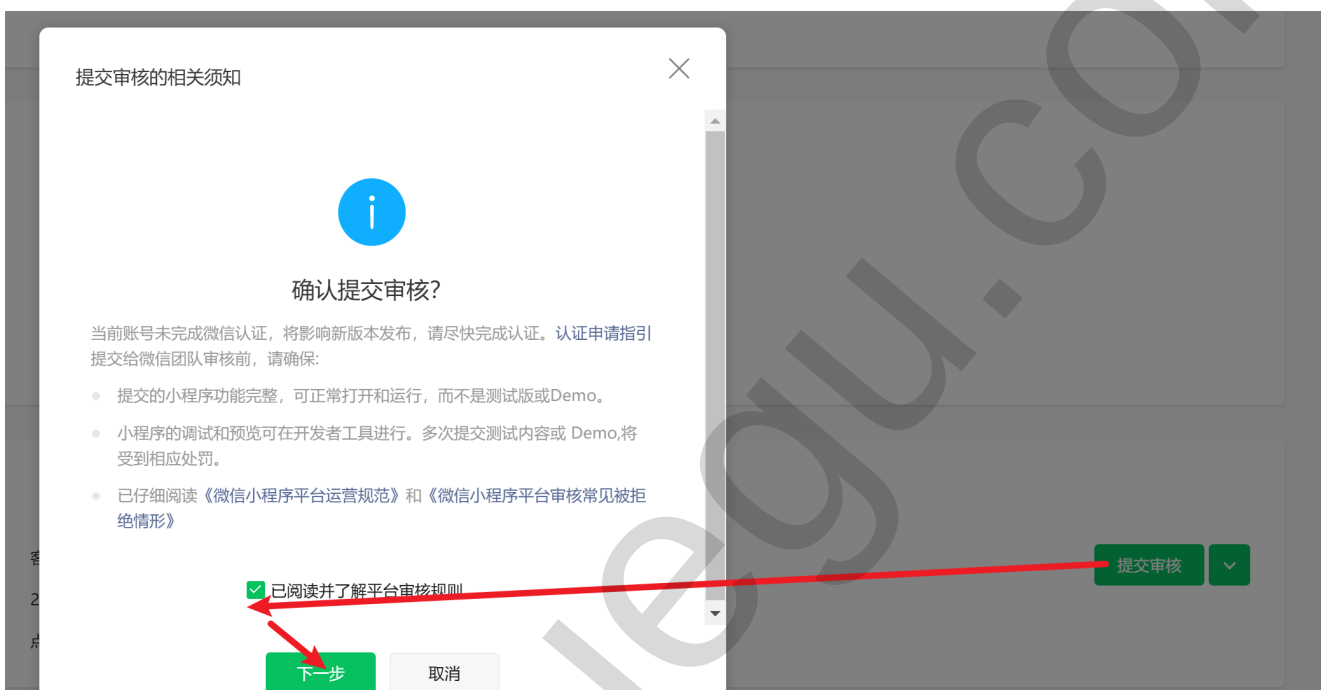
服务类目

服务类目最多添加5个，本月可添加4次。联系客服

设置主营类目

添加类目

4. 上面设置主营类目后，重新回到 版本管理页面， 进行刷新页面的，再重新提交审核



5. 提示接口无权限，则点击去申请，具体操作查看下一节



接口无权限

app.json文件中部分接口暂无权限，请删除接口，或前往「微信公众平台 > 开发管理 > 接口设置」申请相关权限。

暂无权限接口: wx.chooseLocation、wx.getLocation

去申请

取消

接口权限申请开通

1. 找到如下 wx.getLocation 和 wx.chooseLocation 进行申请开通，

有些类目不支持开通 wx.getLocation 权限，则需要选择支持的类目（如：生活服务 > 家政服务）



2. 开通 wx.getLocation 权限

申请接口理由：因当前业务涉及到搜索附近门店服务，申请接口以使用户可以找到离他位置最近的门店进行下单

使用场景截图：如下 点单页和附近门店的页面图

当前接口为 "wx.getLocation"

申请案例参考 为避免申请不通过请查看[申请成功案例参考](#)

申请接口理由 详细描述每个接口能力的具体使用场景，以及接口在小程序内业务场景使用的必要性。

因当前业务涉及到搜索附近门店服务，申请接口以使用户可以找到离他位置最近的门店进行下单

42/300

使用场景截图 可上传小程序截图辅助审核，最多上传10张图片。图片支持jpg、jpeg、bmp、gif或png格式,每张图片大小不超过5M。



使用场景视频 (选填) 为提高审核通过率，可补充小程序内调用地理位置信息等核心能力的录屏视频。最多上传1个视频。视频支持mp4格式，视频大小不超过20M。

上传视频

小程序官网链接 (选填) 例如<https://www.tencent.com>

+ 添加新网址

提交

3. 开通 wx.chooseLocation 权限

申请接口理由：因当前业务涉及到搜索附近门店服务，申请接口以使用户可以找到离他位置最近的门店进行下单

使用场景截图：如下 点单页和附近门店的页面图

当前接口为 "wx.chooseLocation"

申请案例参考 为避免申请不通过请查看申请成功案例参考

申请接口理由 详细描述每个接口能力的具体使用场景，以及接口在小程序内业务场景使用的必要性。

因当前业务涉及到搜索附近门店服务，申请接口以使用户可以找到离他位置最近的门店进行下单

42/300

使用场景截图 可上传小程序截图辅助审核，最多上传10张图片。图片支持jpg、jpeg、bmp、gif或png格式，每张图片大小不超过5M。



使用场景视频 (选填) 为提高审核通过率，可补充小程序内调用地理位置信息等核心能力的录屏视频。最多上传1个视频。视频支持mp4格式，视频大小不超过20M。

上传视频

小程序官网链接 (选填) 例如https://www.tencent.com

⊕ 添加新网址

提交